api_reference/accounting.md

Accounting

api_reference/adapters/betfair.md

Betfair

Client

Common

Config

Data

Data Types

Execution

Factories

OrderBook

Providers

Sockets

api_reference/adapters/binance.md

Binance

Config

Factories

Enums

Types

Futures

Data

Enums

Execution

Providers

Types

Spot

Data

Enums

Execution

Providers

api_reference/adapters/bybit.md

Bybit

Config

Factories

Enums

Providers

Data

Execution

api_reference/adapters/coinbase_intx.md

Coinbase International

Config

Constants

Factories

Providers

Data

Execution

api_reference/adapters/databento.md

Databento

Config

Factories

Enums

Types

Loaders

Providers

Data

api_reference/adapters/dydx.md

dYdX

Config

Factories

Enums

Providers

Data

Execution

api_reference/adapters/index.md

Adapters

api_reference/adapters/interactive_brokers.md

Interactive Brokers

Common

Config

Data

Execution

Factories

Providers

api_reference/adapters/okx.md

OKX

Config

Factories

Enums

Providers

Data

Execution

api_reference/adapters/polymarket.md

Polymarket

Config

Factories

Enums

Providers

Data

Execution

api_reference/adapters/tardis.md

Tardis

Loaders

Config

Providers

Factories

Data

api_reference/analysis.md

Analysis

api_reference/backtest.md

Backtest

api_reference/cache.md

Cache

api_reference/common.md

Common

Component

Executor

Generators

api_reference/config.md

Config

Backtest

Cache

Common

Data

Execution

Live

Persistence

Risk

System

Trading

api_reference/core.md

Core

Datetime

Finite-State Machine (FSM)

Message

Stats

UUID

api_reference/data.md

Data

Aggregation

Client

Engine

Messages

api_reference/execution.md

Execution

Components

Messages

Reports

api_reference/index.md

Python API

Welcome to the Python API reference for NautilusTrader!

The API reference provides detailed technical documentation for the NautilusTrader framework, including its modules, classes, methods, and functions. The reference is automatically generated from the latest NautilusTrader source code using [Sphinx](https://www.sphinx-doc.org/en/master/).

Please note that there are separate references for different versions of NautilusTrader:

- Latest: This API reference is built from the head of the develop branch and represents the latest stable release.
- Nightly: This API reference is built from the head of the nightly branch and represents bleeding edge and experimental changes/features currently in development.

You can select the desired API reference from the Versions top left drop down menu.

Use the right navigation sidebar to explore the available modules and their contents.
You can click on any item to view its detailed documentation, including parameter descriptions, and return value explanations.

Why Python?

Python was originally created decades ago as a simple scripting language with a clean straight forward syntax. It has since evolved into a fully fledged general purpose object-oriented programming language. Based on the TIOBE index, Python is currently the most popular programming language in the world.
Not only that, Python has become the de facto lingua franca of data science, machine learning, and artificial intelligence.

The language out of the box is not without its drawbacks however, especially in the context of implementing large performance-critical systems. Cython has addressed a lot of these issues, offering all the advantages
of a statically typed language, embedded into Python's rich ecosystem of software libraries and developer/user communities.

api_reference/indicators.md

Indicators

api_reference/live.md

Live

api_reference/model/book.md

Order Book

api_reference/model/data.md

Data

api_reference/model/events.md

Events

api_reference/model/identifiers.md

Identifiers

api_reference/model/index.md

Model

api_reference/model/instruments.md

Instruments

api_reference/model/objects.md

Objects

api_reference/model/orders.md

Orders

api_reference/model/position.md

Position

api_reference/model/tick_scheme.md

Tick Scheme

api_reference/persistence.md

Persistence

api_reference/portfolio.md

Portfolio

api_reference/risk.md

Risk

api_reference/serialization.md

Serialization

api_reference/system.md

System

api_reference/trading.md

Trading

concepts/actors.md

Actors

:::info
We are currently working on this concept guide.
:::

The Actor serves as the foundational component for interacting with the trading system.
It provides core functionality for receiving market data, handling events, and managing state within the trading environment. The Strategy class inherits from Actor and extends its capabilities with order management methods.

Key capabilities:

- Event subscription and handling
- Market data reception
- State management
- System interaction primitives

Basic example

Just like strategies, actors support configuration through a very similar pattern.

Data handling and callbacks

When working with data in Nautilus, it's important to understand the relationship between data requests/subscriptions and their corresponding callback handlers. The system uses different handlers depending on whether the data is historical or real-time.

Historical vs real-time data

The system distinguishes between two types of data flow:

1. Historical data (from requests):
   - Obtained through methods like requestbars(), requestquoteticks(), etc.
   - Processed through the onhistoricaldata() handler.
   - Used for initial data loading and historical analysis.

2. Real-time data (from subscriptions):
   - Obtained through methods like subscribebars(), subscribequoteticks(), etc.
   - Processed through specific handlers like onbar(), onquotetick(), etc.
   - Used for live data processing.

Callback handlers

Here's how different data operations map to their handlers:

Operation	Category	Handler	Purpose
subscribedata()	Real-time	ondata()	Live data updates
subscribeinstrument()	Real-time	oninstrument()	Live instrument definition updates
subscribeinstruments()	Real-time	oninstrument()	Live instrument definition updates (for venue)
subscribeorderbookdeltas()	Real-time	onorderbookdeltas()	Live order book updates
subscribequoteticks()	Real-time	onquotetick()	Live quote updates
subscribetradeticks()	Real-time	ontradetick()	Live trade updates

subscribemarkprices()	Real-time	onmarkprice()	Live mark price updates
subscribeindexprices()	Real-time	onindexprice()	Live index price updates
subscribebars()	Real-time	onbar()	Live bar updates
subscribeinstrumentstatus()	Real-time	oninstrumentstatus()	Live instrument status updates
subscribeinstrumentclose()	Real-time	oninstrumentclose()	Live instrument close updates
requestdata()	Historical	onhistoricaldata()	Historical data processing
requestinstrument()	Historical	oninstrument()	Instrument definition updates
requestinstruments()	Historical	oninstrument()	Instrument definition updates
requestquoteticks()	Historical	onhistoricaldata()	Historical quotes processing
requesttradeticks()	Historical	onhistoricaldata()	Historical trades processing
requestbars()	Historical	onhistoricaldata()	Historical bars processing
requestaggregatedbars()	Historical	onhistoricaldata()	Historical aggregated bars (on-the-fly)

Example

Here's an example demonstrating both historical and real-time data handling:

This separation between historical and real-time data handlers allows for different processing logic based on the data context. For example, you might want to:

- Use historical data to initialize indicators or establish baseline metrics.
- Process real-time data differently for live trading decisions.
- Apply different validation or logging for historical vs real-time data.

:::tip
When debugging data flow issues, check that you're looking at the correct handler for your data source. If you're not seeing data in onbar() but see log messages about receiving bars, check onhistoricaldata() as the data might be coming from a request rather than a subscription.
:::

concepts/adapters.md

Adapters

The NautilusTrader design integrates data providers and/or trading venues
through adapter implementations. These can be found in the top-level adapters subpackage.

An integration adapter is typically comprised of the following main components:

- HttpClient
- WebSocketClient
- InstrumentProvider
- DataClient
- ExecutionClient

Instrument providers

Instrument providers do as their name suggests - instantiating Nautilus
Instrument objects by parsing the raw API of the publisher or venue.

The use cases for the instruments available from an InstrumentProvider are either:

- Used standalone to discover the instruments available for an integration, using these for research or backtesting purposes
- Used in a sandbox or live [environment context](architecture.md#environment-contexts) for consumption by actors/strategies

Research and backtesting

Here is an example of discovering the current instruments for the Binance Futures testnet:

Live trading

Each integration is implementation specific, and there are generally two options for the behavior of an InstrumentProvider within a TradingNode for live trading,
as configured:

- All instruments are automatically loaded on start:

- Only those instruments explicitly specified in the configuration are loaded on start:

Data clients

Requests

An Actor or Strategy can request custom data from a DataClient by sending a DataRequest. If the client that receives the
DataRequest implements a handler for the request, data will be returned to the Actor or Strategy.

Example

An example of this is a DataRequest for an Instrument, which the Actor class implements (copied below). Any Actor or
Strategy can call a requestinstrument method with an InstrumentId to request the instrument from a DataClient.

In this particular case, the Actor implements a separate method requestinstrument. A similar type of
DataRequest could be instantiated and called from anywhere and/or anytime in the actor/strategy code.

A simplified version of requestinstrument for an actor/strategy is:

A simplified version of the request handler implemented in a LiveMarketDataClient that will retrieve the data
and send it back to actors/strategies is for example:

The DataEngine which is an important component in Nautilus links a request with a DataClient.
For example a simplified version of handling an instrument request is:

concepts/architecture.md

Architecture

Welcome to the architectural overview of NautilusTrader.

This guide dives deep into the foundational principles, structures, and designs that underpin the platform. Whether you're a developer, system architect, or just curious about the inner workings of NautilusTrader, this section covers:

- The design philosophy that drives decisions and shapes the system's evolution.
- The overarching system architecture providing a bird's-eye view of the entire system framework.
- How the framework is organized to facilitate modularity and maintainability.
- The code structure that ensures readability and scalability.
- A breakdown of component organization and interaction to understand how different parts communicate and collaborate.
- And finally, the implementation techniques that are crucial for performance, reliability, and robustness.

:::note
Throughout the documentation, the term "Nautilus system boundary" refers to operations within the runtime of a single Nautilus node (also known as a "trader instance").
:::

Design Philosophy

The major architectural techniques and design patterns employed by NautilusTrader are:

- [Domain driven design (DDD)](https://en.wikipedia.org/wiki/Domain-drivendesign)
- [Event-driven architecture](https://en.wikipedia.org/wiki/Event-drivenprogramming)
- [Messaging patterns](https://en.wikipedia.org/wiki/Messagingpattern) (Pub/Sub, Req/Rep, point-to-point)
- [Ports and adapters](https://en.wikipedia.org/wiki/Hexagonalarchitecture(software))
- [Crash-only design](https://en.wikipedia.org/wiki/Crash-onlysoftware)

These techniques have been utilized to assist in achieving certain architectural quality attributes.

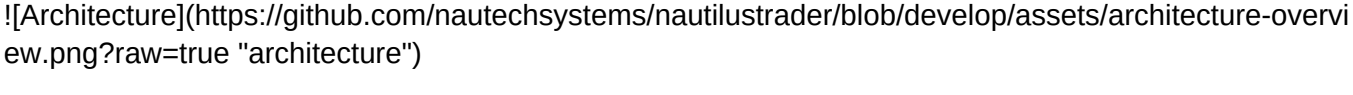
Quality Attributes

Architectural decisions are often a trade-off between competing priorities. The below is a list of some of the most important quality attributes which are considered when making design and architectural decisions, roughly in order of 'weighting'.

- Reliability
- Performance
- Modularity
- Testability
- Maintainability
- Deployability

System Architecture

The NautilusTrader codebase is actually both a framework for composing trading systems, and a set of default system implementations which can operate in various [environment contexts](#environment-contexts).



Core Components

The platform is built around several key components that work together to provide a comprehensive trading system:

NautilusKernel

The central orchestration component responsible for:

- Initializing and managing all system components.
- Configuring the messaging infrastructure.
- Maintaining environment-specific behaviors.
- Coordinating shared resources and lifecycle management.
- Providing a unified entry point for system operations.

MessageBus

The backbone of inter-component communication, implementing:

- Publish/Subscribe patterns: For broadcasting events and data to multiple consumers.
- Request/Response communication: For operations requiring acknowledgment.
- Command/Event messaging: For triggering actions and notifying state changes.
- Optional state persistence: Using Redis for durability and restart capabilities.

Cache

High-performance in-memory storage system that:

- Stores instruments, accounts, orders, positions, and more.
- Provides performant fetching capabilities for trading components.
- Maintains consist state across the system.
- Supports both read and write operations with optimized access patterns.

DataEngine

Processes and routes market data throughout the system:

- Handles multiple data types (quotes, trades, bars, order books, custom data, and more).
- Routes data to appropriate consumers based on subscriptions.
- Manages data flow from external sources to internal components.

ExecutionEngine

Manages order lifecycle and execution:

- Routes trading commands to the appropriate adapter clients.
- Tracks order and position states.
- Coordinates with risk management systems.
- Handles execution reports and fills from venues.
- Handles reconciliation of external execution state.

RiskEngine

Provides comprehensive risk management:

- Pre-trade risk checks and validation.
- Position and exposure monitoring.
- Real-time risk calculations.
- Configurable risk rules and limits.

Environment Contexts

An environment context in NautilusTrader defines the type of data and trading venue you are working with. Understanding these contexts is crucial for effective backtesting, development, and live trading.

Here are the available environments you can work with:

- Backtest: Historical data with simulated venues.
- Sandbox: Real-time data with simulated venues.
- Live: Real-time data with live venues (paper trading or real accounts).

Common Core

The platform has been designed to share as much common code between backtest, sandbox and live trading systems as possible.
This is formalized in the system subpackage, where you will find the NautilusKernel class,
providing a common core system 'kernel'.

The ports and adapters architectural style enables modular components to be integrated into the core system, providing various hooks for user-defined or custom component implementations.

Data and Execution Flow Patterns

Understanding how data and execution flow through the system is crucial for effective use of the platform:

Data Flow Pattern

1. External Data Ingestion: Market data enters via venue-specific DataClient adapters where it is normalized.
2. Data Processing: The DataEngine handles data processing for internal components.
3. Caching: Processed data is stored in the high-performance Cache for fast access.
4. Event Publishing: Data events are published to the MessageBus.
5. Consumer Delivery: Subscribed components (Actors, Strategies) receive relevant data events.

Execution Flow Pattern

1. Command Generation: User strategies create trading commands.
2. Command Publishing: Commands are sent through the MessageBus.
3. Risk Validation: The RiskEngine validates trading commands against configured risk rules.
4. Execution Routing: The ExecutionEngine routes commands to appropriate venues.
5. External Submission: The ExecutionClient submits orders to external trading venues.
6. Event Flow Back: Order events (fills, cancellations) flow back through the system.
7. State Updates: Portfolio and position states are updated based on execution events.

Component State Management

All components follow a finite state machine pattern with well-defined states:

- PREINITIALIZED: Component is created but not yet wired up to the system.
- READY: Component is configured and wired up, but not yet running.
- RUNNING: Component is actively processing messages and performing operations.
- STOPPED: Component has been gracefully stopped and is no longer processing.
- DEGRADED: Component is running but with reduced functionality due to errors.
- FAULTED: Component has encountered a critical error and cannot continue.
- DISPOSED: Component has been cleaned up and resources have been released.

Messaging

To facilitate modularity and loose coupling, an extremely efficient MessageBus passes messages (data, commands and events) between components.

From a high level architectural view, it's important to understand that the platform has been designed to run efficiently
on a single thread, for both backtesting and live trading. Much research and testing
resulted in arriving at this design, as it was found the overhead of context switching between threads didn't actually result in improved performance.

When considering the logic of how your algo trading will work within the system boundary, you can expect each component to consume messages
in a deterministic synchronous way (similar to the [actor model](https://en.wikipedia.org/wiki/Actormodel)).

:::note
Of interest is the LMAX exchange architecture, which achieves award winning performance running on a single thread. You can read about their disruptor pattern based architecture in [this interesting article](https://martinfowler.com/articles/lmax.html) by Martin Fowler.
:::

Framework organization

The codebase is organized with a layering of abstraction levels, and generally
grouped into logical subpackages of cohesive concepts. You can navigate to the documentation
for each of these subpackages from the left nav menu.

Core / low-Level

- core: Constants, functions and low-level components used throughout the framework.
- common: Common parts for assembling the frameworks various components.
- network: Low-level base components for networking clients.
- serialization: Serialization base components and serializer implementations.
- model: Defines a rich trading domain model.

Components

- accounting: Different account types and account management machinery.
- adapters: Integration adapters for the platform including brokers and exchanges.
- analysis: Components relating to trading performance statistics and analysis.
- cache: Provides common caching infrastructure.
- data: The data stack and data tooling for the platform.
- execution: The execution stack for the platform.
- indicators: A set of efficient indicators and analyzers.
- persistence: Data storage, cataloging and retrieval, mainly to support backtesting.
- portfolio: Portfolio management functionality.
- risk: Risk specific components and tooling.
- trading: Trading domain specific components and tooling.

System implementations

- backtest: Backtesting componentry as well as a backtest engine and node implementations.
- live: Live engine and client implementations as well as a node for live trading.
- system: The core system kernel common between backtest, sandbox, live [environment contexts](#environment-contexts).

Code structure

The foundation of the codebase is the crates directory, containing a collection of core Rust crates including a C foreign function interface (FFI) generated by cbindgen.

The bulk of the production code resides in the nautilustrader directory, which contains a collection of

Python/Cython subpackages and modules.

Python bindings for the Rust core are provided by statically linking the Rust libraries to the C extension modules generated by Cython at compile time (effectively extending the CPython API).

Dependency flow

:::note
Both Rust and Cython are build dependencies. The binary wheels produced from a build do not require Rust or Cython to be installed at runtime.
:::

Type safety

The design of the platform prioritizes software correctness and safety at the highest level.

The Rust codebase in nautiluscore is always type safe and memory safe as guaranteed by the rustc compiler,
and so is correct by construction (unless explicitly marked unsafe, see the Rust section of the [Developer Guide](../developerguide/rust.md)).

Cython provides type safety at the C level at both compile time, and runtime:

:::info
If you pass an argument with an invalid type to a Cython implemented module with typed parameters, then you will receive a TypeError at runtime.
:::

If a function or method's parameter is not explicitly typed to accept None, passing None as an argument will result in a ValueError at runtime.

:::warning
The above exceptions are not explicitly documented to prevent excessive bloating of the docstrings.
:::

Errors and exceptions

Every attempt has been made to accurately document the possible exceptions which can be raised from NautilusTrader code, and the conditions which will trigger them.

:::warning
There may be other undocumented exceptions which can be raised by Pythons standard library, or from third party library dependencies.
:::

Processes and threads

:::tip
For optimal performance and to prevent potential issues related to Python's memory model and equality, it is highly recommended to run each trader instance in a separate process.
:::

concepts/backtesting.md

Backtesting
Backtesting with NautilusTrader is a methodical simulation process that replicates trading
activities using a specific system implementation. This system is composed of various components
including the built-in engines, Cache, [MessageBus](messagebus.md), Portfolio, [Actors](actors.md), [Strategies](strategies.md), [Execution Algorithms](execution.md),
and other user-defined modules. The entire trading simulation is predicated on a stream of historical data processed by a
BacktestEngine. Once this data stream is exhausted, the engine concludes its operation, producing
detailed results and performance metrics for in-depth analysis.

It's important to recognize that NautilusTrader offers two distinct API levels for setting up and conducting backtests:

- High-level API: Uses a BacktestNode and configuration objects (BacktestEngines are used internally).
- Low-level API: Uses a BacktestEngine directly with more "manual" setup.

Choosing an API level

Consider using the low-level API when:

- Your entire data stream can be processed within the available machine resources (e.g., RAM).
- You prefer not to store data in the Nautilus-specific Parquet format.
- You have a specific need or preference to retain raw data in its original format (e.g., CSV, binary, etc.).
- You require fine-grained control over the BacktestEngine, such as the ability to re-run backtests on identical datasets while swapping out components (e.g., actors or strategies) or adjusting parameter configurations.

Consider using the high-level API when:

- Your data stream exceeds available memory, requiring streaming data in batches.
- You want to leverage the performance and convenience of the ParquetDataCatalog for storing data in the Nautilus-specific Parquet format.
- You value the flexibility and functionality of passing configuration objects to define and manage multiple backtest runs across various engines simultaneously.

Low-level API

The low-level API centers around a BacktestEngine, where inputs are initialized and added manually via a Python script.
An instantiated BacktestEngine can accept the following:

- Lists of Data objects, which are automatically sorted into monotonic order based on tsinit.
- Multiple venues, manually initialized.
- Multiple actors, manually initialized and added.
- Multiple execution algorithms, manually initialized and added.

This approach offers detailed control over the backtesting process, allowing you to manually configure each component.

High-level API

The high-level API centers around a BacktestNode, which orchestrates the management of multiple BacktestEngine instances,
each defined by a BacktestRunConfig. Multiple configurations can be bundled into a list and processed

by the node in one run.

Each BacktestRunConfig object consists of the following:

- A list of BacktestDataConfig objects.
- A list of BacktestVenueConfig objects.
- A list of ImportableActorConfig objects.
- A list of ImportableStrategyConfig objects.
- A list of ImportableExecAlgorithmConfig objects.
- An optional ImportableControllerConfig object.
- An optional BacktestEngineConfig object, with a default configuration if not specified.

Data

Data provided for backtesting drives the execution flow. Since a variety of data types can be used, it's crucial that your venue configurations align with the data being provided for backtesting. Mismatches between data and configuration can lead to unexpected behavior during execution.

NautilusTrader is primarily designed and optimized for order book data, which provides a complete representation of every price level or order in the market, reflecting the real-time behavior of a trading venue.
This ensures the highest level of execution granularity and realism. However, if granular order book data is either not available or necessary, then the platform has the capability of processing market data in the following descending order of detail:

1. Order Book Data/Deltas (L3 market-by-order):
   - Providing comprehensive market depth and detailed order flow, with visibility of all individual orders.

2. Order Book Data/Deltas (L2 market-by-price):
   - Providing market depth visibility across all price levels.

3. Quote Ticks (L1 market-by-price):
   - Representing the "top of the book" by capturing only the best bid and ask prices and sizes.

4. Trade Ticks:
   - Reflecting actual executed trades, offering a precise view of transaction activity.

5. Bars:
   - Aggregating trading activity - typically over fixed time intervals, such as 1-minute, 1-hour, or 1-day.

Choosing data: Cost vs. Accuracy

For many trading strategies, bar data (e.g., 1-minute) can be sufficient for backtesting and strategy development. This is particularly important because bar data is typically much more accessible and cost-effective compared to tick or order book data.

Given this practical reality, Nautilus is designed to support bar-based backtesting with advanced features that maximize simulation accuracy, even when working with lower granularity data.

:::tip
For some trading strategies, it can be practical to start development with bar data to validate core trading ideas.
If the strategy looks promising, but is more sensitive to precise execution timing (e.g., requires fills at specific prices between OHLC levels, or uses tight take-profit/stop-loss levels), you can then invest in higher granularity data

for more accurate validation.
:::

Venues

When initializing a venue for backtesting, you must specify its internal order booktype for execution processing from the following options:

- L1MBP: Level 1 market-by-price (default). Only the top level of the order book is maintained.
- L2MBP: Level 2 market-by-price. Order book depth is maintained, with a single order aggregated per price level.
- L3MBO: Level 3 market-by-order. Order book depth is maintained, with all individual orders tracked as provided by the data.

:::note
The granularity of the data must match the specified order booktype. Nautilus cannot generate higher granularity data (L2 or L3) from lower-level data such as quotes, trades, or bars.
:::

:::warning
If you specify L2MBP or L3MBO as the venue's booktype, all non-order book data (such as quotes, trades, and bars) will be ignored for execution processing.
This may cause orders to appear as though they are never filled. We are actively working on improved validation logic to prevent configuration and data mismatches.
:::

:::warning
When providing L2 or higher order book data, ensure that the booktype is updated to reflect the data's granularity.
Failing to do so will result in data aggregation: L2 data will be reduced to a single order per level, and L1 data will reflect only top-of-book levels.
:::

Execution

Bar Based Execution

Bar data provides a summary of market activity with four key prices for each time period (assuming bars are aggregated by trades):

- Open: opening price (first trade)
- High: highest price traded
- Low: lowest price traded
- Close: closing price (last trade)

While this gives us an overview of price movement, we lose some important details that we'd have with more granular data:

- We don't know in what order the market hit the high and low prices.
- We can't see exactly when prices changed within the time period.
- We don't know the actual sequence of trades that occurred.

This is why Nautilus processes bar data through a sophisticated system that attempts to maintain the most realistic yet conservative market behavior possible, despite these limitations.
At its core, the platform always maintains an order book simulation - even when you provide less granular data such as quotes, trades, or bars (although the simulation will only have a top level book).

Processing Bar Data

Even when you provide bar data, Nautilus maintains an internal order book for each instrument - just like a real venue would.

1. Time Processing:
   - Nautilus has a specific way of handling the timing of bar data for execution that's crucial for accurate simulation.
   - Bar timestamps (tsevent) are expected to represent the close time of the bar. This approach is most logical because it represents the moment when the bar is fully formed and its aggregation is complete.
   - The initialization time (tsinit) can be controlled using the tsinitdelta parameter in BarDataWrangler, which should typically be set to the bar's step size (timeframe) in nanoseconds.
   - The platform ensures all events happen in the correct sequence based on these timestamps, preventing any possibility of look-ahead bias in your backtests.

2. Price Processing:
   - The platform converts each bar's OHLC prices into a sequence of market updates.
   - These updates always follow the same order: Open High Low Close.
   - If you provide multiple timeframes (like both 1-minute and 5-minute bars), the platform uses the more granular data for highest accuracy.

3. Executions:
   - When you place orders, they interact with the simulated order book just like they would on a real venue.
   - For MARKET orders, execution happens at the current simulated market price plus any configured latency.
   - For LIMIT orders working in the market, they'll execute if any of the bar's prices reach or cross your limit price (see below).
   - The matching engine continuously processes orders as OHLC prices move, rather than waiting for complete bars.

OHLC Prices Simulation

During backtest execution, each bar is converted into a sequence of four price points:

1. Opening price
2. High price (Order between High/Low is configurable. See baradaptivehighlowordering below.)
3. Low price
4. Closing price

The trading volume for that bar is split evenly among these four points (25% each). In marginal cases, if the original bar's volume divided by 4 is less than the instrument's minimum sizeincrement, we still use the minimum sizeincrement per price point to ensure valid market activity (e.g., 1 contract for CME group exchanges).

How these price points are sequenced can be controlled via the baradaptivehighlowordering parameter when configuring a venue.

Nautilus supports two modes of bar processing:

1. Fixed Ordering (baradaptivehighlowordering=False, default)
   - Processes every bar in a fixed sequence: Open High Low Close.
   - Simple and deterministic approach.

2. Adaptive Ordering (baradaptivehighlowordering=True)
   - Uses bar structure to estimate likely price path:
     - If Open is closer to High: processes as Open High Low Close.
     - If Open is closer to Low: processes as Open Low High Close.
     - [Research](https://gist.github.com/stefansimik/d387e1d9ff784a8973feca0cde51e363) shows this

approach achieves ~75-85% accuracy in predicting correct High/Low sequence (compared to statistical ~50% accuracy with fixed ordering).
  - This is particularly important when both take-profit and stop-loss levels occur within the same bar - as the sequence determines which order is filled first.

Here's how to configure adaptive bar ordering for a venue, including account setup:

Slippage and Spread Handling

When backtesting with different types of data, Nautilus implements specific handling for slippage and spread simulation:

For L2 (market-by-price) or L3 (market-by-order) data, slippage is simulated with high accuracy by:

- Filling orders against actual order book levels.
- Matching available size at each price level sequentially.
- Maintaining realistic order book depth impact (per order fill).

For L1 data types (e.g., L1 order book, trades, quotes, bars), slippage is handled through:

Initial fill slippage (probslippage):

- Controlled by the probslippage parameter of the FillModel.
- Determines if the initial fill will occur one tick away from current market price.
- Example: With probslippage=0.5, a market BUY has 50% chance of filling one tick above the best ask price.

:::note
When backtesting with bar data, be aware that the reduced granularity of price information affects the slippage mechanism.
For the most realistic backtesting results, consider using higher granularity data sources such as L2 or L3 order book data when available.
:::

Fill Model

The FillModel helps simulate order queue position and execution in a simple probabilistic way during backtesting.
It addresses a fundamental challenge: even with perfect historical market data, we can't fully simulate how orders may have interacted with other
market participants in real-time.

The FillModel simulates two key aspects of trading that exist in real markets regardless of data quality:

1. Queue Position for Limit Orders:
  - When multiple traders place orders at the same price level, the order's position in the queue affects if and when it gets filled.

2. Market Impact and Competition:
  - When taking liquidity with market orders, you compete with other traders for available liquidity, which can affect your fill price.

Configuration and Parameters

probfillonlimit (default: 1.0)

- Purpose:
  - Simulates the probability of a limit order getting filled when its price level is reached in the market.
- Details:
  - Simulates your position in the order queue at a given price level.

  - Applies to all data types (e.g., L1/L2/L3 order book, quotes, trades, bars).
  - New random probability check occurs each time market price touches your order price (but does not move through it).
  - On successful probability check, fills entire remaining order quantity.

Examples:

- With probfillonlimit=0.0:
  - Limit BUY orders never fill when best ask reaches the limit price.
  - Limit SELL orders never fill when best bid reaches the limit price.
  - This simulates being at the very back of the queue and never reaching the front.
- With probfillonlimit=0.5:
  - Limit BUY orders have 50% chance of filling when best ask reaches the limit price.
  - Limit SELL orders have 50% chance of filling when best bid reaches the limit price.
  - This simulates being in the middle of the queue.
- With probfillonlimit=1.0 (default):
  - Limit BUY orders always fill when best ask reaches the limit price.
  - Limit SELL orders always fill when best bid reaches the limit price.
  - This simulates being at the front of the queue with guaranteed fills.

probslippage (default: 0.0)

- Purpose:
  - Simulates the probability of experiencing price slippage when executing market orders.
- Details:
  - Only applies to L1 data types (e.g., quotes, trades, bars).
  - When triggered, moves fill price one tick against your order direction.
    - Affects all market-type orders (MARKET, MARKETTOLIMIT, MARKETIFTOUCHED, STOPMARKET).
  - Not utilized with L2/L3 data where order book depth can determine slippage.

Examples:

- With probslippage=0.0 (default):
  - No artificial slippage is applied, representing an idealized scenario where you always get filled at the current market price.
- With probslippage=0.5:
  - Market BUY orders have 50% chance of filling one tick above the best ask price, and 50% chance at the best ask price.
  - Market SELL orders have 50% chance of filling one tick below the best bid price, and 50% chance at the best bid price.
- With probslippage=1.0:
  - Market BUY orders always fill one tick above the best ask price.
  - Market SELL orders always fill one tick below the best bid price.
  - This simulates consistent adverse price movement against your orders.

probfillonstop (default: 1.0)

- Stop order is just shorter name for stop-market order, that convert to market orders when market-price touches the stop-price.
- That means, stop order order-fill mechanics is simply market-order mechanics, that is controlled by the probslippage parameter.

:::warning
The probfillonstop parameter is deprecated and will be removed in a future version (use probslippage instead).
:::

How Simulation Varies by Data Type

The behavior of the FillModel adapts based on the order book type being used:

L2/L3 Orderbook data

With full order book depth, the FillModel focuses purely on simulating queue position for limit orders through probfillonlimit.
The order book itself handles slippage naturally based on available liquidity at each price level.

- probfillonlimit is active - simulates queue position.
- probslippage is not used - real order book depth determines price impact.

L1 Orderbook data

With only best bid/ask prices available, the FillModel provides additional simulation:

- probfillonlimit is active - simulates queue position.
- probslippage is active - simulates basic price impact since we lack real depth information.

Bar/Quote/Trade data

When using less granular data, the same behaviors apply as L1:

- probfillonlimit is active - simulates queue position.
- probslippage is active - simulates basic price impact.

Important Considerations

The FillModel has certain limitations to keep in mind:

- Partial fills are supported with L2/L3 order book data - when there is no longer any size available in the order book, no more fills will be generated and the order will remain in a partially filled state. This accurately simulates real market conditions where not enough liquidity is available at the desired price levels.
- With L1 data, slippage is limited to a fixed 1-tick, at which entire order's quantity is filled.

:::note
As the FillModel continues to evolve, future versions may introduce more sophisticated simulation of order execution dynamics, including:

- Partial fill simulation
- Variable slippage based on order size
- More complex queue position modeling

:::

Account Types

When you attach a venue to the engine-either for live trading or a backtest-you must pick one of three accounting modes by passing the accounttype parameter:

Account type	Typical use-case	What the engine locks
Cash	Spot trading (e.g. BTC/USDT, stocks)	Notional value for every position a pending order would open.
Margin	Derivatives or any product that allows leverage	Initial margin for each order plus maintenance margin for open positions.
Betting	Sports betting, bookmaking	Stake required by the venue; no

leverage. |

Example of adding a CASH account for a backtest venue:

Cash Accounts

Cash accounts settle trades in full; there is no leverage and therefore no concept of margin.

Margin Accounts

A margin account facilitates trading of instruments requiring margin, such as futures or leveraged products.
It tracks account balances, calculates required margins, and manages leverage to ensure sufficient collateral for positions and orders.

Key Concepts:

- Leverage: Amplifies trading exposure relative to account equity. Higher leverage increases potential returns and risks.
- Initial Margin: Collateral required to submit an order to open a position.
- Maintenance Margin: Minimum collateral required to maintain an open position.
- Locked Balance: Funds reserved as collateral, unavailable for new orders or withdrawals.

:::note
Reduce-only orders do not contribute to balancelocked in cash accounts,
nor do they add to initial margin in margin accounts-as they can only reduce existing exposure.
:::

Betting Accounts

Betting accounts are specialised for venues where you stake an amount to win or lose a fixed payout (some prediction markets, sports books, etc.).
The engine locks only the stake required by the venue; leverage and margin are not applicable.

concepts/cache.md

Cache

The Cache is a central in-memory database that automatically stores and manages all trading-related data.
Think of it as your trading system's memory - storing everything from market data to order history to custom calculations.

The Cache serves multiple key purposes:

1. Stores market data:
   - Stores recent market history (e.g., order books, quotes, trades, bars).
   - Gives you access to both current and historical market data for your strategy.

2. Tracks trading data:
   - Maintains complete Order history and current execution state.
   - Tracks all Positions and Account information.
   - Stores Instrument definitions and Currency information.

3. Stores custom data:
   - Any user-defined objects or data can be stored in the Cache for later use.
   - Enables data sharing between different strategies.

How Cache works

Built-in types:

- Data is automatically added to the Cache as it flows through the system.
- In live contexts, updates happen asynchronously - which means there might be a small delay between an event occurring and it appearing in the Cache.
- All data flows through the Cache before reaching your strategy's callbacks - see the diagram below:

Basic example

Within a strategy, you can access the Cache through self.cache. Here's a typical example:

:::note
Anywhere you find self, it refers mostly to the Strategy itself.
:::

Configuration

The Cache's behavior and capacity can be configured through the CacheConfig class.
You can provide this configuration either to a BacktestEngine or a TradingNode, depending on your [environment context](architecture.md#environment-contexts).

Here's a basic example of configuring the Cache:

:::tip
By default, the Cache keeps the last 10,000 bars for each bar type and 10,000 trade ticks per instrument.
These limits provide a good balance between memory usage and data availability. Increase them if your strategy needs more historical data.
:::

Configuration Options

The CacheConfig class supports these parameters:

:::note
Each bar type maintains its own separate capacity. For example, if you're using both 1-minute and 5-minute bars, each will store up to barcapacity bars.
When barcapacity is reached, the oldest data is automatically removed from the Cache.
:::

Database Configuration

For persistence between system restarts, you can configure a database backend.

When is it useful to use persistence?

- Long-running systems: If you want your data to survive system restarts, upgrading, or unexpected failures, having a database configuration helps to pick up exactly where you left off.
- Historical insights: When you need to preserve past trading data for detailed post-analysis or audits.
- Multi-node or distributed setups: If multiple services or nodes need to access the same state, a persistent store helps ensure shared and consistent data.

Using the Cache

Accessing Market data

The Cache provides a comprehensive interface for accessing different types of market data, including order books, quotes, trades, bars.
All market data in the cache are stored with reverse indexing - meaning the most recent data is at index 0.

Bar(s) access

Quote ticks

Trade ticks

Order Book

Price access

Bar types

Simple example

Trading Objects

The Cache provides comprehensive access to all trading objects within the system, including:

- Orders
- Positions
- Accounts
- Instruments

Orders

Orders can be accessed and queried through multiple methods, with flexible filtering options by venue, strategy, instrument, and order side.

Basic Order Access

Order State Queries

Order Statistics

Positions

The Cache maintains a record of all positions and offers several ways to query them.

Position Access

Position State Queries

Position Statistics

Accounts

Instruments and Currencies

Instruments

Currencies

Custom Data

The Cache can also store and retrieve custom data types in addition to built-in market data and trading objects.
You can keep any user-defined data you want to share between system components (mostly Actors / Strategies).

Basic Storage and Retrieval

For more complex use cases, the Cache can store custom data objects that inherit from the nautilustrader.core.Data base class.

:::warning
The Cache is not designed to be a full database replacement. For large datasets or complex querying needs, consider using a dedicated database system.
:::

Best Practices and Common Questions

Cache vs. Portfolio Usage

The Cache and Portfolio components serve different but complementary purposes in NautilusTrader:

Cache:

- Maintains the historical knowledge and current state of the trading system.
- Updates immediately for local state changes (initializing an order to be submitted)
- Updates asynchronously as external events occur (order is filled).
- Provides complete history of trading activity and market data.
- All data a strategy has received (events/updates) is stored in Cache.

Portfolio:

- Aggregated position/exposure and account information.
- Provides current state without history.

Example:

Cache vs. Strategy variables

Choosing between storing data in the Cache versus strategy variables depends on your specific needs:

Cache Storage:

- Use for data that needs to be shared between strategies.

- Best for data that needs to persist between system restarts.
- Acts as a central database accessible to all components.
- Ideal for state that needs to survive strategy resets.

Strategy Variables:

- Use for strategy-specific calculations.
- Better for temporary values and intermediate results.
- Provides faster access and better encapsulation.
- Best for data that only your strategy needs.

Example:

Example that clarifies how you might store data in the Cache so multiple strategies can access the same information.

How another strategy (running in parallel) can retrieve cached data above:

concepts/data.md

Data

The NautilusTrader platform provides a set of built-in data types specifically designed to represent a trading domain.
These data types include:

- OrderBookDelta (L1/L2/L3): Represents the most granular order book updates.
- OrderBookDeltas (L1/L2/L3): Batches multiple order book deltas for more efficient processing.
- OrderBookDepth10: Aggregated order book snapshot (up to 10 levels per bid and ask side).
- QuoteTick: Represents the best bid and ask prices along with their sizes at the top-of-book.
- TradeTick: A single trade/match event between counterparties.
- Bar: OHLCV (Open, High, Low, Close, Volume) bar/candle, aggregated using a specified aggregation method.
- InstrumentStatus: An instrument-level status event.
- InstrumentClose: The closing price of an instrument.

NautilusTrader is designed primarily to operate on granular order book data, providing the highest realism
for execution simulations in backtesting.
However, backtests can also be conducted on any of the supported market data types, depending on the desired simulation fidelity.

Order books

A high-performance order book implemented in Rust is available to maintain order book state based on provided data.

OrderBook instances are maintained per instrument for both backtesting and live trading, with the following book types available:

- L3MBO: Market by order (MBO) or L3 data, uses every order book event at every price level, keyed by order ID.
- L2MBP: Market by price (MBP) or L2 data, aggregates order book events by price level.
- L1MBP: Market by price (MBP) or L1 data, also known as best bid and offer (BBO), captures only top-level updates.

:::note
Top-of-book data, such as QuoteTick, TradeTick and Bar, can also be used for backtesting, with markets operating on L1MBP book types.
:::

Instruments

The following instrument definitions are available:

- Betting: Represents an instrument in a betting market.
- BinaryOption: Represents a generic binary option instrument.
- Cfd: Represents a Contract for Difference (CFD) instrument.
- Commodity: Represents a commodity instrument in a spot/cash market.
- CryptoFuture: Represents a deliverable futures contract instrument, with crypto assets as underlying and for settlement.
- CryptoPerpetual: Represents a crypto perpetual futures contract instrument (a.k.a. perpetual swap).
- CurrencyPair: Represents a generic currency pair instrument in a spot/cash market.
- Equity: Represents a generic equity instrument.
- FuturesContract: Represents a generic deliverable futures contract instrument.

- FuturesSpread: Represents a generic deliverable futures spread instrument.
- Index: Represents a generic index instrument.
- OptionContract: Represents a generic option contract instrument.
- OptionSpread: Represents a generic option spread instrument.
- Synthetic: Represents a synthetic instrument with prices derived from component instruments using a formula.

Bars and aggregation

Introduction to Bars

A bar (also known as a candle, candlestick or kline) is a data structure that represents price and volume information over a specific period, including:

- Opening price
- Highest price
- Lowest price
- Closing price
- Traded volume (or ticks as a volume proxy)

These bars are generated using an aggregation method, which groups data based on specific criteria.

Purpose of data aggregation

Data aggregation in NautilusTrader transforms granular market data into structured bars or candles for several reasons:

- To provide data for technical indicators and strategy development.
- Because time-aggregated data (like minute bars) are often sufficient for many strategies.
- To reduce costs compared to high-frequency L1/L2/L3 market data.

Aggregation methods

The platform implements various aggregation methods:

Name	Description	Category
TICK	Aggregation of a number of ticks.	Threshold
TICKIMBALANCE	Aggregation of the buy/sell imbalance of ticks.	Threshold
TICKRUNS	Aggregation of sequential buy/sell runs of ticks.	Information
VOLUME	Aggregation of traded volume.	Threshold
VOLUMEIMBALANCE	Aggregation of the buy/sell imbalance of traded volume.	Threshold
VOLUMERUNS	Aggregation of sequential runs of buy/sell traded volume.	Information
VALUE	Aggregation of the notional value of trades (also known as "Dollar bars").	Threshold
VALUEIMBALANCE	Aggregation of the buy/sell imbalance of trading by notional value.	Information
VALUERUNS	Aggregation of sequential buy/sell runs of trading by notional value.	Threshold
MILLISECOND	Aggregation of time intervals with millisecond granularity.	Time
SECOND	Aggregation of time intervals with second granularity.	Time
MINUTE	Aggregation of time intervals with minute granularity.	Time
HOUR	Aggregation of time intervals with hour granularity.	Time
DAY	Aggregation of time intervals with day granularity.	Time
WEEK	Aggregation of time intervals with week granularity.	Time

| MONTH | Aggregation of time intervals with month granularity. | Time |

Types of aggregation

NautilusTrader implements three distinct data aggregation methods:

1. Trade-to-bar aggregation: Creates bars from TradeTick objects (executed trades)
   - Use case: For strategies analyzing execution prices or when working directly with trade data.
   - Always uses the LAST price type in the bar specification.

2. Quote-to-bar aggregation: Creates bars from QuoteTick objects (bid/ask prices)
   - Use case: For strategies focusing on bid/ask spreads or market depth analysis.
   - Uses BID, ASK, or MID price types in the bar specification.

3. Bar-to-bar aggregation: Creates larger-timeframe Bar objects from smaller-timeframe Bar objects
   - Use case: For resampling existing smaller timeframe bars (1-minute) into larger timeframes (5-minute, hourly).
   - Always requires the @ symbol in the specification.

Bar types and Components

NautilusTrader defines a unique bar type (BarType class) based on the following components:

- Instrument ID (InstrumentId): Specifies the particular instrument for the bar.
- Bar Specification (BarSpecification):
  - step: Defines the interval or frequency of each bar.
  - aggregation: Specifies the method used for data aggregation (see the above table).
  - pricetype: Indicates the price basis of the bar (e.g., bid, ask, mid, last).
- Aggregation Source (AggregationSource): Indicates whether the bar was aggregated internally (within Nautilus)
- or externally (by a trading venue or data provider).

Bar types can also be classified as either standard or composite:

- Standard: Generated from granular market data, such as quote-ticks or trade-ticks.
- Composite: Derived from a higher-granularity bar type through subsampling (like 5-MINUTE bars aggregate from 1-MINUTE bars).

Aggregation sources

Bar data aggregation can be either internal or external:

- INTERNAL: The bar is aggregated inside the local Nautilus system boundary.
- EXTERNAL: The bar is aggregated outside the local Nautilus system boundary (typically by a trading venue or data provider).

For bar-to-bar aggregation, the target bar type is always INTERNAL (since you're doing the aggregation within NautilusTrader),
but the source bars can be either INTERNAL or EXTERNAL, i.e., you can aggregate externally provided bars or already
aggregated internal bars.

Defining Bar Types with String Syntax

Standard bars

You can define standard bar types from strings using the following convention:

{instrumentid}-{step}-{aggregation}-{pricetype}-{INTERNAL | EXTERNAL}

For example, to define a BarType for AAPL trades (last price) on Nasdaq (XNAS) using a 5-minute

interval
aggregated from trades locally by Nautilus:

Composite bars

Composite bars are derived by aggregating higher-granularity bars into the desired bar type. To define a composite bar,
use this convention:

{instrumentid}-{step}-{aggregation}-{pricetype}-INTERNAL@{step}-{aggregation}-{INTERNAL | EXTERNAL}

Notes:

- The derived bar type must use an INTERNAL aggregation source (since this is how the bar is aggregated).
- The sampled bar type must have a higher granularity than the derived bar type.
- The sampled instrument ID is inferred to match that of the derived bar type.
- Composite bars can be aggregated from INTERNAL or EXTERNAL aggregation sources.

For example, to define a BarType for AAPL trades (last price) on Nasdaq (XNAS) using a 5-minute interval
aggregated locally by Nautilus, from 1-minute interval bars aggregated externally:

Aggregation syntax examples

The BarType string format encodes both the target bar type and, optionally, the source data type:

The part after the @ symbol is optional and only used for bar-to-bar aggregation:

- Without @: Aggregates from TradeTick objects (when pricetype is LAST) or QuoteTick objects (when pricetype is BID, ASK, or MID).
- With @: Aggregates from existing Bar objects (specifying the source bar type).

Trade-to-bar example

Quote-to-bar example

Bar-to-bar example

Advanced bar-to-bar example

You can create complex aggregation chains where you aggregate from already aggregated bars:

Working with Bars: Request vs. Subscribe

NautilusTrader provides two distinct operations for working with bars:

- requestbars(): Fetches historical data processed by the onhistoricaldata() handler.
- subscribebars(): Establishes a real-time data feed processed by the onbar() handler.

These methods work together in a typical workflow:

1. First, requestbars() loads historical data to initialize indicators or state of strategy with past market behavior.
2. Then, subscribebars() ensures the strategy continues receiving new bars as they form in real-time.

Example usage in onstart():

Required handlers in your strategy to receive the data:

Historical data requests with aggregation

When requesting historical bars for backtesting or initializing indicators, you can use the requestbars() method, which supports both direct requests and aggregation:

If historical aggregated bars are needed, you can use specialized request requestaggregatedbars() method:

Common Pitfalls

Register indicators before requesting data: Ensure indicators are registered before requesting historical data so they get updated properly.

Timestamps

The platform uses two fundamental timestamp fields that appear across many objects, including market data, orders, and events.
These timestamps serve distinct purposes and help maintain precise timing information throughout the system:

- tsevent: UNIX timestamp (nanoseconds) representing when an event actually occurred.
- tsinit: UNIX timestamp (nanoseconds) representing when Nautilus created the internal object representing that event.

Examples

Event Type	tsevent	tsinit
TradeTick	Time when trade occurred at the exchange.	Time when Nautilus received the trade data.
QuoteTick	Time when quote occurred at the exchange.	Time when Nautilus received the quote data.
OrderBookDelta	Time when order book update occurred at the exchange.	Time when Nautilus received the order book update.
Bar	Time of the bar's closing (exact minute/hour).	Time when Nautilus generated (for internal bars) or received the bar data (for external bars).
OrderFilled	Time when order was filled at the exchange.	Time when Nautilus received and processed the fill confirmation.
OrderCanceled	Time when cancellation was processed at the exchange.	Time when Nautilus received and processed the cancellation confirmation.
NewsEvent	Time when the news was published.	Time when the event object was created (if internal event) or received (if external event) in Nautilus.
Custom event	Time when event conditions actually occurred.	Time when the event object was created (if internal event) or received (if external event) in Nautilus.

:::note
The tsinit field represents a more general concept than just the "time of reception" for events.
It denotes the timestamp when an object, such as a data point or command, was initialized within Nautilus.
This distinction is important because tsinit is not exclusive to "received events" - it applies to any internal
initialization process.

For example, the tsinit field is also used for commands, where the concept of reception does not apply.
This broader definition ensures consistent handling of initialization timestamps across various object types in the system.
:::

Latency analysis

The dual timestamp system enables latency analysis within the platform:

- Latency can be calculated as tsinit - tsevent.
- This difference represents total system latency, including network transmission time, processing overhead, and any queueing delays.
- It's important to remember that the clocks producing these timestamps are likely not synchronized.

Environment specific behavior

Backtesting environment

- Data is ordered by tsinit using a stable sort.
- This behavior ensures deterministic processing order and simulates realistic system behavior, including latencies.

Live trading environment

- Data is processed as it arrives, ensuring minimal latency and allowing for real-time decision-making.
  - tsinit field records the exact moment when data is received by Nautilus in real-time.
  - tsevent reflects the time the event occurred externally, enabling accurate comparisons between external event timing and system reception.
- We can use the difference between tsinit and tsevent to detect network or processing delays.

Other notes and considerations

- For data from external sources, tsinit is always the same as or later than tsevent.
- For data created within Nautilus, tsinit and tsevent can be the same because the object is initialized at the same time the event happens.
- Not every type with a tsinit field necessarily has a tsevent field. This reflects cases where:
  - The initialization of an object happens at the same time as the event itself.
  - The concept of an external event time does not apply.

Persisted Data

The tsinit field indicates when the message was originally received.

Data flow

The platform ensures consistency by flowing data through the same pathways across all system [environment contexts](/concepts/architecture.md#environment-contexts)
(e.g., backtest, sandbox, live). Data is primarily transported via the MessageBus to the DataEngine and then distributed to subscribed or registered handlers.

For users who need more flexibility, the platform also supports the creation of custom data types.
For details on how to implement user-defined data types, see the [Custom Data](#custom-data) section below.

Loading data

NautilusTrader facilitates data loading and conversion for three main use cases:

- Providing data for a BacktestEngine to run backtests.
- Persisting the Nautilus-specific Parquet format for the data catalog via ParquetDataCatalog.writedata(...) to be later used with a BacktestNode.
- For research purposes (to ensure data is consistent between research and backtesting).

Regardless of the destination, the process remains the same: converting diverse external data formats into Nautilus data structures.

To achieve this, two main components are necessary:

- A type of DataLoader (normally specific per raw source/format) which can read the data and return a pd.DataFrame with the correct schema for the desired Nautilus object
- A type of DataWrangler (specific per data type) which takes this pd.DataFrame and returns a list[Data] of Nautilus objects

Data loaders

Data loader components are typically specific for the raw source/format and per integration. For instance, Binance order book data is stored in its raw CSV file form with
an entirely different format to [Databento Binary Encoding (DBN)](https://databento.com/docs/knowledge-base/new-users/dbn-encoding/getting-started-with-dbn) files.

Data wranglers

Data wranglers are implemented per specific Nautilus data type, and can be found in the nautilustrader.persistence.wranglers module.
Currently there exists:

- OrderBookDeltaDataWrangler
- QuoteTickDataWrangler
- TradeTickDataWrangler
- BarDataWrangler

:::warning
There are a number of DataWrangler v2 components, which will take a pd.DataFrame typically
with a different fixed width Nautilus Arrow v2 schema, and output PyO3 Nautilus objects which are only compatible with the new version
of the Nautilus core, currently in development.

These PyO3 provided data objects are not compatible where the legacy Cython objects are currently used (e.g., adding directly to a BacktestEngine).
:::

Transformation pipeline

Process flow:

1. Raw data (e.g., CSV) is input into the pipeline.
2. DataLoader processes the raw data and converts it into a pd.DataFrame.
3. DataWrangler further processes the pd.DataFrame to generate a list of Nautilus objects.
4. The Nautilus list[Data] is the output of the data loading process.

The following diagram illustrates how raw data is transformed into Nautilus data structures:

Concretely, this would involve:

- BinanceOrderBookDeltaDataLoader.load(...) which reads CSV files provided by Binance from disk, and returns a pd.DataFrame.
- OrderBookDeltaDataWrangler.process(...) which takes the pd.DataFrame and returns list[OrderBookDelta].

The following example shows how to accomplish the above in Python:

Data catalog

The data catalog is a central store for Nautilus data, persisted in the [Parquet](https://parquet.apache.org) file format.

We have chosen Parquet as the storage format for the following reasons:

- It performs much better than CSV/JSON/HDF5/etc in terms of compression ratio (storage size) and read performance.
- It does not require any separate running components (for example a database).
- It is quick and simple to get up and running with.

The Arrow schemas used for the Parquet format are either single sourced in the core persistence Rust crate, or available
from the /serialization/arrow/schema.py module.

:::note
2023-10-14: The current plan is to eventually phase out the Python schemas module, so that all schemas are single sourced in the Rust core.
:::

Initializing

The data catalog can be initialized from a NAUTILUSPATH environment variable, or by explicitly passing in a path like object.

The following example shows how to initialize a data catalog where there is pre-existing data already written to disk at the given path.

Writing data

New data can be stored in the catalog, which is effectively writing the given data to disk in the Nautilus-specific Parquet format.
All Nautilus built-in Data objects are supported, and any data which inherits from Data can be written.

The following example shows the above list of Binance OrderBookDelta objects being written:

Basename template

Nautilus makes no assumptions about how data may be partitioned between files for a particular
data type and instrument ID.

The basenametemplate keyword argument is an additional optional naming component for the output files.
The template should include placeholders that will be filled in with actual values at runtime.
These values can be automatically derived from the data or provided as additional keyword arguments.

For example, using a basename template like "{date}" for AUD/USD.SIM quote tick data,
and assuming "date" is a provided or derivable field, could result in a filename like
"2023-01-01.parquet" under the "quotetick/audusd.sim/" catalog directory.
If not provided, a default naming scheme will be applied. This parameter should be specified as a keyword argument, like writedata(data, basenametemplate="{date}").

:::warning
Any data which already exists under a filename will be overwritten.
If a basenametemplate is not provided, then its very likely existing data for the data type and instrument ID will
be overwritten. To prevent data loss, ensure that the basenametemplate (or the default naming scheme)
generates unique filenames for different data sets.
:::

Rust Arrow schema implementations are available for the follow data types (enhanced performance):

- OrderBookDelta
- QuoteTick

- TradeTick
- Bar

:::warning
By default any data which already exists under a filename will be overwritten.

You can use one of the following write mode with catalog.writedata:

- CatalogWriteMode.OVERWRITE
- CatalogWriteMode.APPEND
- CatalogWriteMode.PREPEND
- CatalogWriteMode.NEWFILE, which will create a file name of the form part-{i}.parquet where i is an integer starting at 0.

:::

Reading data

Any stored data can then be read back into memory:

Streaming data

When running backtests in streaming mode with a BacktestNode, the data catalog can be used to stream the data in batches.

The following example shows how to achieve this by initializing a BacktestDataConfig configuration object:

This configuration object can then be passed into a BacktestRunConfig and then in turn passed into a BacktestNode as part of a run.
See the [Backtest (high-level API)](../gettingstarted/backtesthighlevel.md) tutorial for further details.

Data migrations

NautilusTrader defines an internal data format specified in the nautilusmodel crate.
These models are serialized into Arrow record batches and written to Parquet files.
Nautilus backtesting is most efficient when using these Nautilus-format Parquet files.

However, migrating the data model between [precision modes](../gettingstarted/installation.md#precision-mode) and schema changes can be challenging.
This guide explains how to handle data migrations using our utility tools.

Migration tools

The nautiluspersistence crate provides two key utilities:

tojson

Converts Parquet files to JSON while preserving metadata:

- Creates two files:

- <input>.json: Contains the deserialized data
  - <input>.metadata.json: Contains schema metadata and row group configuration

- Automatically detects data type from filename:

- OrderBookDelta (contains "deltas" or "orderbookdelta")
  - QuoteTick (contains "quotes" or "quotetick")
  - TradeTick (contains "trades" or "tradetick")
  - Bar (contains "bars")

toparquet

Converts JSON back to Parquet format:

- Reads both the data JSON and metadata JSON files
- Preserves row group sizes from original metadata
- Uses ZSTD compression
- Creates <input>.parquet

Migration Process

The following migration examples both use trades data (you can also migrate the other data types in the same way).
All commands should be run from the root of the persistence crate directory.

Migrating from standard-precision (64-bit) to high-precision (128-bit)

This example describes a scenario where you want to migrate from standard-precision schema to high-precision schema.

:::note
If you're migrating from a catalog that used the Int64 and UInt64 Arrow data types for prices and sizes, be sure to check out commit [e284162](https://github.com/nautechsystems/nautilustrader/commit/e284162cf27a3222115aeb5d10d599c8cf09cf50)
before compiling the code that writes the initial JSON.
:::

1. Convert from standard-precision Parquet to JSON:

This will create trades.json and trades.metadata.json files.

2. Convert from JSON to high-precision Parquet:

Add the --features high-precision flag to write data as high-precision (128-bit) schema Parquet.

This will create a trades.parquet file with high-precision schema data.

Migrating schema changes

This example describes a scenario where you want to migrate from one schema version to another.

1. Convert from old schema Parquet to JSON:

Add the --features high-precision flag if the source data uses a high-precision (128-bit) schema.

This will create trades.json and trades.metadata.json files.

2. Switch to new schema version:

3. Convert from JSON back to new schema Parquet:

This will create a trades.parquet file with the new schema.

Best Practices

- Always test migrations with a small dataset first.
- Maintain backups of original files.
- Verify data integrity after migration.
- Perform migrations in a staging environment before applying them to production data.

Custom Data

Due to the modular nature of the Nautilus design, it is possible to set up systems with very flexible data streams, including custom user-defined data types. This guide covers some possible use cases for this functionality.

It's possible to create custom data types within the Nautilus system. First you will need to define your data by subclassing from Data.

:::info
As Data holds no state, it is not strictly necessary to call super().init().
:::

The Data abstract base class acts as a contract within the system and requires two properties for all types of data: tsevent and tsinit. These represent the UNIX nanosecond timestamps for when the event occurred and when the object was initialized, respectively.

The recommended approach to satisfy the contract is to assign tsevent and tsinit to backing fields, and then implement the @property for each as shown above (for completeness, the docstrings are copied from the Data base class).

:::info
These timestamps enable Nautilus to correctly order data streams for backtests using monotonically increasing tsinit UNIX nanoseconds.
:::

We can now work with this data type for backtesting and live trading. For instance, we could now create an adapter which is able to parse and create objects of this type - and send them back to the DataEngine for consumption by subscribers.

You can publish a custom data type within your actor/strategy using the message bus in the following way:

The metadata dictionary optionally adds more granular information that is used in the topic name to publish data with the message bus.

Extra metadata information can also be passed to a BacktestDataConfig configuration object in order to enrich and describe custom data objects used in a backtesting context:

You can subscribe to custom data types within your actor/strategy in the following way:

The clientid provides an identifier to route the data subscription to a specific client.

This will result in your actor/strategy passing these received MyDataPoint objects to your ondata method. You will need to check the type, as this method acts as a flexible handler for all custom data.

Publishing and receiving signal data

Here is an example of publishing and receiving signal data using the MessageBus from an actor or strategy.
A signal is an automatically generated custom data identified by a name containing only one value of a basic type
(str, float, int, bool or bytes).

Option Greeks example

This example demonstrates how to create a custom data type for option Greeks, specifically the delta. By following these steps, you can create custom data types, subscribe to them, publish them, and store them in the Cache or ParquetDataCatalog for efficient retrieval.

Publishing and receiving data

Here is an example of publishing and receiving data using the MessageBus from an actor or strategy:

Writing and reading data using the cache

Here is an example of writing and reading data using the Cache from an actor or strategy:

Writing and reading data using a catalog

For streaming custom data to feather files or writing it to parquet files in a catalog
(registerarrow needs to be used):

Creating a custom data class automatically

The @customdataclass decorator enables the creation of a custom data class with default
implementations for all the features described above.

Each method can also be overridden if needed. Here is an example of its usage:

Custom data type stub

To enhance development convenience and improve code suggestions in your IDE, you can create a .pyi
stub file with the proper constructor signature for your custom data types as well as type hints for attributes.
This is particularly useful when the constructor is dynamically generated at runtime, as it allows the IDE to recognize
and provide suggestions for the class's methods and attributes.

For instance, if you have a custom data class defined in greeks.py, you can create a corresponding greeks.pyi file
with the following constructor signature:

concepts/execution.md

Execution

NautilusTrader can handle trade execution and order management for multiple strategies and venues simultaneously (per instance). Several interacting components are involved in execution, making it crucial to understand the possible flows of execution messages (commands and events).

The main execution-related components include:

- Strategy
- ExecAlgorithm (execution algorithms)
- OrderEmulator
- RiskEngine
- ExecutionEngine or LiveExecutionEngine
- ExecutionClient or LiveExecutionClient

Execution flow

The Strategy base class inherits from Actor and so contains all of the common data related methods. It also provides methods for managing orders and trade execution:

- submitorder(...)
- submitorderlist(...)
- modifyorder(...)
- cancelorder(...)
- cancelorders(...)
- cancelallorders(...)
- closeposition(...)
- closeallpositions(...)
- queryorder(...)

These methods create the necessary execution commands under the hood and send them on the message
bus to the relevant components (point-to-point), as well as publishing any events (such as the initialization of new orders i.e. OrderInitialized events).

The general execution flow looks like the following (each arrow indicates movement across the message bus):

Strategy -> OrderEmulator -> ExecAlgorithm -> RiskEngine -> ExecutionEngine -> ExecutionClient

The OrderEmulator and ExecAlgorithm(s) components are optional in the flow, depending on individual order parameters (as explained below).

This diagram illustrates message flow (commands and events) across the Nautilus execution components.

Order Management System (OMS)

An order management system (OMS) type refers to the method used for assigning orders to positions and tracking those positions for an instrument.
OMS types apply to both strategies and venues (simulated and real). Even if a venue doesn't explicitly state the method in use, an OMS type is always in effect. The OMS type for a component can be specified
using the OmsType enum.

The OmsType enum has three variants:

- UNSPECIFIED: The OMS type defaults based on where it is applied (details below)
- NETTING: Positions are combined into a single position per instrument ID
- HEDGING: Multiple positions per instrument ID are supported (both long and short)

The table below describes different configuration combinations and their applicable scenarios.
When the strategy and venue OMS types differ, the ExecutionEngine handles this by overriding or assigning positionid values for received OrderFilled events.
A "virtual position" refers to a position ID that exists within the Nautilus system but not on the venue in reality.

Strategy OMS	Venue OMS	Description
NETTING	NETTING	The strategy uses the venues native OMS type, with a single position ID per instrument ID.
HEDGING	HEDGING	The strategy uses the venues native OMS type, with multiple position IDs per instrument ID (both LONG and SHORT).
NETTING	HEDGING	The strategy overrides the venues native OMS type. The venue tracks multiple positions per instrument ID, but Nautilus maintains a single position ID.
HEDGING	NETTING	The strategy overrides the venues native OMS type. The venue tracks a single position per instrument ID, but Nautilus maintains multiple position IDs.

:::note
Configuring OMS types separately for strategies and venues increases platform complexity but allows for a wide range of trading styles and preferences (see below).
:::

OMS config examples:

- Most cryptocurrency exchanges use a NETTING OMS type, representing a single position per market. It may be desirable for a trader to track multiple "virtual" positions for a strategy.
- Some FX ECNs or brokers use a HEDGING OMS type, tracking multiple positions both LONG and SHORT. The trader may only care about the NET position per currency pair.

:::info
Nautilus does not yet support venue-side hedging modes such as Binance BOTH vs. LONG/SHORT where the venue nets per direction.
It is advised to keep Binance account configurations as BOTH so that a single position is netted.
:::

OMS configuration

If a strategy OMS type is not explicitly set using the omstype configuration option,
it will default to UNSPECIFIED. This means the ExecutionEngine will not override any venue positionids,
and the OMS type will follow the venue's OMS type.

:::tip
When configuring a backtest, you can specify the omstype for the venue. To enhance backtest accuracy, it is recommended to match this with the actual OMS type used by the venue in practice.
:::

Risk engine

The RiskEngine is a core component of every Nautilus system, including backtest, sandbox, and live environments.

Every order command and event passes through the RiskEngine unless specifically bypassed in the RiskEngineConfig.

The RiskEngine includes several built-in pre-trade risk checks, including:

- Price precisions correct for the instrument
- Prices are positive (unless an option type instrument)
- Quantity precisions correct for the instrument
- Below maximum notional for the instrument
- Within maximum or minimum quantity for the instrument
- Only reducing position when a reduceonly execution instruction is specified for the order

If any risk check fails, an OrderDenied event is generated, effectively closing the order and preventing it from progressing further. This event includes a human-readable reason for the denial.

Trading state

Additionally, the current trading state of a Nautilus system affects order flow.

The TradingState enum has three variants:

- ACTIVE: The system operates normally
- HALTED: The system will not process further order commands until the state changes
- REDUCING: The system will only process cancels or commands that reduce open positions

:::info
See the RiskEngineConfig [API Reference](../apireference/config#risk) for further details.
:::

Execution algorithms

The platform supports customized execution algorithm components and provides some built-in algorithms, such as the Time-Weighted Average Price (TWAP) algorithm.

TWAP (Time-Weighted Average Price)

The TWAP execution algorithm aims to execute orders by evenly spreading them over a specified time horizon. The algorithm receives a primary order representing the total size and direction then splits this by spawning smaller child orders, which are then executed at regular intervals throughout the time horizon.

This helps to reduce the impact of the full size of the primary order on the market, by minimizing the concentration of trade size at any given time.

The algorithm will immediately submit the first order, with the final order submitted being the primary order at the end of the horizon period.

Using the TWAP algorithm as an example (found in /examples/algorithms/twap.py), this example demonstrates how to initialize and register a TWAP execution algorithm directly with a BacktestEngine (assuming an engine is already initialized):

For this particular algorithm, two parameters must be specified:

- horizonsecs
- intervalsecs

The horizonsecs parameter determines the time period over which the algorithm will execute, while the intervalsecs parameter sets the time between individual order executions. These parameters determine how a primary order is split into a series of spawned orders.

Alternatively, you can specify these parameters dynamically per order, determining them based on

actual market conditions. In this case, the strategy configuration parameters could be provided to an execution model which determines the horizon and interval.

:::info
There is no limit to the number of execution algorithm parameters you can create. The parameters just need to be a dictionary with string keys and primitive values (values that can be serialized over the wire, such as ints, floats, and strings).
:::

Writing execution algorithms

To implement a custom execution algorithm you must define a class which inherits from ExecAlgorithm.

An execution algorithm is a type of Actor, so it's capable of the following:

- Request and subscribe to data
- Access the Cache
- Set time alerts and/or timers using a Clock

Additionally it can:

- Access the central Portfolio
- Spawn secondary orders from a received primary (original) order

Once an execution algorithm is registered, and the system is running, it will receive orders off the messages bus which are addressed to its ExecAlgorithmId via the execalgorithmid order parameter. The order may also carry the execalgorithmparams being a dict[str, Any].

:::warning
Because of the flexibility of the execalgorithmparams dictionary. It's important to thoroughly validate all of the key value pairs for correct operation of the algorithm (for starters that the dictionary is not None and all necessary parameters actually exist).
:::

Received orders will arrive via the following onorder(...) method. These received orders are know as "primary" (original) orders when being handled by an execution algorithm.

When the algorithm is ready to spawn a secondary order, it can use one of the following methods:

- spawnmarket(...) (spawns a MARKET order)
- spawnmarkettolimit(...) (spawns a MARKETTOLIMIT order)
- spawnlimit(...) (spawns a LIMIT order)

:::note
Additional order types will be implemented in future versions, as the need arises.
:::

Each of these methods takes the primary (original) Order as the first argument. The primary order quantity will be reduced by the quantity passed in (becoming the spawned orders quantity).

:::warning
There must be enough primary order quantity remaining (this is validated).
:::

Once the desired number of secondary orders have been spawned, and the execution routine is over, the intention is that the algorithm will then finally send the primary (original) order.

Spawned orders

All secondary orders spawned from an execution algorithm will carry a execspawnid which is

simply the ClientOrderId of the primary (original) order, and whose clientorderid derives from this original identifier with the following convention:

- execspawnid (primary order clientorderid value)
- spawnsequence (the sequence number for the spawned order)

e.g. O-20230404-001-000-E1 (for the first spawned order)

:::note
The "primary" and "secondary" / "spawn" terminology was specifically chosen to avoid conflict or confusion with the "parent" and "child" contingent orders terminology (an execution algorithm may also deal with contingent orders).
:::

Managing execution algorithm orders

The Cache provides several methods to aid in managing (keeping track of) the activity of an execution algorithm. Calling the below method will return all execution algorithm orders for the given query filters.

As well as more specifically querying the orders for a certain execution series/spawn. Calling the below method will return all orders for the given execspawnid (if found).

:::note
This also includes the primary (original) order.
:::

concepts/index.md

Concepts

Concept guides introduce and explain the foundational ideas, components, and best practices that underpin the NautilusTrader platform.
These guides are designed to provide both conceptual and practical insights, helping you navigate the system's architecture, strategies, data management, execution flow, and more.
Explore the following guides to deepen your understanding and make the most of NautilusTrader.

[Overview](overview.md)

The Overview guide covers the main features and intended use cases for the platform.

[Architecture](architecture.md)

The Architecture guide dives deep into the foundational principles, structures, and designs that underpin
the platform. It is ideal for developers, system architects, or anyone curious about the inner workings of NautilusTrader.

[Actors](actors.md)

The Actor serves as the foundational component for interacting with the trading system.
The Actors guide covers capabilities and implementation specifics.

[Strategies](strategies.md)

The Strategy is at the heart of the NautilusTrader user experience when writing and working with trading strategies. The Strategies guide covers how to implement strategies for the platform.

[Instruments](instruments.md)

The Instruments guide covers the different instrument definition specifications for tradable assets and contracts.

[Data](data.md)

The NautilusTrader platform defines a range of built-in data types crafted specifically to represent a trading domain. The Data guide covers working with both built-in and custom data.

[Execution](execution.md)

NautilusTrader can handle trade execution and order management for multiple strategies and venues simultaneously (per instance). The Execution guide covers components involved in execution, as well as the flow of execution messages (commands and events).

[Orders](orders.md)

The Orders guide provides more details about the available order types for the platform, along with the execution instructions supported for each. Advanced order types and emulated orders are also covered.

[Cache](cache.md)

The Cache is a central in-memory data store for managing all trading-related data.
The Cache guide covers capabilities and best practices of the cache.

[Message Bus](messagebus.md)

The MessageBus is the core communication system enabling decoupled messaging patterns between

components,
including point-to-point, publish/subscribe, and request/response.
The Message Bus guide covers capabilities and best practices of the MessageBus.

[Portfolio](portfolio.md)

The Portfolio serves as the central hub for managing and tracking all positions across active strategies for the trading node or backtest.
It consolidates position data from multiple instruments, providing a unified view of your holdings, risk exposure, and overall performance.
Explore this section to understand how NautilusTrader aggregates and updates portfolio state to support effective trading and risk management.

[Logging](logging.md)

The platform provides logging for both backtesting and live trading using a high-performance logger implemented in Rust.

[Backtesting](backtesting.md)

Backtesting with NautilusTrader is a methodical simulation process that replicates trading
activities using a specific system implementation.

[Live Trading](live.md)

Live trading in NautilusTrader enables traders to deploy their backtested strategies in real-time
without any code changes. This seamless transition ensures consistency and reliability, though there
are key differences between backtesting and live trading.

[Adapters](adapters.md)

The NautilusTrader design allows for integrating data providers and/or trading venues through adapter implementations.
The Adapters guide covers requirements and best practices for developing new integration adapters for the platform.

:::note
The [API Reference](../apireference/index.md) documentation should be considered the source of truth for the platform. If there are any discrepancies between concepts described here and the API Reference,
then the API Reference should be considered the correct information. We are working to ensure that
concepts stay up-to-date with the API Reference and will be introducing doc tests in the near future
to help with this.
:::

concepts/instruments.md

Instruments

The Instrument base class represents the core specification for any tradable asset/contract. There are currently a number of subclasses representing a range of asset classes and instrument classes which are supported by the platform:

- Equity (generic Equity)
- FuturesContract (generic Futures Contract)
- FuturesSpread (generic Futures Spread)
- OptionContract (generic Option Contract)
- OptionSpread (generic Option Spread)
- BinaryOption (generic Binary Option instrument)
- Cfd (Contract for Difference instrument)
- Commodity (commodity instrument in a spot/cash market)
- CurrencyPair (represents a Fiat FX or Cryptocurrency pair in a spot/cash market)
- CryptoOption (Crypto Option instrument)
- CryptoPerpetual (Perpetual Futures Contract a.k.a. Perpetual Swap)
- CryptoFuture (Deliverable Futures Contract with Crypto assets as underlying, and for price quotes and settlement)
- IndexInstrument (generic Index instrument)
- BettingInstrument (Sports, gaming, or other betting)

Symbology

All instruments should have a unique InstrumentId, which is made up of both the native symbol, and venue ID, separated by a period.
For example, on the Binance Futures crypto exchange, the Ethereum Perpetual Futures Contract has the instrument ID ETHUSDT-PERP.BINANCE.

All native symbols should be unique for a venue (this is not always the case e.g. Binance share native symbols between spot and futures markets),
and the {symbol.venue} combination must be unique for a Nautilus system.

:::warning
The correct instrument must be matched to a market dataset such as ticks or order book data for logically sound operation.
An incorrectly specified instrument may truncate data or otherwise produce surprising results.
:::

Backtesting

Generic test instruments can be instantiated through the TestInstrumentProvider:

Exchange specific instruments can be discovered from live exchange data using an adapters InstrumentProvider:

Or flexibly defined by the user through an Instrument constructor, or one of its more specific subclasses:

See the full instrument [API Reference](../apireference/model/instruments.md).

Live trading

Live integration adapters have defined InstrumentProvider classes which work in an automated way to cache the

latest instrument definitions for the exchange. Refer to a particular Instrument object by passing the matching InstrumentId to data and execution related methods and classes that require one.

Finding instruments

Since the same actor/strategy classes can be used for both backtest and live trading, you can get instruments in exactly the same way through the central cache:

It's also possible to subscribe to any changes to a particular instrument:

Or subscribe to all instrument changes for an entire venue:

When an update to the instrument(s) is received by the DataEngine, the object(s) will be passed to the actors/strategies oninstrument() method. A user can override this method with actions to take upon receiving an instrument update:

Precisions and increments

The instrument objects are a convenient way to organize the specification of an instrument through read-only properties. Correct price and quantity precisions, as well as minimum price and size increments, multipliers and standard lot sizes, are available.

:::note
Most of these limits are checked by the Nautilus RiskEngine, otherwise invalid values for prices and quantities can result in the exchange rejecting orders.
:::

Limits

Certain value limits are optional for instruments and can be None, these are exchange dependent and can include:

- maxquantity (maximum quantity for a single order)
- minquantity (minimum quantity for a single order)
- maxnotional (maximum value of a single order)
- minnotional (minimum value of a single order)
- maxprice (maximum valid quote or order price)
- minprice (minimum valid quote or order price)

:::note
Most of these limits are checked by the Nautilus RiskEngine, otherwise exceeding published limits can result in the exchange rejecting orders.
:::

Prices and quantities

Instrument objects also offer a convenient way to create correct prices and quantities based on given values.

:::tip
The above is the recommended method for creating valid prices and quantities, such as when passing them to the order factory to create an order.
:::

Margins and fees

Margin calculations are handled by the MarginAccount class. This section explains how margins work and introduces key concepts you need to know.

When margins apply?

Each exchange (e.g., CME or Binance) operates with a specific account type that determines whether margin calculations are applicable.
When setting up an exchange venue, you'll specify one of these account types:

- AccountType.MARGIN: Accounts that use margin calculations, which are explained below.
- AccountType.CASH: Simple accounts where margin calculations do not apply.
- AccountType.BETTING: Accounts designed for betting, which also do not involve margin calculations.

Vocabulary

To understand trading on margin, let's start with some key terms:

Notional Value: The total contract value in the quote currency. It represents the full market value of your position. For example, with EUR/USD futures on CME (symbol 6E).

- Each contract represents 125,000 EUR (EUR is base currency, USD is quote currency).
- If the current market price is 1.1000, the notional value equals 125,000 EUR × 1.1000 (price of EUR/USD) = 137,500 USD.

Leverage (leverage): The ratio that determines how much market exposure you can control relative to your account deposit. For example, with 10× leverage, you can control 10,000 USD worth of positions with just 1,000 USD in your account.

Initial Margin (margininit): The margin rate required to open a position. It represents the minimum amount of funds that must be available in your account to open new positions. This is only a pre-check - no funds are actually locked.

Maintenance Margin (marginmaint): The margin rate required to keep a position open. This amount is locked in your account to maintain the position. It is always lower than the initial margin. You can view the total blocked funds (sum of maintenance margins for open positions) using the following in your strategy:

Maker/Taker Fees: The fees charged by exchanges based on your order's interaction with the market:

- Maker Fee (makerfee): A fee (typically lower) charged when you "make" liquidity by placing an order that remains on the order book. For example, a limit buy order below the current price adds liquidity, and the maker fee applies when it fills.
- Taker Fee (takerfee): A fee (typically higher) charged when you "take" liquidity by placing an order that executes immediately. For instance, a market buy order or a limit buy above the current price removes liquidity, and the taker fee applies.

:::tip
Not all exchanges or instruments implement maker/taker fees. If absent, set both makerfee and takerfee to 0 for the Instrument (e.g., FuturesContract, Equity, CurrencyPair, Commodity, Cfd, BinaryOption, BettingInstrument).
:::

Margin calculation formula

The MarginAccount class calculates margins using the following formulas:

Key Points:

- Both formulas follow the same structure but use their respective margin rates (margininit and marginmaint).
- Each formula consists of two parts:
  - Primary margin calculation: Based on notional value, leverage, and margin rate.

- Fee Adjustment: Accounts for the maker/taker fee.

Implementation details

For those interested in exploring the technical implementation:

-
[nautilustrader/accounting/accounts/margin.pyx](https://github.com/nautechsystems/nautilustrader/blob/develop/nautilustrader/accounting/accounts/margin.pyx)
- Key methods: calculatemargininit(self, ...) and calculatemarginmaint(self, ...)

Commissions

Trading commissions represent the fees charged by exchanges or brokers for executing trades.
While maker/taker fees are common in cryptocurrency markets, traditional exchanges like CME often employ other fee structures, such as per-contract commissions.
NautilusTrader supports multiple commission models to accommodate diverse fee structures across different markets.

Built-in Fee Models

The framework provides two built-in fee model implementations:

1. MakerTakerFeeModel: Implements the maker/taker fee structure common in cryptocurrency exchanges, where fees are
   calculated as a percentage of the trade value.
2. FixedFeeModel: Applies a fixed commission per trade, regardless of the trade size.

Creating Custom Fee Models

While the built-in fee models cover common scenarios, you might encounter situations requiring specific commission structures.
NautilusTrader's flexible architecture allows you to implement custom fee models by inheriting from the base FeeModel class.

For example, if you're trading futures on exchanges that charge per-contract commissions (like CME), you can implement
a custom fee model. When creating custom fee models, we inherit from the FeeModel base class, which is implemented
in Cython for performance reasons. This Cython implementation is reflected in the parameter naming convention,
where type information is incorporated into parameter names using underscores (like Orderorder or Quantityfillqty).

While these parameter names might look unusual to Python developers, they're a result of Cython's type system and help
maintain consistency with the framework's core components. Here's how you could create a per-contract commission model:

This custom implementation calculates the total commission by multiplying a fixed per-contract fee by the number
of contracts traded. The getcommission(...) method receives information about the order, fill quantity, fill price
and instrument, allowing for flexible commission calculations based on these parameters.

Our new class PerContractFeeModel inherits class FeeModel, which is implemented in Cython,
so notice the Cython-style parameter names in the method signature:

- Orderorder: The order object, with type prefix Order.

- Quantityfillqty: The fill quantity, with type prefix Quantity.
- Pricefillpx: The fill price, with type prefix Price.
- Instrumentinstrument: The instrument object, with type prefix Instrument.

These parameter names follow NautilusTrader's Cython naming conventions, where the prefix indicates the expected type.
While this might seem verbose compared to typical Python naming conventions, it ensures type safety and consistency
with the framework's Cython codebase.

Using Fee Models in Practice

To use any fee model in your trading system, whether built-in or custom, you specify it when setting up the venue.
Here's an example using the custom per-contract fee model:

:::tip
When implementing custom fee models, ensure they accurately reflect the fee structure of your target exchange.
Even small discrepancies in commission calculations can significantly impact strategy performance metrics during backtesting.
:::

Additional info

The raw instrument definition as provided by the exchange (typically from JSON serialized data) is also included as a generic Python dictionary. This is to retain all information
which is not necessarily part of the unified Nautilus API, and is available to the user
at runtime by calling the .info property.

Synthetic Instruments

The platform supports creating customized synthetic instruments, which can generate synthetic quote and trades. These are useful for:

- Enabling Actor and Strategy components to subscribe to quote or trade feeds.
- Triggering emulated orders.
- Constructing bars from synthetic quotes or trades.

Synthetic instruments cannot be traded directly, as they are constructs that only exist locally
within the platform. They serve as analytical tools, providing useful metrics based on their component instruments.

In the future, we plan to support order management for synthetic instruments, enabling trading of their component instruments based on the synthetic instrument's behavior.

:::info
The venue for a synthetic instrument is always designated as 'SYNTH'.
:::

Formula

A synthetic instrument is composed of a combination of two or more component instruments (which can include instruments from multiple venues), as well as a "derivation formula".
Utilizing the dynamic expression engine powered by the [evalexpr](https://github.com/ISibboI/evalexpr) Rust crate, the platform can evaluate the formula to calculate the latest synthetic price tick
from the incoming component instrument prices.

See the evalexpr documentation for a full description of available features, operators and precedence.

:::tip
Before defining a new synthetic instrument, ensure that all component instruments are already defined and exist in the cache.
:::

Subscribing

The following example demonstrates the creation of a new synthetic instrument with an actor/strategy.
This synthetic instrument will represent a simple spread between Bitcoin and
Ethereum spot prices on Binance. For this example, it is assumed that spot instruments for
BTCUSDT.BINANCE and ETHUSDT.BINANCE are already present in the cache.

:::note
The instrumentid for the synthetic instrument in the above example will be structured as {symbol}.{SYNTH}, resulting in 'BTC-ETH:BINANCE.SYNTH'.
:::

Updating formulas

It's also possible to update a synthetic instrument formulas at any time. The following example shows how to achieve this with an actor/strategy.

Trigger instrument IDs

The platform allows for emulated orders to be triggered based on synthetic instrument prices. In the following example, we build upon the previous one to submit a new emulated order.
This order will be retained in the emulator until a trigger from synthetic quotes releases it.
It will then be submitted to Binance as a MARKET order:

Error handling

Considerable effort has been made to validate inputs, including the derivation formula for synthetic instruments. Despite this, caution is advised as invalid or erroneous inputs may lead to undefined behavior.

:::info
See the SyntheticInstrument [API reference](../../apireference/model/instruments.md#class-syntheticinstrument-1) for a detailed understanding of input requirements and potential exceptions.
:::

concepts/live.md

Live Trading

Live trading in NautilusTrader enables traders to deploy their backtested strategies in a real-time trading environment with no code changes. This seamless transition from backtesting to live trading is a core feature of the platform, ensuring consistency and reliability. However, there are key differences to be aware of between backtesting and live trading.

This guide provides an overview of the key aspects of live trading.

Configuration

When operating a live trading system, configuring your execution engine and strategies properly is essential for ensuring reliability, accuracy, and performance. The following is an overview of the key concepts and settings involved for live configuration.

TradingNodeConfig

The main configuration class for live trading systems is TradingNodeConfig, which inherits from NautilusKernelConfig and provides live-specific config options:

Core TradingNodeConfig Parameters

Setting	Default	Description
traderid	"TRADER-001"	Unique trader identifier (name-tag format).
instanceid	None	Optional unique instance identifier.
timeoutconnection	30.0	Connection timeout in seconds.
timeoutreconciliation	10.0	Reconciliation timeout in seconds.
timeoutportfolio	10.0	Portfolio initialization timeout.
timeoutdisconnection	10.0	Disconnection timeout.
timeoutpoststop	5.0	Post-stop cleanup timeout.

Cache Database Configuration

Configure data persistence with a backing database:

Message Bus Configuration

Configure message routing and external streaming:

Multi-venue Configuration

Live trading systems often connect to multiple venues or market types. Here's an example of configuring both spot and futures markets for Binance:

Execution Engine configuration

The LiveExecEngineConfig sets up the live execution engine, managing order processing, execution events, and reconciliation with trading venues.
The following outlines the main configuration options.

By configuring these parameters thoughtfully, you can ensure that your trading system operates efficiently,
handles orders correctly, and remains resilient in the face of potential issues, such as lost events or conflicting data/information.

:::info

See also the LiveExecEngineConfig [API Reference](../apireference/config#class-liveexecengineconfig) for further details.
:::

Reconciliation

Purpose: Ensures that the system state remains consistent with the trading venue by recovering any missed events, such as order and position status updates.

Setting	Default	Description
reconciliation	True	Activates reconciliation at startup, aligning the system's internal state with the venue's state.
reconciliationlookbackmins	None	Specifies how far back (in minutes) the system requests past events to reconcile uncached state.

:::info
See also [Execution reconciliation](../concepts/execution#execution-reconciliation) for further details.
:::

Order filtering

Purpose: Manages which order events and reports should be processed by the system to avoid conflicts with other trading nodes and unnecessary data handling.

Setting	Default	Description
filterunclaimedexternalorders	False	Filters out unclaimed external orders to prevent irrelevant orders from impacting the strategy.
filterpositionreports	False	Filters out position status reports, useful when multiple nodes trade the same account to avoid conflicts.

In-flight order checks

Purpose: Regularly checks the status of in-flight orders (orders that have been submitted, modified or canceled but not yet confirmed) to ensure they are processed correctly and promptly.

Setting	Default	Description
inflightcheckintervalms	2,000 ms	Determines how frequently the system checks in-flight order status.
inflightcheckthresholdms	5,000 ms	Sets the time threshold after which an in-flight order triggers a venue status check. Adjust if colocated to avoid race conditions.
inflightcheckretries	5 retries	Specifies the number of retry attempts the engine will make to verify the status of an in-flight order with the venue, should the initial attempt fail.

Additional execution engine options

The following additional options provide further control over execution behavior:

Setting	Default	Description

----------|
generatemissingorders	True	If MARKET order events will be generated during reconciliation to align position discrepancies.
snapshotorders	False	If order snapshots should be taken on order events.
snapshotpositions	False	If position snapshots should be taken on position events.
snapshotpositionsintervalsecs	None	The interval (seconds) between position snapshots when enabled.
debug	False	Enable debug mode for additional execution logging.

If an in-flight order's status cannot be reconciled after exhausting all retries, the system resolves it by generating one of these events based on its status:

- SUBMITTED -> REJECTED: Assumes the submission failed if no confirmation is received.
- PENDINGUPDATE -> CANCELED: Treats a pending modification as canceled if unresolved.
- PENDINGCANCEL -> CANCELED: Assumes cancellation if the venue doesn't respond.

This ensures the trading node maintains a consistent execution state even under unreliable conditions.

Open order checks

Purpose: Regularly verifies the status of open orders matches the venue, triggering reconciliation if discrepancies are found.

Setting	Default	Description
opencheckintervalsecs	None	Determines how frequently (in seconds) open orders are checked at the venue. Recommended: 5-10 seconds, considering API rate limits.
opencheckopenonly	True	When enabled, only open orders are requested during checks; if disabled, full order history is fetched (resource-intensive).

Order book audit

Purpose: Ensures that the internal representation of own order books matches the venues public order books.

Setting	Default	Description
ownbooksauditintervalsecs	None	Sets the interval (in seconds) between audits of own order books against public ones. Verifies synchronization and logs errors for inconsistencies.

Memory management

Purpose: Periodically purges closed orders, closed positions, and account events from the in-memory cache to optimize resource usage and performance during extended / HFT operations.

Setting	Default	Description
purgeclosedordersintervalmins	None	Sets how frequently (in minutes) closed orders are purged from memory. Recommended: 10-15 minutes. Does not affect database records.

purgeclosedordersbuffermins	None	Specifies how long (in minutes) an order must have been closed before purging. Recommended: 60 minutes to ensure processes complete.
purgeclosedpositionsintervalmins	None	Sets how frequently (in minutes) closed positions are purged from memory. Recommended: 10-15 minutes. Does not affect database records.
purgeclosedpositionsbuffermins	None	Specifies how long (in minutes) a position must have been closed before purging. Recommended: 60 minutes to ensure processes complete.
purgeaccounteventsintervalmins	None	Sets how frequently (in minutes) account events are purged from memory. Recommended: 10-15 minutes. Does not affect database records.
purgeaccounteventslookbackmins	None	Specifies how long (in minutes) an account event must have occurred before purging. Recommended: 60 minutes.

By configuring these memory management settings appropriately, you can prevent memory usage from growing
indefinitely during long-running / HFT sessions while ensuring that recently closed orders, closed positions, and account events
remain available in memory for any ongoing operations that might require them.

Queue management

Purpose: Handles the internal buffering of order events to ensure smooth processing and to prevent system resource overloads.

Setting	Default	Description
qsize	100,000	Sets the size of internal queue buffers, managing the flow of data within the engine.

Strategy configuration

The StrategyConfig class outlines the configuration for trading strategies, ensuring that each strategy operates with the correct parameters and manages orders effectively.
The following outlines the main configuration options.

:::info
See also the StrategyConfig [API Reference](../apireference/config#class-strategyconfig) for further details.
:::

Strategy identification

Purpose: Provides unique identifiers for each strategy to prevent conflicts and ensure proper tracking of orders.

Setting	Default	Description
strategyid	None	A unique ID for the strategy, ensuring it can be distinctly identified.
orderidtag	None	A unique tag for the strategy's orders, differentiating them from multiple strategies.
useuuidclientorderids	False	If UUID4's should be used for client order ID values (required for some venues such as Coinbase Intx).

Order Management System (OMS) type

Purpose: Defines how the order management system handles position IDs, influencing how orders are processed and tracked.

Setting	Default	Description
omstype	None	Specifies the [OMS type](../concepts/execution#oms-configuration), for position ID handling and order processing flow.

External order claims

Purpose: Enables the strategy to claim external orders based on specified instrument IDs, ensuring that relevant external orders are associated with the correct strategy.

Setting	Default	Description
externalorderclaims	None	Lists instrument IDs for external orders the strategy should claim, aiding accurate order management.

Contingent order management

Purpose: Automates the management of contingent orders, such as One-Updates-the-Other (OUO) and One-Cancels-the-Other (OCO) orders, ensuring they are handled correctly.

Setting	Default	Description
managecontingentorders	False	If enabled, the strategy automatically manages contingent orders, reducing manual intervention.

Good Till Date (GTD) expiry management

Purpose: Ensures that orders with GTD time in force instructions are managed properly, with timers reactivated as necessary.

Setting	Default	Description
managegtdexpiry	False	If enabled, the strategy manages GTD expirations, ensuring orders remain active as intended.

Execution reconciliation

Execution reconciliation is the process of aligning the external state of reality for orders and positions (both closed and open) with the current system internal state built from events.
This process is primarily applicable to live trading, which is why only the LiveExecutionEngine has reconciliation capability.

There are two main scenarios for reconciliation:

- Previous cached execution state: Where cached execution state exists, information from reports is used to generate missing events to align the state
- No previous cached execution state: Where there is no cached state, all orders and positions that exist externally are generated from scratch

Common reconciliation issues

- Missing trade reports: Some venues filter out older trades, causing incomplete reconciliation. Increase reconciliationlookbackmins or ensure all events are cached locally.
- Position mismatches: If external orders predate the lookback window, positions may not align. Flatten the account before restarting the system to reset state.

:::tip
Best practice: Persist all execution events to the cache database to minimize reliance on venue history, ensuring full recovery even with short lookback windows.
:::

Reconciliation configuration

Unless reconciliation is disabled by setting the reconciliation configuration parameter to false, the execution engine will perform the execution reconciliation procedure for each venue.
Additionally, you can specify the lookback window for reconciliation by setting the reconciliationlookbackmins configuration parameter.

:::tip
We recommend not setting a specific reconciliationlookbackmins. This allows the requests made to the venues to utilize the maximum execution history available for reconciliation.
:::

:::warning
If executions have occurred prior to the lookback window, any necessary events will be generated to align
internal and external states. This may result in some information loss that could have been avoided with a longer lookback window.

Additionally, some venues may filter or drop execution information under certain conditions, resulting in further information loss. This would not occur if all events were persisted in the cache database.
:::

Each strategy can also be configured to claim any external orders for an instrument ID generated during
reconciliation using the externalorderclaims configuration parameter.
This is useful in situations where, at system start, there is no cached state or it is desirable for a strategy to resume its operations and continue managing existing open orders at the venue for an instrument.

:::info
See the LiveExecEngineConfig [API Reference](../apireference/config#class-liveexecengineconfig) for further details.
:::

Reconciliation procedure

The reconciliation procedure is standardized for all adapter execution clients and uses the following methods to produce an execution mass status:

- generateorderstatusreports
- generatefillreports
- generatepositionstatusreports

The system state is then reconciled with the reports, which represent external "reality":

- Duplicate Check:
  - Check for duplicate order IDs and trade IDs.
- Order Reconciliation:
  - Generate and apply events necessary to update orders from any cached state to the current state.
  - If any trade reports are missing, inferred OrderFilled events are generated.
  - If any client order ID is not recognized or an order report lacks a client order ID, external order events are generated.
- Position Reconciliation:

- Ensure the net position per instrument matches the position reports returned from the venue.
- If the position state resulting from order reconciliation does not match the external state, external order events will be generated to resolve discrepancies.

If reconciliation fails, the system will not continue to start, and an error will be logged.

:::tip
The current reconciliation procedure can experience state mismatches if the lookback window is misconfigured or if the venue omits certain order or trade reports due to filter conditions.

If you encounter reconciliation issues, drop any cached state or ensure the account is flat at system shutdown and startup.
:::

concepts/logging.md

Logging

The platform provides logging for both backtesting and live trading using a high-performance logging subsystem implemented in Rust
with a standardized facade from the log crate.

The core logger operates in a separate thread and uses a multi-producer single-consumer (MPSC) channel to receive log messages.
This design ensures that the main thread remains performant, avoiding potential bottlenecks caused by log string formatting or file I/O operations.

Logging output is configurable and supports:

- stdout/stderr writer for console output
- file writer for persistent storage of logs

:::info
Infrastructure such as [Vector](https://github.com/vectordotdev/vector) can be integrated to collect and aggregate events within your system.
:::

Configuration

Logging can be configured by importing the LoggingConfig object.
By default, log events with an 'INFO' LogLevel and higher are written to stdout/stderr.

Log level (LogLevel) values include (and generally match Rust's tracing level filters).

Python loggers expose the following levels:

- OFF
- DEBUG
- INFO
- WARNING
- ERROR

:::warning
The Python Logger does not provide a trace() method; TRACE level logs are only emitted by the underlying Rust components and cannot be generated directly from Python code.

See the LoggingConfig [API Reference](../apireference/config.md#class-loggingconfig) for further details.
:::

Logging can be configured in the following ways:

- Minimum LogLevel for stdout/stderr
- Minimum LogLevel for log files
- Maximum size before rotating a log file
- Maximum number of backup log files to maintain when rotating
- Automatic log file naming with date or timestamp components, or custom log file name
- Directory for writing log files
- Plain text or JSON log file formatting
- Filtering of individual components by log level
- ANSI colors in log lines
- Bypass logging entirely

- Print Rust config to stdout at initialization
- Optionally initialize logging via the PyO3 bridge (usepyo3) to capture log events emitted by Rust components
- Truncate existing log file on startup if it already exists (clearlogfile)

Standard output logging

Log messages are written to the console via stdout/stderr writers. The minimum log level can be configured using the loglevel parameter.

File logging

Log files are written to the current working directory by default. The naming convention and rotation behavior are configurable and follow specific patterns based on your settings.

You can specify a custom log directory using logdirectory and/or a custom file basename using logfilename. Log files are always suffixed with .log (plain text) or .json (JSON).

For detailed information about log file naming conventions and rotation behavior, see the [Log file rotation](#log-file-rotation) and [Log file naming convention](#log-file-naming-convention) sections below.

Log file rotation

Rotation behavior depends on both the presence of a size limit and whether a custom file name is provided:

- Size-based rotation:
  - Enabled by specifying the logfilemaxsize parameter (e.g., 100000000 for 100 MB).
  - When writing a log entry would make the current file exceed this size, the file is closed and a new one is created.
- Date-based rotation (default naming only):
  - Applies when no logfilemaxsize is specified and no custom logfilename is provided.
  - At each UTC date change (midnight), the current log file is closed and a new one is started, creating one file per UTC day.
- No rotation:
  - When a custom logfilename is provided without a logfilemaxsize, logs continue to append to the same file.
- Backup file management:
  - Controlled by the logfilemaxbackupcount parameter (default: 5), limiting the total number of rotated files kept.
  - When this limit is exceeded, the oldest backup files are automatically removed.

Log file naming convention

The default naming convention ensures log files are uniquely identifiable and timestamped.
The format depends on whether file rotation is enabled:

With file rotation enabled:

- Format: {traderid}{%Y-%m-%d%H%M%S:%3f}{instanceid}.{log|json}
- Example: TESTER-0012025-04-09210721:521d7dc12c8-7008-4042-8ac4-017c3db0fc38.log
- Components:
  - {traderid}: The trader identifier (e.g., TESTER-001).
  - {%Y-%m-%d%H%M%S:%3f}: Full ISO 8601-compliant datetime with millisecond resolution.
  - {instanceid}: A unique instance identifier.
  - {log|json}: File suffix based on format setting.

With file rotation disabled:

- Format: {traderid}{%Y-%m-%d}{instanceid}.{log|json}
- Example: TESTER-0012025-04-09d7dc12c8-7008-4042-8ac4-017c3db0fc38.log
- Components:
  - {traderid}: The trader identifier.
  - {%Y-%m-%d}: Date only (YYYY-MM-DD).
  - {instanceid}: A unique instance identifier.
  - {log|json}: File suffix based on format setting.

Custom naming:

If logfilename is set (e.g., mycustomlog):

- With rotation disabled: The file will be named exactly as provided (e.g., mycustomlog.log).
- With rotation enabled: The file will include the custom name and timestamp (e.g., mycustomlog2025-04-09210721:521.log).

Component log filtering

The logcomponentlevels parameter can be used to set log levels for each component individually.
The input value should be a dictionary of component ID strings to log level strings: dict[str, str].

Below is an example of a trading node logging configuration that includes some of the options mentioned above:

For backtesting, the BacktestEngineConfig class can be used instead of TradingNodeConfig, as the same options are available.

Log Colors

ANSI color codes are utilized to enhance the readability of logs when viewed in a terminal.
These color codes can make it easier to distinguish different parts of log messages.
In environments that do not support ANSI color rendering (such as some cloud environments or text editors),
these color codes may not be appropriate as they can appear as raw text.

To accommodate for such scenarios, the LoggingConfig.logcolors option can be set to false.
Disabling logcolors will prevent the addition of ANSI color codes to the log messages, ensuring compatibility across different environments where color rendering is not supported.

Using a Logger directly

It's possible to use Logger objects directly, and these can be initialized anywhere (very similar to the Python built-in logging API).

If you aren't using an object which already initializes a NautilusKernel (and logging) such as BacktestEngine or TradingNode,
then you can activate logging in the following way:

:::info
See the initlogging [API Reference](../apireference/common) for further details.
:::

:::warning
Only one logging subsystem can be initialized per process with an initlogging call, and the LogGuard which is returned must be kept alive for the lifetime of the program.
:::

LogGuard: Managing log lifecycle

The LogGuard ensures that the logging subsystem remains active and operational throughout the

lifecycle of a process.
It prevents premature shutdown of the logging subsystem when running multiple engines in the same process.

Why use LogGuard?

Without a LogGuard, any attempt to run sequential engines in the same process may result in errors such as:

This occurs because the logging subsystem's underlying channel and Rust Logger are closed when the first engine is disposed.
As a result, subsequent engines lose access to the logging subsystem, leading to these errors.

By leveraging a LogGuard, you can ensure robust logging behavior across multiple backtests or engine runs in the same process.
The LogGuard retains the resources of the logging subsystem and ensures that logs continue to function correctly,
even as engines are disposed and initialized.

:::note
Using LogGuard is critical to maintain consistent logging behavior throughout a process with multiple engines.
:::

Running multiple engines

The following example demonstrates how to use a LogGuard when running multiple engines sequentially in the same process:

Steps

- Initialize LogGuard once: The LogGuard is obtained from the first engine (engine.getlogguard()) and is retained throughout the process. This ensures that the logging subsystem remains active.
- Dispose engines safely: Each engine is safely disposed of after its backtest completes, without affecting the logging subsystem.
- Reuse LogGuard: The same LogGuard instance is reused for subsequent engines, preventing the logging subsystem from shutting down prematurely.

Considerations

- Single LogGuard per process: Only one LogGuard can be used per process.
- Thread safety: The logging subsystem, including LogGuard, is thread-safe, ensuring consistent behavior even in multi-threaded environments.
- Flush logs on termination: Always ensure that logs are properly flushed when the process terminates. The LogGuard automatically handles this as it goes out of scope.

concepts/message_bus.md

Message Bus

The MessageBus is a fundamental part of the platform, enabling communication between system components
through message passing. This design creates a loosely coupled architecture where components can interact
without direct dependencies.

The messaging patterns include:

- Point-to-Point
- Publish/Subscribe
- Request/Response

Messages exchanged via the MessageBus fall into three categories:

- Data
- Events
- Commands

Data and signal publishing

While the MessageBus is a lower-level component that users typically interact with indirectly,
Actor and Strategy classes provide convenient methods built on top of it:

These methods allow you to publish custom data and signals efficiently without needing to work directly with the MessageBus interface.

Direct access

For advanced users or specialized use cases, direct access to the message bus is available within Actor and Strategy
classes through the self.msgbus reference, which provides the full message bus interface.

To publish a custom message directly, you can specify a topic as a str and any Python object as the message payload, for example:

Messaging styles

NautilusTrader is an event-driven framework where components communicate by sending and receiving messages.
Understanding the different messaging styles is crucial for building effective trading systems.

This guide explains the three primary messaging patterns available in NautilusTrader:

Messaging Style	Purpose	Best For
MessageBus - Publish/Subscribe to topics	Low-level, direct access to the message bus	Custom events, system-level communication
Actor-Based - Publish/Subscribe Data	Structured trading data exchange	Trading metrics, indicators, data needing persistence
Actor-Based - Publish/Subscribe Signal	Lightweight notifications	Simple alerts, flags, status updates

Each approach serves different purposes and offers unique advantages. This guide will help you decide which messaging

pattern to use in your NautilusTrader applications.

MessageBus publish/subscribe to topics

Concept

The MessageBus is the central hub for all messages in NautilusTrader. It enables a publish/subscribe pattern
where components can publish events to named topics, and other components can subscribe to receive those messages.
This decouples components, allowing them to interact indirectly via the message bus.

Key benefits and use cases

The message bus approach is ideal when you need:

- Cross-component communication within the system.
- Flexibility to define any topic and send any type of payload (any Python object).
- Decoupling between publishers and subscribers who don't need to know about each other.
- Global Reach where messages can be received by multiple subscribers.
- Working with events that don't fit within the predefined Actor model.
- Advanced scenarios requiring full control over messaging.

Considerations

- You must track topic names manually (typos could result in missed messages).
- You must define handlers manually.

Quick overview code

Full example

[MessageBus
Example](https://github.com/nautechsystems/nautilustrader/tree/develop/examples/backtest/example09messagingwithmsgbus)

Actor-based publish/subscribe data

Concept

This approach provides a way to exchange trading specific data between Actors in the system.
(note: each Strategy inherits from Actor). It inherits from Data, which ensures proper timestamping and ordering of events - crucial for correct backtest processing.

Key Benefits and Use Cases

The Data publish/subscribe approach excels when you need:

- Exchange of structured trading data like market data, indicators, custom metrics, or option greeks.
- Proper event ordering via built-in timestamps (tsevent, tsinit) crucial for backtest accuracy.
- Data persistence and serialization through the @customdataclass decorator, integrating seamlessly with NautilusTrader's data catalog system.
- Standardized trading data exchange between system components.

Considerations

- Requires defining a class that inherits from Data or uses @customdataclass.

Inheriting from Data vs. using @customdataclass

Inheriting from Data class:

- Defines abstract properties tsevent and tsinit that must be implemented by the subclass. These ensure proper data ordering in backtests based on timestamps.

The @customdataclass decorator:

- Adds tsevent and tsinit attributes if they are not already present.
- Provides serialization functions: todict(), fromdict(), tobytes(), toarrow(), etc.
- Enables data persistence and external communication.

Quick overview code

Full example

[Actor-Based Data Example](https://github.com/nautechsystems/nautilustrader/tree/develop/examples/backtest/example10messagingwithactordata)

Actor-based publish/subscribe signal

Concept

Signals are a lightweight way to publish and subscribe to simple notifications within the actor framework.
This is the simplest messaging approach, requiring no custom class definitions.

Key Benefits and Use Cases

The Signal messaging approach shines when you need:

- Simple, lightweight notifications/alerts like "RiskThresholdExceeded" or "TrendUp".
- Quick, on-the-fly messaging without defining custom classes.
- Broadcasting alerts or flags as primitive data (int, float, or str).
- Easy API integration with straightforward methods (publishsignal, subscribesignal).
- Multiple subscriber communication where all subscribers receive signals when published.
- Minimal setup overhead with no class definitions required.

Considerations

- Each signal can contain only single value of type: int, float, and str. That means no support for complex data structures or other Python types.
- In the onsignal handler, you can only differentiate between signals using signal.value, as the signal name is not accessible in the handler.

Quick overview code

Full example

[Actor-Based Signal Example](https://github.com/nautechsystems/nautilustrader/tree/develop/examples/backtest/example11messagingwithactorsignals)

Summary and decision guide

Here's a quick reference to help you decide which messaging style to use:

Decision guide: Which style to choose?

Use Case	Recommended Approach	Setup required

Custom events or system-level communication	MessageBus + Pub/Sub to topic	Topic + Handler management
Structured trading data	Actor + Pub/Sub Data + optional @customdataclass if serialization is needed	New class definition inheriting from Data (handler ondata is predefined)
Simple alerts/notifications	Actor + Pub/Sub Signal	Just signal name

External publishing

The MessageBus can be backed with any database or message broker technology which has an integration written for it, this then enables external publishing of messages.

:::info
Redis is currently supported for all serializable messages which are published externally.
The minimum supported Redis version is 6.2 (required for [streams](https://redis.io/docs/latest/develop/data-types/streams/) functionality).
:::

Under the hood, when a backing database (or any other compatible technology) is configured,
all outgoing messages are first serialized, then transmitted via a Multiple-Producer Single-Consumer (MPSC) channel to a separate thread (implemented in Rust).
In this separate thread, the message is written to its final destination, which is presently Redis streams.

This design is primarily driven by performance considerations. By offloading the I/O operations to a separate thread,
we ensure that the main thread remains unblocked and can continue its tasks without being hindered by the potentially
time-consuming operations involved in interacting with a database or client.

Serialization

Nautilus supports serialization for:

- All Nautilus built-in types (serialized as dictionaries dict[str, Any] containing serializable primitives).
- Python primitive types (str, int, float, bool, bytes).

You can add serialization support for custom types by registering them through the serialization subpackage.

- cls: The type to register.
- todict: The delegate to instantiate a dict of primitive types from the object.
- fromdict: The delegate to instantiate the object from a dict of primitive types.

Configuration

The message bus external backing technology can be configured by importing the MessageBusConfig object and passing this to
your TradingNodeConfig. Each of these config options will be described below.

Database config

A DatabaseConfig must be provided, for a default Redis setup on the local
loopback you can pass a DatabaseConfig(), which will use defaults to match.

Encoding

Two encodings are currently supported by the built-in Serializer used by the MessageBus:

- JSON (json)
- MessagePack (msgpack)

Use the encoding config option to control the message writing encoding.

:::tip
The msgpack encoding is used by default as it offers the most optimal serialization and memory performance.
We recommend using json encoding for human readability when performance is not a primary concern.
:::

Timestamp formatting

By default timestamps are formatted as UNIX epoch nanosecond integers. Alternatively you can configure ISO 8601 string formatting by setting the timestampsasiso8601 to True.

Message stream keys

Message stream keys are essential for identifying individual trader nodes and organizing messages within streams.
They can be tailored to meet your specific requirements and use cases. In the context of message bus streams, a trader key is typically structured as follows:

The following options are available for configuring message stream keys:

Trader prefix

If the key should begin with the trader string.

Trader ID

If the key should include the trader ID for the node.

Instance ID

Each trader node is assigned a unique 'instance ID,' which is a UUIDv4. This instance ID helps distinguish individual traders when messages
are distributed across multiple streams. You can include the instance ID in the trader key by setting the useinstanceid configuration option to True.
This is particularly useful when you need to track and identify traders across various streams in a multi-node trading system.

Streams prefix

The streamsprefix string enables you to group all streams for a single trader instance or organize messages for multiple instances. Configure this by passing a string to the streamsprefix configuration option, ensuring other prefixes are set to false.

Stream per topic

Indicates whether the producer will write a separate stream for each topic. This is particularly useful for Redis backings, which do not support wildcard topics when listening to streams.
If set to False, all messages will be written to the same stream.

:::info
Redis does not support wildcard stream topics. For better compatibility with Redis, it is recommended to set this option to False.
:::

Types filtering

When messages are published on the message bus, they are serialized and written to a stream if a backing
for the message bus is configured and enabled. To prevent flooding the stream with data like

high-frequency
quotes, you may filter out certain types of messages from external publication.

To enable this filtering mechanism, pass a list of type objects to the typesfilter parameter in the message bus configuration,
specifying which types of messages should be excluded from external publication.

Stream auto-trimming

The autotrimmins configuration parameter allows you to specify the lookback window in minutes for automatic stream trimming in your message streams.
Automatic stream trimming helps manage the size of your message streams by removing older messages, ensuring that the streams remain manageable in terms of storage and performance.

:::info
The current Redis implementation will maintain the autotrimmins as a maximum width (plus roughly a minute, as streams are trimmed no more than once per minute).
Rather than a maximum lookback window based on the current wall clock time.
:::

External streams

The message bus within a TradingNode (node) is referred to as the "internal message bus".
A producer node is one which publishes messages onto an external stream (see [external publishing](#external-publishing)).
The consumer node listens to external streams to receive and publish deserialized message payloads on its internal message bus.

:::tip
Set the LiveDataEngineConfig.externalclients with the list of clientids intended to represent the external streaming clients.
The DataEngine will filter out subscription commands for these clients, ensuring that the external streaming provides the necessary data for any subscriptions to these clients.
:::

Example configuration

The following example details a streaming setup where a producer node publishes Binance data externally,
and a downstream consumer node publishes these data messages onto its internal message bus.

Producer node

We configure the MessageBus of the producer node to publish to a "binance" stream.
The settings usetraderid, usetraderprefix, and useinstanceid are all set to False
to ensure a simple and predictable stream key that the consumer nodes can register for.

Consumer node

We configure the MessageBus of the consumer node to receive messages from the same "binance" stream.
The node will listen to the external stream keys to publish these messages onto its internal message bus.
Additionally, we declare the client ID "BINANCEEXT" as an external client. This ensures that the
DataEngine does not attempt to send data commands to this client ID, as we expect these messages to be
published onto the internal message bus from the external stream, to which the node has subscribed to the relevant topics.

concepts/orders.md

Orders

This guide provides further details about the available order types for the platform, along with the execution instructions supported for each.

Orders are one of the fundamental building blocks of any algorithmic trading strategy.
NautilusTrader supports a broad set of order types and execution instructions, from standard to advanced,
exposing as much of a trading venue's functionality as possible. This enables traders to define instructions
and contingencies for order execution and management, facilitating the creation of virtually any trading strategy.

Overview

All order types are derived from two fundamentals: Market and Limit orders. In terms of liquidity, they are opposites.
Market orders consume liquidity by executing immediately at the best available price, whereas Limit orders provide liquidity by resting in the order book at a specified price until matched.

The order types available for the platform are (using the enum values):

- MARKET
- LIMIT
- STOPMARKET
- STOPLIMIT
- MARKETTOLIMIT
- MARKETIFTOUCHED
- LIMITIFTOUCHED
- TRAILINGSTOPMARKET
- TRAILINGSTOPLIMIT

:::info
NautilusTrader provides a unified API for many order types and execution instructions, but not all venues support every option.
If an order contains an instruction or option that the target venue does not support, the system should not submit the order;
instead, it logs an error with a clear explanatory message.
:::

Terminology

- An order is aggressive if its type is MARKET or if it executes as a marketable order (i.e., takes liquidity).
- An order is passive if it is not marketable (i.e., provides liquidity).
- An order is active local if it remains within the local system boundary in one of the following three non-terminal statuses:
  - INITIALIZED
  - EMULATED
  - RELEASED
- An order is in-flight when at one of the following statuses:
  - SUBMITTED
  - PENDINGUPDATE
  - PENDINGCANCEL

- An order is open when at one of the following (non-terminal) statuses:
  - ACCEPTED
  - TRIGGERED
  - PENDINGUPDATE
  - PENDINGCANCEL
  - PARTIALLYFILLED
- An order is closed when at one of the following (terminal) statuses:
  - DENIED
  - REJECTED
  - CANCELED
  - EXPIRED
  - FILLED

Execution instructions

Certain venues allow a trader to specify conditions and restrictions on how an order will be processed and executed. The following is a brief summary of the different execution instructions available.

Time in force

The order's time in force specifies how long the order will remain open or active before any remaining quantity is canceled.

- GTC (Good Till Cancel): The order remains active until canceled by the trader or the venue.
- IOC (Immediate or Cancel / Fill and Kill): The order executes immediately, with any unfilled portion canceled.
- FOK (Fill or Kill): The order executes immediately in full or not at all.
- GTD (Good Till Date): The order remains active until a specified expiration date and time.
- DAY (Good for session/day): The order remains active until the end of the current trading session.
- ATTHEOPEN (OPG): The order is only active at the open of the trading session.
- ATTHECLOSE: The order is only active at the close of the trading session.

Expire time

This instruction is to be used in conjunction with the GTD time in force to specify the time at which the order will expire and be removed from the venue's order book (or order management system).

Post-only

An order which is marked as postonly will only ever participate in providing liquidity to the limit order book, and never initiating a trade which takes liquidity as an aggressor. This option is important for market makers, or traders seeking to restrict the order to a liquidity maker fee tier.

Reduce-only

An order which is set as reduceonly will only ever reduce an existing position on an instrument and never open a new position (if already flat). The exact behavior of this instruction can vary between venues.

However, the behavior in the Nautilus SimulatedExchange is typical of a real venue.

- Order will be canceled if the associated position is closed (becomes flat).
- Order quantity will be reduced as the associated position's size decreases.

Display quantity

The displayqty specifies the portion of a Limit order which is displayed on the limit order book.

These are also known as iceberg orders as there is a visible portion to be displayed, with more quantity which is hidden.
Specifying a display quantity of zero is also equivalent to setting an order as hidden.

Trigger type

Also known as [trigger method](https://guides.interactivebrokers.com/tws/usersguidebook/configuretws/modifythestoptriggermethod.htm)
which is applicable to conditional trigger orders, specifying the method of triggering the stop price.

- DEFAULT: The default trigger type for the venue (typically LAST or BIDASK).
- LAST: The trigger price will be based on the last traded price.
- BIDASK: The trigger price will be based on the BID for buy orders and ASK for sell orders.
- DOUBLELAST: The trigger price will be based on the last two consecutive LAST prices.
- DOUBLEBIDASK: The trigger price will be based on the last two consecutive BID or ASK prices as applicable.
- LASTORBIDASK: The trigger price will be based on the LAST or BID/ASK.
- MIDPOINT: The trigger price will be based on the mid-point between the BID and ASK.
- MARK: The trigger price will be based on the venue's mark price for the instrument.
- INDEX: The trigger price will be based on the venue's index price for the instrument.

Trigger offset type

Applicable to conditional trailing-stop trigger orders, specifies the method of triggering modification of the stop price based on the offset from the market (bid, ask or last price as applicable).

- DEFAULT: The default offset type for the venue (typically PRICE).
- PRICE: The offset is based on a price difference.
- BASISPOINTS: The offset is based on a price percentage difference expressed in basis points (100bp = 1%).
- TICKS: The offset is based on a number of ticks.
- PRICETIER: The offset is based on a venue-specific price tier.

Contingent orders

More advanced relationships can be specified between orders.
For example, child orders can be assigned to trigger only when the parent is activated or filled, or orders can be
linked so that one cancels or reduces the quantity of another. See the [Advanced Orders](#advanced-orders) section for more details.

Order factory

The easiest way to create new orders is by using the built-in OrderFactory, which is automatically attached to every Strategy class. This factory will take care of lower level details - such as ensuring the correct trader ID and strategy ID are assigned, generation of a necessary initialization ID and timestamp, and abstracts away parameters which don't necessarily apply to the order type being created, or are only needed to specify more advanced execution instructions.

This leaves the factory with simpler order creation methods to work with, all the examples will leverage an OrderFactory from within a Strategy context.

:::info
See the OrderFactory [API Reference](../apireference/common.md#class-orderfactory) for further details.
:::

Order Types

The following describes the order types which are available for the platform with a code example.
Any optional parameters will be clearly marked with a comment which includes the default value.

Market

A Market order is an instruction by the trader to immediately trade
the given quantity at the best price available. You can also specify several
time in force options, and indicate whether this order is only intended to reduce
a position.

In the following example we create a Market order on the Interactive Brokers [IdealPro](https://ibkr.info/node/1708) Forex ECN
to BUY 100,000 AUD using USD:

:::info
See the MarketOrder [API Reference](../apireference/model/orders.md#class-marketorder) for further details.
:::

Limit

A Limit order is placed on the limit order book at a specific price, and will only
execute at that price (or better).

In the following example we create a Limit order on the Binance Futures Crypto exchange to SELL 20 ETHUSDT-PERP Perpetual Futures
contracts at a limit price of 5000 USDT, as a market maker.

:::info
See the LimitOrder [API Reference](../apireference/model/orders.md#class-limitorder) for further details.
:::

Stop-Market

A Stop-Market order is a conditional order which once triggered, will immediately
place a Market order. This order type is often used as a stop-loss to limit losses, either
as a SELL order against LONG positions, or as a BUY order against SHORT positions.

In the following example we create a Stop-Market order on the Binance Spot/Margin exchange
to SELL 1 BTC at a trigger price of 100,000 USDT, active until further notice:

:::info
See the StopMarketOrder [API Reference](../apireference/model/orders.md#class-stopmarketorder) for further details.
:::

Stop-Limit

A Stop-Limit order is a conditional order which once triggered will immediately place
a Limit order at the specified price.

In the following example we create a Stop-Limit order on the Currenex FX ECN to BUY 50,000 GBP at a limit price of 1.3000 USD
once the market hits the trigger price of 1.30010 USD, active until midday 6th June, 2022 (UTC):

:::info
See the StopLimitOrder [API Reference](../apireference/model/orders.md#class-stoplimitorder) for

further details.
:::

Market-To-Limit

A Market-To-Limit order is submitted as a market order to execute at the current best market price.
If the order is only partially filled, the remainder of the order is canceled and re-submitted as a Limit order with
the limit price equal to the price at which the filled portion of the order executed.

In the following example we create a Market-To-Limit order on the Interactive Brokers [IdealPro](https://ibkr.info/node/1708) Forex ECN
to BUY 200,000 USD using JPY:

:::info
See the MarketToLimitOrder [API Reference](../apireference/model/orders.md#class-markettolimitorder) for further details.
:::

Market-If-Touched

A Market-If-Touched order is a conditional order which once triggered will immediately
place a Market order. This order type is often used to enter a new position on a stop price,
or to take profits for an existing position, either as a SELL order against LONG positions,
or as a BUY order against SHORT positions.

In the following example we create a Market-If-Touched order on the Binance Futures exchange
to SELL 10 ETHUSDT-PERP Perpetual Futures contracts at a trigger price of 10,000 USDT, active until further notice:

:::info
See the MarketIfTouchedOrder [API Reference](../apireference/model/orders.md#class-marketiftouchedorder) for further details.
:::

Limit-If-Touched

A Limit-If-Touched order is a conditional order which once triggered will immediately place
a Limit order at the specified price.

In the following example we create a Limit-If-Touched order to BUY 5 BTCUSDT-PERP Perpetual Futures contracts on the
Binance Futures exchange at a limit price of 30,100 USDT (once the market hits the trigger price of 30,150 USDT),
active until midday 6th June, 2022 (UTC):

:::info
See the LimitIfTouched [API Reference](../apireference/model/orders.md#class-limitiftouchedorder-1) for further details.
:::

Trailing-Stop-Market

A Trailing-Stop-Market order is a conditional order which trails a stop trigger price
a fixed offset away from the defined market price. Once triggered a Market order will
immediately be placed.

In the following example we create a Trailing-Stop-Market order on the Binance Futures exchange to SELL 10 ETHUSD-PERP COINM margined

Perpetual Futures Contracts activating at a price of 5,000 USD, then trailing at an offset of 1% (in basis points) away from the current last traded price:

:::info
See the TrailingStopMarketOrder [API Reference](../apireference/model/orders.md#class-trailingstopmarketorder-1) for further details.
:::

Trailing-Stop-Limit

A Trailing-Stop-Limit order is a conditional order which trails a stop trigger price
a fixed offset away from the defined market price. Once triggered a Limit order will
immediately be placed at the defined price (which is also updated as the market moves until triggered).

In the following example we create a Trailing-Stop-Limit order on the Currenex FX ECN to BUY 1,250,000 AUD using USD
at a limit price of 0.71000 USD, activating at 0.72000 USD then trailing at a stop offset of 0.00100 USD away from the current ask price, active until further notice:

:::info
See the TrailingStopLimitOrder [API Reference](../apireference/model/orders.md#class-trailingstoplimitorder-1) for further details.
:::

Advanced Orders

The following guide should be read in conjunction with the specific documentation from the broker or venue
involving these order types, lists/groups and execution instructions (such as for Interactive Brokers).

Order Lists

Combinations of contingent orders, or larger order bulks can be grouped together into a list with a common
orderlistid. The orders contained in this list may or may not have a contingent relationship with
each other, as this is specific to how the orders themselves are constructed, and the
specific venue they are being routed to.

Contingency Types

- OTO (One-Triggers-Other) - a parent order that, once executed, automatically places one or more child orders.
  - Full-trigger model: child order(s) are released only after the parent is completely filled. Common at most retail equity/option brokers (e.g. Schwab, Fidelity, TD Ameritrade) and many spot-crypto venues (Binance, Coinbase).
  - Partial-trigger model: child order(s) are released pro-rata to each partial fill. Used by professional-grade platforms such as Interactive Brokers, most futures/FX OMSs, and Kraken Pro.

- OCO (One-Cancels-Other) - two (or more) linked live orders where executing one cancels the remainder.

- OUO (One-Updates-Other) - two (or more) linked live orders where executing one reduces the open quantity of the remainder.

:::info
These contingency types relate to ContingencyType FIX tag <1385> <https://www.onixs.biz/fix-dictionary/5.0.sp2/tagnum1385.html>.
:::

One-Triggers-Other (OTO)

An OTO order involves two parts:

1. Parent order - submitted to the matching engine immediately.
2. Child order(s) - held off-book until the trigger condition is met.

Trigger models

| Trigger model | When are child orders released?
|
|---|---|
| Full trigger | When the parent order's cumulative quantity equals its original quantity (i.e., it is fully filled). |
| Partial trigger | Immediately upon each partial execution of the parent; the child's quantity matches the executed amount and is increased as further fills occur. |

:::info
The default backtest venue for NautilusTrader uses a partial-trigger model for OTO orders.
A future update will add configuration to opt-in to a full-trigger model.
:::

> Why the distinction matters
> Full trigger leaves a risk window: any partially filled position is live without its protective exit until the remaining quantity fills.
> Partial trigger mitigates that risk by ensuring every executed lot instantly has its linked stop/limit, at the cost of creating more order traffic and updates.

An OTO order can use any supported asset type on the venue (e.g. stock entry with option hedge, futures entry with OCO bracket, crypto spot entry with TP/SL).

Venue / Adapter ID	Asset classes	Trigger rule for child	Practical notes
Binance / Binance Futures (BINANCE)	Spot, perpetual futures	Partial or full - fires on first fill.	OTOCO/TP-SL children appear instantly; monitor margin usage.
Bybit Spot (BYBIT)	Spot	Full - child placed after completion.	TP-SL preset activates only once the limit order is fully filled.
Bybit Perps (BYBIT)	Perpetual futures	Partial and full - configurable.	"Partial-position" mode sizes TP-SL as fills arrive.
Kraken Futures (KRAKEN)	Futures & perps	Partial and full - automatic.	Child quantity matches every partial execution.
OKX (OKX)	Spot, futures, options	Full - attached stop waits for fill.	Position-level TP-SL can be added separately.
Interactive Brokers (INTERACTIVEBROKERS)	Stocks, options, FX, fut	Configurable - OCA can pro-rate.	OcaType 2/3 reduces remaining child quantities.
Coinbase International (COINBASEINTX)	Spot & perps	Full - bracket added post-execution.	Entry plus bracket not simultaneous; added once position is live.
dYdX v4 (DYDX)	Perpetual futures (DEX)	On-chain condition (size exact).	TP-SL triggers by oracle price; partial fill not applicable.
Polymarket (POLYMARKET)	Prediction market (DEX)	N/A.	Advanced contingency handled entirely at the strategy layer.
Betfair (BETFAIR)	Sports betting	N/A.	Advanced contingency handled entirely at the strategy layer.

One-Cancels-Other (OCO)

An OCO order is a set of linked orders where the execution of any order (full or partial) triggers a best-efforts cancellation of the others.
Both orders are live simultaneously; once one starts filling, the venue attempts to cancel the unexecuted portion of the remainder.

One-Updates-Other (OUO)

An OUO order is a set of linked orders where execution of one order causes an immediate reduction of open quantity in the other order(s).
Both orders are live concurrently, and each partial execution proportionally updates the remaining quantity of its peer order on a best-effort basis.

Bracket Orders

Bracket orders are an advanced order type that allows traders to set both take-profit and stop-loss levels for a position simultaneously. This involves placing a parent order (entry order) and two child orders: a take-profit LIMIT order and a stop-loss STOPMARKET order. When the parent order is executed,
the child orders are placed in the market. The take-profit order closes the position with profits if the market moves favorably, while the stop-loss order limits losses if the market moves unfavorably.

Bracket orders can be easily created using the [OrderFactory](../apireference/common.md#class-orderfactory),
which supports various order types, parameters, and instructions.

:::warning
You should be aware of the margin requirements of positions, as bracketing a position will consume more order margin.
:::

Emulated Orders

Introduction

Before diving into the technical details, it's important to understand the fundamental purpose of emulated orders
in Nautilus Trader. At its core, emulation allows you to use certain order types even when your trading venue
doesn't natively support them.

This works by having Nautilus locally mimic the behavior of these order types (such as STOPLIMIT or TRAILINGSTOP orders)
locally, while using only simple MARKET and LIMIT orders for actual execution on the venue.

When you create an emulated order, Nautilus continuously tracks a specific type of market price (specified by the
emulationtrigger parameter) and based on the order type and conditions you've set, will automatically submit
the appropriate fundamental order (MARKET / LIMIT) when the triggering condition is met.

For example, if you create an emulated STOPLIMIT order, Nautilus will monitor the market price until your stop
price is reached, and then automatically submits a LIMIT order to the venue.

To perform emulation, Nautilus needs to know which type of market price it should monitor.
By default, it uses bid and ask prices (quotes), which is why you'll often see emulationtrigger=TriggerType.DEFAULT

in examples (this is equivalent to using TriggerType.BIDASK). However, Nautilus supports various other price types,
that can guide the emulation process.

Submitting order for emulation

The only requirement to emulate an order is to pass a TriggerType to the emulationtrigger parameter of an Order constructor, or OrderFactory creation method. The following emulation trigger types are currently supported:

- NOTRIGGER: disables local emulation completely and order is fully submitted to the venue.
- DEFAULT: which is the same as BIDASK.
- BIDASK: emulated using quotes to trigger.
- LAST: emulated using trades to trigger.

The choice of trigger type determines how the order emulation will behave:

- For STOP orders, the trigger price of order will be compared against the specified trigger type.
- For TRAILINGSTOP orders, the trailing offset will be updated based on the specified trigger type.
- For LIMIT orders, the limit price of order will be compared against the specified trigger type.

Here are all the available values you can set into emulationtrigger parameter and their purposes:

Trigger Type	Description	Common use cases
NOTRIGGER	Disables emulation completely. The order is sent directly to the venue without any local processing.	When you want to use the venue's native order handling, or for simple order types that don't need emulation.
DEFAULT	Same as BIDASK. This is the standard choice for most emulated orders.	General-purpose emulation when you want to work with the "default" type of market prices.
BIDASK	Uses the best bid and ask prices (quotes) to guide emulation.	Stop orders, trailing stops, and other orders that should react to the current market spread.
LASTPRICE	Uses the price of the most recent trade to guide emulation.	Orders that should trigger based on actual executed trades rather than quotes.
DOUBLELAST	Uses two consecutive last trade prices to confirm the trigger condition.	When you want additional confirmation of price movement before triggering.
DOUBLEBIDASK	Uses two consecutive bid/ask price updates to confirm the trigger condition.	When you want extra confirmation of quote movements before triggering.
LASTORBIDASK	Triggers on either last trade price or bid/ask prices.	When you want to be more responsive to any type of price movement.
MIDPOINT	Uses the middle point between the best bid and ask prices.	Orders that should trigger based on the theoretical fair price.
MARKPRICE	Uses the mark price (common in derivatives markets) for triggering.	Particularly useful for futures and perpetual contracts.
INDEXPRICE	Uses an underlying index price for triggering.	When trading derivatives that track an index.

Technical implementation

The platform makes it possible to emulate most order types locally, regardless of whether the type is supported on a trading venue. The logic and code paths for order emulation are exactly the same for all [environment

contexts](/concepts/architecture.md#environment-contexts)
and utilize a common OrderEmulator component.

:::note
There is no limitation on the number of emulated orders you can have per running instance.
:::

Life cycle

An emulated order will progress through the following stages:

1. Submitted by a Strategy through the submitorder method.
2. Sent to the RiskEngine for pre-trade risk checks (it may be denied at this point).
3. Sent to the OrderEmulator where it is held / emulated.
4. Once triggered, emulated order is transformed into a MARKET or LIMIT order and released (submitted to the venue).
5. Released order undergoes final risk checks before venue submission.

:::note
Emulated orders are subject to the same risk controls as regular orders, and can be
modified and canceled by a trading strategy in the normal way. They will also be included
when canceling all orders.
:::

:::info
An emulated order will retain its original client order ID throughout its entire life cycle, making it easy to query
through the cache.
:::

Held emulated orders

The following will occur for an emulated order now held by the OrderEmulator component:

- The original SubmitOrder command will be cached.
- The emulated order will be processed inside a local MatchingCore component.
- The OrderEmulator will subscribe to any needed market data (if not already) to update the matching core.
- The emulated order can be modified (by the trader) and updated (by the market) until released or canceled.

Released emulated orders

Once an emulated order is triggered / matched locally based on the arrival of data, the following
release actions will occur:

- The order will be transformed to either a MARKET or LIMIT order (see below table) through an additional OrderInitialized event.
- The orders emulationtrigger will be set to NONE (it will no longer be treated as an emulated order by any component).
- The order attached to the original SubmitOrder command will be sent back to the RiskEngine for additional checks since any modification/updates.
- If not denied, then the command will continue to the ExecutionEngine and on to the trading venue via an ExecutionClient as normal.

The following table lists which order types are possible to emulate, and
which order type they transform to when being released for submission to the
trading venue.

Order types, which can be emulated

The following table lists which order types are possible to emulate, and
which order type they transform to when being released for submission to the
trading venue.

Order type for emulation	Can emulate	Released type
MARKET		n/a
MARKETTOLIMIT		n/a
LIMIT		MARKET
STOPMARKET		MARKET
STOPLIMIT		LIMIT
MARKETIFTOUCHED		MARKET
LIMITIFTOUCHED		LIMIT
TRAILINGSTOPMARKET		MARKET
TRAILINGSTOPLIMIT		LIMIT

Querying

When writing trading strategies, it may be necessary to know the state of emulated orders in the system.
There are several ways to query emulation status:

Through the Cache

The following Cache methods are available:

- self.cache.ordersemulated(...): Returns all currently emulated orders.
- self.cache.isorderemulated(...): Checks if a specific order is emulated.
- self.cache.ordersemulatedcount(...): Returns the count of emulated orders.

See the full [API reference](../../apireference/cache) for additional details.

Direct order queries

You can query order objects directly using:

- order.isemulated

If either of these return False, then the order has been released from the
OrderEmulator, and so is no longer considered an emulated order (or was never an emulated order).

:::warning
It's not advised to hold a local reference to an emulated order, as the order
object will be transformed when/if the emulated order is released. You should rely
on the Cache which is made for the job.
:::

Persistence and recovery

If a running system either crashes or shuts down with active emulated orders, then
they will be reloaded inside the OrderEmulator from any configured cache database.
This ensures order state persistence across system restarts and recoveries.

Best practices

When working with emulated orders, consider the following best practices:

1. Always use the Cache for querying or tracking emulated orders rather than storing local references

2. Be aware that emulated orders transform to different types when released
3. Remember that emulated orders undergo risk checks both at submission and release

:::note
Order emulation allows you to use advanced order types even on venues that don't natively support them,
making your trading strategies more portable across different venues.
:::

concepts/overview.md

Overview

Introduction

NautilusTrader is an open-source, high-performance, production-grade algorithmic trading platform, providing quantitative traders with the ability to backtest portfolios of automated trading strategies on historical data with an event-driven engine, and also deploy those same strategies live, with no code changes.

The platform is AI-first, designed to develop and deploy algorithmic trading strategies within a highly performant and robust Python-native environment. This helps to address the parity challenge of keeping the Python research/backtest environment consistent with the production live trading environment.

NautilusTrader's design, architecture, and implementation philosophy prioritizes software correctness and safety at the highest level, with the aim of supporting Python-native, mission-critical, trading system backtesting and live deployment workloads.

The platform is also universal and asset-class-agnostic - with any REST API or WebSocket stream able to be integrated via modular adapters. It supports high-frequency trading across a wide range of asset classes and instrument types including FX, Equities, Futures, Options, Crypto and Betting, enabling seamless operations across multiple venues simultaneously.

Features

- Fast: Core is written in Rust with asynchronous networking using [tokio](https://crates.io/crates/tokio).
- Reliable: Rust-powered type- and thread-safety, with optional Redis-backed state persistence.
- Portable: OS independent, runs on Linux, macOS, and Windows. Deploy using Docker.
- Flexible: Modular adapters mean any REST API or WebSocket stream can be integrated.
- Advanced: Time in force IOC, FOK, GTC, GTD, DAY, ATTHEOPEN, ATTHECLOSE, advanced order types and conditional triggers. Execution instructions post-only, reduce-only, and icebergs. Contingency orders including OCO, OUO, OTO.
- Customizable: Add user-defined custom components, or assemble entire systems from scratch leveraging the [cache](cache.md) and [message bus](messagebus.md).
- Backtesting: Run with multiple venues, instruments and strategies simultaneously using historical quote tick, trade tick, bar, order book and custom data with nanosecond resolution.
- Live: Use identical strategy implementations between backtesting and live deployments.
- Multi-venue: Multiple venue capabilities facilitate market-making and statistical arbitrage strategies.
- AI Training: Backtest engine fast enough to be used to train AI trading agents (RL/ES).

> nautilus - from ancient Greek 'sailor' and naus 'ship'.
>
> The nautilus shell consists of modular chambers with a growth factor which approximates a logarithmic spiral.
> The idea is that this can be translated to the aesthetics of design and architecture.

Why NautilusTrader?

- Highly performant event-driven Python: Native binary core components.

- Parity between backtesting and live trading: Identical strategy code.
- Reduced operational risk: Enhanced risk management functionality, logical accuracy, and type safety.
- Highly extendable: Message bus, custom components and actors, custom data, custom adapters.

Traditionally, trading strategy research and backtesting might be conducted in Python
using vectorized methods, with the strategy then needing to be reimplemented in a more event-driven way
using C++, C#, Java or other statically typed language(s). The reasoning here is that vectorized backtesting code cannot
express the granular time and event dependent complexity of real-time trading, where compiled languages have
proven to be more suitable due to their inherently higher performance, and type safety.

One of the key advantages of NautilusTrader here, is that this reimplementation step is now circumvented - as the critical core components of the platform
have all been written entirely in [Rust](https://www.rust-lang.org/) or [Cython](https://cython.org/).
This means we're using the right tools for the job, where systems programming languages compile performant binaries,
with CPython C extension modules then able to offer a Python-native environment, suitable for professional quantitative traders and trading firms.

Use Cases

There are three main use cases for this software package:

- Backtest trading systems on historical data (backtest).
- Simulate trading systems with real-time data and virtual execution (sandbox).
- Deploy trading systems live on real or paper accounts (live).

The project's codebase provides a framework for implementing the software layer of systems which achieve the above. You will find
the default backtest and live system implementations in their respectively named subpackages. A sandbox environment can
be built using the sandbox adapter.

:::note

- All examples will utilize these default system implementations.
- We consider trading strategies to be subcomponents of end-to-end trading systems, these systems include the application and infrastructure layers.

:::

Distributed

The platform is designed to be easily integrated into a larger distributed system.
To facilitate this, nearly all configuration and domain objects can be serialized using JSON, MessagePack or Apache Arrow (Feather) for communication over the network.

Common Core

The common system core is utilized by all node [environment contexts](/concepts/architecture.md#environment-contexts) (backtest, sandbox, and live).
User-defined Actor, Strategy and ExecAlgorithm components are managed consistently across these environment contexts.

Backtesting

Backtesting can be achieved by first making data available to a BacktestEngine either directly or via

a higher level BacktestNode and ParquetDataCatalog, and then running the data through the system with nanosecond resolution.

Live Trading

A TradingNode can ingest data and events from multiple data and execution clients.
Live deployments can use both demo/paper trading accounts, or real accounts.

For live trading, a TradingNode can ingest data and events from multiple data and execution clients.
The platform supports both demo/paper trading accounts and real accounts. High performance can be achieved by running
asynchronously on a single [event loop](https://docs.python.org/3/library/asyncio-eventloop.html),
with the potential to further boost performance by leveraging the [uvloop](https://github.com/MagicStack/uvloop) implementation (available for Linux and macOS).

Domain Model

The platform features a comprehensive trading domain model that includes various value types such as Price and Quantity, as well as more complex entities such as Order and Position objects,
which are used to aggregate multiple events to determine state.

Timestamps

All timestamps within the platform are recorded at nanosecond precision in UTC.

Timestamp strings follow ISO 8601 (RFC 3339) format with either 9 digits (nanoseconds) or 3 digits (milliseconds) of decimal precision,
(but mostly nanoseconds) always maintaining all digits including trailing zeros.
These can be seen in log messages, and debug/display outputs for objects.

A timestamp string consists of:

- Full date component always present: YYYY-MM-DD.
- T separator between date and time components.
- Always nanosecond precision (9 decimal places) or millisecond precision (3 decimal places) for certain cases such as GTD expiry times.
- Always UTC timezone designated by Z suffix.

Example: 2024-01-05T15:30:45.123456789Z

For the complete specification, refer to [RFC 3339: Date and Time on the Internet](https://datatracker.ietf.org/doc/html/rfc3339).

UUIDs

The platform uses Universally Unique Identifiers (UUID) version 4 (RFC 4122) for unique identifiers.
Our high-performance implementation leverages the uuid crate for correctness validation when parsing from strings,
ensuring input UUIDs comply with the specification.

A valid UUID v4 consists of:

- 32 hexadecimal digits displayed in 5 groups.
- Groups separated by hyphens: 8-4-4-4-12 format.
- Version 4 designation (indicated by the third group starting with "4").
- RFC 4122 variant designation (indicated by the fourth group starting with "8", "9", "a", or "b").

Example: 2d89666b-1a1e-4a75-b193-4eb3b454c757

For the complete specification, refer to [RFC 4122: A Universally Unique IDentifier (UUID) URN

Namespace](https://datatracker.ietf.org/doc/html/rfc4122).

Data Types

The following market data types can be requested historically, and also subscribed to as live streams when available from a venue / data provider, and implemented in an integrations adapter.

- OrderBookDelta (L1/L2/L3)
- OrderBookDeltas (container type)
- OrderBookDepth10 (fixed depth of 10 levels per side)
- QuoteTick
- TradeTick
- Bar
- Instrument
- InstrumentStatus
- InstrumentClose

The following PriceType options can be used for bar aggregations:

- BID
- ASK
- MID
- LAST

Bar Aggregations

The following BarAggregation methods are available:

- MILLISECOND
- SECOND
- MINUTE
- HOUR
- DAY
- WEEK
- MONTH
- TICK
- VOLUME
- VALUE (a.k.a Dollar bars)
- TICKIMBALANCE
- TICKRUNS
- VOLUMEIMBALANCE
- VOLUMERUNS
- VALUEIMBALANCE
- VALUERUNS

The price types and bar aggregations can be combined with step sizes >= 1 in any way through a BarSpecification.
This enables maximum flexibility and now allows alternative bars to be aggregated for live trading.

Account Types

The following account types are available for both live and backtest environments:

- Cash single-currency (base currency)
- Cash multi-currency
- Margin single-currency (base currency)
- Margin multi-currency
- Betting single-currency

Order Types

The following order types are available (when possible on a venue):

- MARKET
- LIMIT
- STOPMARKET
- STOPLIMIT
- MARKETTOLIMIT
- MARKETIFTOUCHED
- LIMITIFTOUCHED
- TRAILINGSTOPMARKET
- TRAILINGSTOPLIMIT

Value Types

The following value types are backed by either 128-bit or 64-bit raw integer values, depending on the [precision mode](../gettingstarted/installation.md#precision-mode) used during compilation.

- Price
- Quantity
- Money

High-precision mode (128-bit)

When the high-precision feature flag is enabled (default), values use the specification:

Type	Raw backing	Max precision	Min value	Max value
Price	i128	16	-17,014,118,346,046	17,014,118,346,046
Money	i128	16	-17,014,118,346,046	17,014,118,346,046
Quantity	u128	16	0	34,028,236,692,093

Standard-precision mode (64-bit)

When the high-precision feature flag is disabled, values use the specification:

Type	Raw backing	Max precision	Min value	Max value
Price	i64	9	-9,223,372,036	9,223,372,036
Money	i64	9	-9,223,372,036	9,223,372,036
Quantity	u64	9	0	18,446,744,073

concepts/portfolio.md

Portfolio

:::info
We are currently working on this concept guide.
:::

The Portfolio serves as the central hub for managing and tracking all positions across active strategies for the trading node or backtest.
It consolidates position data from multiple instruments, providing a unified view of your holdings, risk exposure, and overall performance.
Explore this section to understand how NautilusTrader aggregates and updates portfolio state to support effective trading and risk management.

Portfolio Statistics

There are a variety of [built-in portfolio statistics](https://github.com/nautechsystems/nautilustrader/tree/develop/nautilustrader/analysis/statistics)
which are used to analyse a trading portfolios performance for both backtests and live trading.

The statistics are generally categorized as follows.

- PnLs based statistics (per currency)
- Returns based statistics
- Positions based statistics
- Orders based statistics

It's also possible to call a traders PortfolioAnalyzer and calculate statistics at any arbitrary
time, including during a backtest, or live trading session.

Custom Statistics

Custom portfolio statistics can be defined by inheriting from the PortfolioStatistic
base class, and implementing any of the calculate methods.

For example, the following is the implementation for the built-in WinRate statistic:

These statistics can then be registered with a traders PortfolioAnalyzer.

:::tip
Ensure your statistic is robust to degenerate inputs such as None, empty series, or insufficient data.

The expectation is that you would then return None, NaN or a reasonable default.
:::

Backtest Analysis

Following a backtest run, a performance analysis will be carried out by passing realized PnLs, returns, positions and orders data to each registered
statistic in turn, calculating their values (with a default configuration). Any output is then displayed in the tear sheet
under the Portfolio Performance heading, grouped as.

- Realized PnL statistics (per currency)
- Returns statistics (for the entire portfolio)
- General statistics derived from position and order data (for the entire portfolio)

concepts/strategies.md

Strategies

The heart of the NautilusTrader user experience is in writing and working with
trading strategies. Defining a strategy involves inheriting the Strategy class and
implementing the methods required by the strategy's logic.

Key capabilities:

- All Actor capabilities
- Order management

Relationship with actors:
The Strategy class inherits from Actor, which means strategies have access to all actor functionality
plus order management capabilities.

:::tip
We recommend reviewing the [Actors](actors.md) guide before diving into strategy development.
:::

Strategies can be added to Nautilus systems in any [environment contexts](/concepts/architecture.md#environment-contexts) and will start sending commands and receiving
events based on their logic as soon as the system starts.

Using the basic building blocks of data ingest, event handling, and order management (which we will discuss
below), it's possible to implement any type of strategy including directional, momentum, re-balancing,
pairs, market making etc.

:::info
See the Strategy [API Reference](../apireference/trading.md) for a complete description
of all available methods.
:::

There are two main parts of a Nautilus trading strategy:

- The strategy implementation itself, defined by inheriting the Strategy class
- The optional strategy configuration, defined by inheriting the StrategyConfig class

:::tip
Once a strategy is defined, the same source code can be used for backtesting and live trading.
:::

The main capabilities of a strategy include:

- Historical data requests
- Live data feed subscriptions
- Setting time alerts or timers
- Cache access
- Portfolio access
- Creating and managing orders and positions

Strategy implementation

Since a trading strategy is a class which inherits from Strategy, you must define
a constructor where you can handle initialization. Minimally the base/super class needs to be initialized:

From here, you can implement handlers as necessary to perform actions based on state transitions and events.

:::warning
Do not call components such as clock and logger in the init constructor (which is prior to registration). This is because the systems clock and logging subsystem have not yet been initialized.
:::

Handlers

Handlers are methods within the Strategy class which may perform actions based on different types of events or on state changes.
These methods are named with the prefix on. You can choose to implement any or all of these handler methods depending on the specific goals and needs of your strategy.

The purpose of having multiple handlers for similar types of events is to provide flexibility in handling granularity.
This means that you can choose to respond to specific events with a dedicated handler, or use a more generic
handler to react to a range of related events (using typical switch statement logic).
The handlers are called in sequence from the most specific to the most general.

Stateful actions

These handlers are triggered by lifecycle state changes of the Strategy. It's recommended to:

- Use the onstart method to initialize your strategy (e.g., fetch instruments, subscribe to data)
- Use the onstop method for cleanup tasks (e.g., cancel open orders, close open positions, unsubscribe from data)

Data handling

These handlers receive data updates, including built-in market data and custom user-defined data.
You can use these handlers to define actions upon receiving data object instances.

Order management

These handlers receive events related to orders.
OrderEvent type messages are passed to handlers in the following sequence:

1. Specific handler (e.g., onorderaccepted, onorderrejected, etc.)
2. onorderevent(...)
3. onevent(...)

Position management

These handlers receive events related to positions.
PositionEvent type messages are passed to handlers in the following sequence:

1. Specific handler (e.g., onpositionopened, onpositionchanged, etc.)
2. onpositionevent(...)
3. onevent(...)

Generic event handling

This handler will eventually receive all event messages which arrive at the strategy, including those for which no other specific handler exists.

Handler example

The following example shows a typical onstart handler method implementation (taken from the example

EMA cross strategy).
Here we can see the following:

- Indicators being registered to receive bar updates
- Historical data being requested (to hydrate the indicators)
- Live data being subscribed to

Clock and timers

Strategies have access to a Clock which provides a number of methods for creating
different timestamps, as well as setting time alerts or timers to trigger TimeEvents.

:::info
See the Clock [API reference](../apireference/common.md) for a complete list of available methods.
:::

Current timestamps

While there are multiple ways to obtain current timestamps, here are two commonly used methods as examples:

To get the current UTC timestamp as a tz-aware pd.Timestamp:

To get the current UTC timestamp as nanoseconds since the UNIX epoch:

Time alerts

Time alerts can be set which will result in a TimeEvent being dispatched to the onevent handler at the specified alert time. In a live context, this might be slightly delayed by a few microseconds.

This example sets a time alert to trigger one minute from the current time:

Timers

Continuous timers can be set up which will generate a TimeEvent at regular intervals until the timer expires
or is canceled.

This example sets a timer to fire once per minute, starting immediately:

Cache access

The trader instances central Cache can be accessed to fetch data and execution objects (orders, positions etc).
There are many methods available often with filtering functionality, here we go through some basic use cases.

Fetching data

The following example shows how data can be fetched from the cache (assuming some instrument ID attribute is assigned):

Fetching execution objects

The following example shows how individual order and position objects can be fetched from the cache:

:::info
See the Cache [API Reference](../apireference/cache.md) for a complete description
of all available methods.
:::

Portfolio access

The traders central Portfolio can be accessed to fetch account and positional information.
The following shows a general outline of available methods.

Account and positional information

:::info
See the Portfolio [API Reference](../apireference/portfolio.md) for a complete description
of all available methods.
:::

Reports and analysis

The Portfolio also makes a PortfolioAnalyzer available, which can be fed with a flexible amount of data (to accommodate different lookback windows). The analyzer can provide tracking for and generating of performance
metrics and statistics.

:::info
See the PortfolioAnalyzer [API Reference](../apireference/analysis.md) for a complete description
of all available methods.
:::

:::info
See the [Portfolio statistics](../concepts/advanced/portfoliostatistics.md) guide.
:::

Trading commands

NautilusTrader offers a comprehensive suite of trading commands, enabling granular order management
tailored for algorithmic trading. These commands are essential for executing strategies, managing risk, and ensuring seamless interaction with various trading venues. In the following sections, we will
delve into the specifics of each command and its use cases.

:::info
The [Execution](../concepts/execution.md) guide explains the flow through the system, and can be helpful to read in conjunction with the below.
:::

Submitting orders

An OrderFactory is provided on the base class for every Strategy as a convenience, reducing
the amount of boilerplate required to create different Order objects (although these objects
can still be initialized directly with the Order.init(...) constructor if the trader prefers).

The component a SubmitOrder or SubmitOrderList command will flow to for execution depends on the following:

- If an emulationtrigger is specified, the command will firstly be sent to the OrderEmulator
- If an execalgorithmid is specified (with no emulationtrigger), the command will firstly be sent to the relevant ExecAlgorithm
- Otherwise, the command will firstly be sent to the RiskEngine

This example submits a LIMIT BUY order for emulation (see [OrderEmulator](advanced/emulatedorders.md)):

:::info
You can specify both order emulation and an execution algorithm.
:::

This example submits a MARKET BUY order to a TWAP execution algorithm:

Canceling orders

Orders can be canceled individually, as a batch, or all orders for an instrument (with an optional side filter).

If the order is already closed or already pending cancel, then a warning will be logged.

If the order is currently open then the status will become PENDINGCANCEL.

The component a CancelOrder, CancelAllOrders or BatchCancelOrders command will flow to for execution depends on the following:

- If the order is currently emulated, the command will firstly be sent to the OrderEmulator
- If an execalgorithmid is specified (with no emulationtrigger), and the order is still active within the local system, the command will firstly be sent to the relevant ExecAlgorithm
- Otherwise, the order will firstly be sent to the ExecutionEngine

:::info
Any managed GTD timer will also be canceled after the command has left the strategy.
:::

The following shows how to cancel an individual order:

The following shows how to cancel a batch of orders:

The following shows how to cancel all orders:

Modifying orders

Orders can be modified individually when emulated, or open on a venue (if supported).

If the order is already closed or already pending cancel, then a warning will be logged.
If the order is currently open then the status will become PENDINGUPDATE.

:::warning
At least one value must differ from the original order for the command to be valid.
:::

The component a ModifyOrder command will flow to for execution depends on the following:

- If the order is currently emulated, the command will firstly be sent to the OrderEmulator
- Otherwise, the order will firstly be sent to the RiskEngine

:::info
Once an order is under the control of an execution algorithm, it cannot be directly modified by a strategy (only canceled).
:::

The following shows how to modify the size of LIMIT BUY order currently open on a venue:

:::info
The price and trigger price can also be modified (when emulated or supported by a venue).
:::

Strategy configuration

The main purpose of a separate configuration class is to provide total flexibility
over where and how a trading strategy can be instantiated. This includes being able
to serialize strategies and their configurations over the wire, making distributed backtesting

and firing up remote live trading possible.

This configuration flexibility is actually opt-in, in that you can actually choose not to have any strategy configuration beyond the parameters you choose to pass into your strategies' constructor. If you would like to run distributed backtests or launch live trading servers remotely, then you will need to define a configuration.

Here is an example configuration:

When implementing strategies, it's recommended to access configuration values directly through self.config.
This provides clear separation between:

- Configuration data (accessed via self.config):
  - Contains initial settings, that define how the strategy works
  - Example: self.config.tradesize, self.config.instrumentid

- Strategy state variables (as direct attributes):
  - Track any custom state of the strategy
  - Example: self.timestarted, self.countofprocessedbars

This separation makes code easier to understand and maintain.

:::note
Even though it often makes sense to define a strategy which will trade a single instrument. The number of instruments a single strategy can work with is only limited by machine resources.
:::

Managed GTD expiry

It's possible for the strategy to manage expiry for orders with a time in force of GTD (Good 'till Date). This may be desirable if the exchange/broker does not support this time in force option, or for any reason you prefer the strategy to manage this.

To use this option, pass managegtdexpiry=True to your StrategyConfig. When an order is submitted with a time in force of GTD, the strategy will automatically start an internal time alert.
Once the internal GTD time alert is reached, the order will be canceled (if not already closed).

Some venues (such as Binance Futures) support the GTD time in force, so to avoid conflicts when using managedgtdexpiry you should set usegtd=False for your execution client config.

Multiple strategies

If you intend running multiple instances of the same strategy, with different configurations (such as trading different instruments), then you will need to define a unique orderidtag for each of these strategies (as shown above).

:::note
The platform has built-in safety measures in the event that two strategies share a duplicated strategy ID, then an exception will be raised that the strategy ID has already been registered.
:::

The reason for this is that the system must be able to identify which strategy various commands and events belong to. A strategy ID is made up of the strategy class name, and the strategies orderidtag separated by a hyphen. For example the above config would result in a strategy ID of MyStrategy-001.

:::note
See the StrategyId [API Reference](../apireference/model/identifiers.md) for further details.
:::

developer_guide/adapters.md

Adapters

Introduction

This developer guide provides instructions on how to develop an integration adapter for the NautilusTrader platform.
Adapters provide connectivity to trading venues and data providers-translating raw venue APIs into Nautilus's unified interface and normalized domain model.

Structure of an adapter

An adapter typically consists of several components:

1. Instrument Provider: Supplies instrument definitions
2. Data Client: Handles live market data feeds and historical data requests
3. Execution Client: Handles order execution and management
4. Configuration: Configures the client settings

Adapter implementation steps

1. Create a new Python subpackage for your adapter
2. Implement the Instrument Provider by inheriting from InstrumentProvider and implementing the necessary methods to load instruments
3. Implement the Data Client by inheriting from either the LiveDataClient and LiveMarketDataClient class as applicable, providing implementations for the required methods
4. Implement the Execution Client by inheriting from LiveExecutionClient and providing implementations for the required methods
5. Create configuration classes to hold your adapter's settings
6. Test your adapter thoroughly to ensure all methods are correctly implemented and the adapter works as expected (see the [Testing Guide](testing.md)).

Template for building an adapter

Below is a step-by-step guide to building an adapter for a new data provider using the provided template.

InstrumentProvider

The InstrumentProvider supplies instrument definitions available on the venue. This
includes loading all available instruments, specific instruments by ID, and applying filters to the instrument list.

Key Methods:

- loadallasync: Loads all instruments asynchronously, optionally applying filters
- loadidsasync: Loads specific instruments by their IDs
- loadasync: Loads a single instrument by its ID

DataClient

The LiveDataClient handles the subscription and management of data feeds that are not specifically related to market data. This might include news feeds, custom data streams, or other data sources that enhance trading strategies but do not directly represent market activity.

Key Methods:

- connect: Establishes a connection to the data provider

- disconnect: Closes the connection to the data provider
- reset: Resets the state of the client
- dispose: Disposes of any resources held by the client
- subscribe: Subscribes to a specific data type
- unsubscribe: Unsubscribes from a specific data type
- request: Requests data from the provider

MarketDataClient

The MarketDataClient handles market-specific data such as order books, top-of-book quotes and trades,
and instrument status updates. It focuses on providing historical and real-time market data that is essential for
trading operations.

Key Methods:

- connect: Establishes a connection to the venues APIs
- disconnect: Closes the connection to the venues APIs
- reset: Resets the state of the client
- dispose: Disposes of any resources held by the client
- subscribeinstruments: Subscribes to market data for multiple instruments
- unsubscribeinstruments: Unsubscribes from market data for multiple instruments
- subscribeorderbookdeltas: Subscribes to order book delta updates
- unsubscribeorderbookdeltas: Unsubscribes from order book delta updates

---

RESTAPI field-mapping guideline

When translating a venue's REST payload into our domain model avoid renaming the upstream fields unless there is a compelling reason (e.g. a clash with reserved keywords). The only transformation we apply by default is camelCase snakecase.

Keeping the external names intact makes it trivial to debug payloads, compare captures against the Rust structs, and speeds up onboarding for new contributors who have the venue's API reference open side-by-side.

ExecutionClient

The ExecutionClient is responsible for order management, including submission, modification, and cancellation of orders. It is a crucial component of the adapter that interacts with the venues trading system to manage and execute trades.

Key Methods:

- connect: Establishes a connection to the venues APIs
- disconnect: Closes the connection to the venues APIs
- submitorder: Submits a new order to the venue
- modifyorder: Modifies an existing order on the venue
- cancelorder: Cancels a specific order on the venue
- cancelallorders: Cancels all orders for an instrument on the venue
- batchcancelorders: Cancels a batch of orders for an instrument on the venue
- generateorderstatusreport: Generates a report for a specific order on the venue
- generateorderstatusreports: Generates reports for all orders on the venue
- generatefillreports: Generates reports for filled orders on the venue
- generatepositionstatusreports: Generates reports for position status on the venue

Configuration

The configuration file defines settings specific to the adapter, such as API keys and connection details. These settings are essential for initializing and managing the adapter's connection to the data provider.

Key Attributes:

- apikey: The API key for authenticating with the data provider
- apisecret: The API secret for authenticating with the data provider
- baseurl: The base URL for connecting to the data provider's API

developer_guide/benchmarking.md

Benchmarking

This guide explains how NautilusTrader measures Rust performance, when to
use each tool and the conventions you should follow when adding new benches.

Tooling overview

Nautilus Trader relies on two complementary benchmarking frameworks:

Framework	What is it?	What it measures	When to prefer it
[Criterion](https://docs.rs/criterion/latest/criterion/)	Statistical benchmark harness that produces detailed HTML reports and performs outlier detection.	Wall-clock run time with confidence intervals.	End-to-end scenarios, anything slower than 100 ns, visual comparisons.
[iai](https://docs.rs/iai/latest/iai/)	Deterministic micro-benchmark harness that counts retired CPU instructions via hardware counters.	Exact instruction counts (noise-free).	Ultra-fast functions, CI gating via instruction diff.

Most hot code paths benefit from both kinds of measurements.

Directory layout

Each crate keeps its performance tests in a local benches/ folder:

Cargo.toml must list every benchmark explicitly so cargo bench discovers
them:

Writing Criterion benchmarks

1. Perform all expensive set-up outside the timing loop (b.iter).
2. Wrap inputs/outputs in blackbox to prevent the optimizer from removing
   work.
3. Group related cases with benchmarkgroup! and set throughput or
   samplesize when the defaults aren't ideal.

Writing iai benchmarks

iai requires functions that take no parameters and return a value (which
can be ignored). Keep them as small as possible so the measured instruction
count is meaningful.

Running benches locally

- All benches for every crate: make cargo-bench (delegates to cargo bench).
- Single crate: cargo bench -p nautilus-core.
- Single benchmark file: cargo bench -p nautilus-core --bench time.

Criterion writes HTML reports to target/criterion/; open target/criterion/report/index.html in your browser.

Generating a flamegraph

cargo-flamegraph (a thin wrapper around Linux perf) lets you see a sampled call-stack profile of a single benchmark.

1. Install once per machine (the crate is called flamegraph; it installs a cargo flamegraph subcommand automatically). Linux requires perf to be available (sudo apt install linux-tools-common linux-tools-$(uname -r) on Debian/Ubuntu):

2. Run a specific bench with the symbol-rich bench profile:

3. Open the generated flamegraph.svg (or .png) in your browser and zoom into hot paths.

If you see an error mentioning perfeventparanoid you need to relax the kernel's perf restrictions for the current session (root required):

A value of 1 is typically enough; set it back to 2 (default) or make the change permanent via /etc/sysctl.conf if desired.

Because [profile.bench] keeps full debug symbols the SVG will show readable function names without bloating production binaries (which still use panic = "abort" and are built via [profile.release]).

> Note Benchmark binaries are compiled with the custom [profile.bench]
> defined in the workspace Cargo.toml. That profile inherits from
> release-debugging, preserving full optimisation and debug symbols so that
> tools like cargo flamegraph or perf produce human-readable stack traces.

Templates

Ready-to-copy starter files live in docs/devtemplates/.

- Criterion: criteriontemplate.rs
- iai: iaitemplate.rs

Copy the template into benches/, adjust imports and names, and start measuring!

developer_guide/coding_standards.md

Coding Standards

Code Style

The current codebase can be used as a guide for formatting conventions.
Additional guidelines are provided below.

Universal formatting rules

The following applies to all source files (Rust, Python, Cython, shell, etc.):

- Use spaces only, never hard tab characters.
- Lines should generally stay below 100 characters; wrap thoughtfully when necessary.
- Prefer American English spelling (color, serialize, behavior).

Comment conventions

1. Generally leave one blank line above every comment block or docstring so it is visually separated from code.
2. Use sentence case - capitalize the first letter, keep the rest lowercase unless proper nouns or acronyms.
3. Do not use double spaces after periods.
4. Single-line comments must not end with a period unless the line ends with a URL or inline Markdown link - in those cases leave the punctuation exactly as the link requires.
5. Multi-line comments should separate sentences with commas (not period-per-line). The final line should end with a period.
6. Keep comments concise; favor clarity and only explain the non-obvious - less is more.
7. Avoid emoji symbols in text.

Doc comment / docstring mood

- Python docstrings should be written in the imperative mood - e.g. "Return a cached client."
- Rust doc comments should be written in the indicative mood - e.g. "Returns a cached client."

These conventions align with the prevailing styles of each language ecosystem and make generated documentation feel natural to end-users.

Formatting

1. For longer lines of code, and when passing more than a couple of arguments, you should take a new line which aligns at the next logical indent (rather than attempting a hanging 'vanity' alignment off an opening parenthesis). This practice conserves space to the right, ensures important code is more central in view, and is also robust to function/method name changes.

2. The closing parenthesis should be located on a new line, aligned at the logical indent.

3. Also ensure multiple hanging parameters or arguments end with a trailing comma:

PEP-8

The codebase generally follows the PEP-8 style guide. Even though C typing is taken advantage of in the Cython parts of the codebase, we still aim to be idiomatic of Python where possible.
One notable departure is that Python truthiness is not always taken advantage of to check if an argument is None for everything other than collections.

There are two reasons for this;

1. Cython can generate more efficient C code from is None and is not None, rather than entering the Python runtime to check the PyObject truthiness.

2. As per the [Google Python Style Guide](https://google.github.io/styleguide/pyguide.html) - it's discouraged to use truthiness to check if an argument is/is not None, when there is a chance an unexpected object could be passed into the function or method which will yield an unexpected truthiness evaluation (which could result in a logical error type bug).

"Always use if foo is None: (or is not None) to check for a None value. E.g., when testing whether a variable or argument that defaults to None was set to some other value. The other value might be a value that's false in a boolean context!"

There are still areas that aren't performance-critical where truthiness checks for None (if foo is None: vs if not foo:) will be acceptable for clarity.

:::note
Use truthiness to check for empty collections (e.g., if not mylist:) rather than comparing explicitly to None or empty.
:::

We welcome all feedback on where the codebase departs from PEP-8 for no apparent reason.

Python Style Guide

Type Hints

All function and method signatures must include comprehensive type annotations:

Generic Types: Use TypeVar for reusable components

Docstrings

The [NumPy docstring spec](https://numpydoc.readthedocs.io/en/latest/format.html) is used throughout the codebase.
This needs to be adhered to consistently to ensure the docs build correctly.

Test method naming: Descriptive names explaining the scenario:

Ruff

[ruff](https://astral.sh/ruff) is utilized to lint the codebase. Ruff rules can be found in the top-level pyproject.toml, with ignore justifications typically commented.

Commit messages

Here are some guidelines for the style of your commit messages:

1. Limit subject titles to 60 characters or fewer. Capitalize subject line and do not end with period.

2. Use 'imperative voice', i.e. the message should describe what the commit will do if applied.

3. Optional: Use the body to explain change. Separate from subject with a blank line. Keep under 100 character width. You can use bullet points with or without terminating periods.

4. Optional: Provide # references to relevant issues or tickets.

5. Optional: Provide any hyperlinks which are informative.

developer_guide/cython.md

Cython

Here you will find guidance and tips for working on NautilusTrader using the Cython language.
More information on Cython syntax and conventions can be found by reading the [Cython docs](https://cython.readthedocs.io/en/latest/index.html).

What is Cython?

Cython is a superset of Python that compiles to C extension modules, enabling optional static typing and optimized performance. NautilusTrader relies on Cython for its Python bindings and performance-critical components.

Function and method signatures

Ensure that all functions and methods returning void or a primitive C type (such as bint, int, double) include the except keyword in the signature.

This will ensure Python exceptions are not ignored, and instead are "bubbled up" to the caller as expected.

Debugging

PyCharm

Improved debugging support for Cython has remained a highly up-voted PyCharm
feature for many years. Unfortunately, it's safe to assume that PyCharm will not
be receiving first class support for Cython debugging
<https://youtrack.jetbrains.com/issue/PY-9476>.

Cython Docs

The following recommendations are contained in the Cython docs:
<https://cython.readthedocs.io/en/latest/src/userguide/debugging.html>

The summary is it involves manually running a specialized version of gdb from the command line.
We don't recommend this workflow.

Tips

When debugging and seeking to understand a complex system such as NautilusTrader, it can be
quite helpful to step through the code with a debugger. With this not being available
for the Cython part of the codebase, there are a few things which can help:

- Ensure LogLevel.DEBUG is configured for the backtesting or live system you are debugging.
    This is available on BacktestEngineConfig(logging=LoggingConfig(loglevel="DEBUG")) or TradingNodeConfig(logging=LoggingConfig=loglevel="DEBUG")).
  With DEBUG mode active you will see more granular and verbose log traces which could be what you need to understand the flow.
- Beyond this, if you still require more granular visibility around a part of the system, we recommend some well-placed calls
  to a components logger (normally self.log.debug(f"HERE {variable}" is enough).

developer_guide/docs.md

Documentation Style Guide

This guide outlines the style conventions and best practices for writing documentation for NautilusTrader.

General Principles

- We favor simplicity over complexity, less is more.
- We favor concise yet readable prose and documentation.
- We value standardization in conventions, style, patterns, etc.
- Documentation should be accessible to users of varying technical backgrounds.

Markdown Tables

Column Alignment and Spacing

- Use symmetrical column widths based on the space necessary dictated by the widest content in each column.
- Align column separators (|) vertically for better readability.
- Use consistent spacing around cell content.

Notes and Descriptions

- All notes and descriptions should have terminating periods.
- Keep notes concise but informative.
- Use sentence case (capitalize only the first letter and proper nouns).

Example

Support Indicators

- Use for supported features.
- Use - for unsupported features (not or other symbols).
- When adding notes for unsupported features, emphasize with italics: Not supported.
- Leave cells empty when no content is needed.

Code References

- Use backticks for inline code, method names, class names, and configuration options.
- Use code blocks for multi-line examples.
- When referencing functions or code locations, include the pattern filepath:linenumber to allow easy navigation.

Headings

- Use title case for main headings (## Level 2).
- Use sentence case for subheadings (### Level 3 and below).
- Ensure proper heading hierarchy (don't skip levels).

Lists

- Use hyphens (-) for unordered-list bullets; avoid or + to keep the Markdown style consistent across the project.
- Use numbered lists only when order matters.
- Maintain consistent indentation for nested lists.
- End list items with periods when they are complete sentences.

Links and References

- Use descriptive link text (avoid "click here" or "this link").
- Reference external documentation when appropriate.
- Ensure all internal links are relative and accurate.

Technical Terminology

- Base capability matrices on the Nautilus domain model, not exchange-specific terminology.
- Mention exchange-specific terms in parentheses or notes when necessary for clarity.
- Use consistent terminology throughout the documentation.

Examples and Code Samples

- Provide practical, working examples.
- Include necessary imports and context.
- Use realistic variable names and values.
- Add comments to explain non-obvious parts of examples.

Warnings and Notes

- Use appropriate admonition blocks for important information:
  - :::note for general information.
  - :::warning for important caveats.
  - :::tip for helpful suggestions.

API Documentation

- Document parameters and return types clearly.
- Include usage examples for complex APIs.
- Explain any side effects or important behavior.
- Keep parameter descriptions concise but complete.

developer_guide/environment_setup.md

Environment Setup

For development we recommend using the PyCharm Professional edition IDE, as it interprets Cython syntax. Alternatively, you could use Visual Studio Code with a Cython extension.

[uv](https://docs.astral.sh/uv) is the preferred tool for handling all Python virtual environments and dependencies.

[pre-commit](https://pre-commit.com/) is used to automatically run various checks, auto-formatters and linting tools at commit.

NautilusTrader uses increasingly more [Rust](https://www.rust-lang.org), so Rust should be installed on your system as well
([installation guide](https://www.rust-lang.org/tools/install)).

:::info
NautilusTrader must compile and run on Linux, macOS, and Windows. Please keep portability in mind (use std::path::Path, avoid Bash-isms in shell scripts, etc.).
:::

Setup

The following steps are for UNIX-like systems, and only need to be completed once.

1. Follow the [installation guide](../gettingstarted/installation.md) to set up the project with a modification to the final command to install development and test dependencies:

or

If you're developing and iterating frequently, then compiling in debug mode is often sufficient and significantly faster than a fully optimized build.
To install in debug mode, use:

2. Set up the pre-commit hook which will then run automatically at commit:

Before opening a pull-request run the formatting and lint suite locally so that CI passes on the first attempt:

Make sure the Rust compiler reports zero errors - broken builds slow everyone down.

3. Optional: For frequent Rust development, configure the PYO3PYTHON variable in .cargo/config.toml with the path to the Python interpreter. This helps reduce recompilation times for IDE/rust-analyzer based cargo check:

Since .cargo/config.toml is tracked, configure git to skip any local modifications:

To restore tracking: git update-index --no-skip-worktree .cargo/config.toml

Builds

Following any changes to .rs, .pyx or .pxd files, you can re-compile by running:

or

If you're developing and iterating frequently, then compiling in debug mode is often sufficient and significantly faster than a fully optimized build.
To compile in debug mode, use:

Faster builds

The cranelift backends reduces build time significantly for dev, testing and IDE checks. However, cranelift is available on the nightly toolchain and needs extra configuration. Install the nightly toolchain

Activate the nightly feature and use "cranelift" backend for dev and testing profiles in workspace Cargo.toml. You can apply the below patch using git apply <patch>. You can remove it using git apply -R <patch> before pushing changes.

Pass RUSTUPTOOLCHAIN=nightly when running make build-debug like commands and include it in in all [rust analyzer settings](#rust-analyzer-settings) for faster builds and IDE checks.

Services

You can use docker-compose.yml file located in .docker directory
to bootstrap the Nautilus working environment. This will start the following services:

If you only want specific services running (like postgres for example), you can start them with command:

Used services are:

- postgres: Postgres database with root user POSTRESUSER which defaults to postgres, POSTGRESPASSWORD which defaults to pass and POSTGRESDB which defaults to postgres.
- redis: Redis server.
- pgadmin: PgAdmin4 for database management and administration.

:::info
Please use this as development environment only. For production, use a proper and more secure setup.
:::

After the services has been started, you must log in with psql cli to create nautilus Postgres database.
To do that you can run, and type POSTGRESPASSWORD from docker service setup

After you have logged in as postgres administrator, run CREATE DATABASE command with target db name (we use nautilus):

Nautilus CLI Developer Guide

Introduction

The Nautilus CLI is a command-line interface tool for interacting with the NautilusTrader ecosystem.
It offers commands for managing the PostgreSQL database and handling various trading operations.

:::note
The Nautilus CLI command is only supported on UNIX-like systems.
:::

Install

You can install the Nautilus CLI using the below Makefile target, which leverages cargo install under the hood.
This will place the nautilus binary in your system's PATH, assuming Rust's cargo is properly configured.

Commands

You can run nautilus --help to view the CLI structure and available command groups:

Database

These commands handle bootstrapping the PostgreSQL database.
To use them, you need to provide the correct connection configuration,

either through command-line arguments or a .env file located in the root directory or the current working directory.

- --host or POSTGRESHOST for the database host
- --port or POSTGRESPORT for the database port
- --user or POSTGRESUSER for the root administrator (typically the postgres user)
- --password or POSTGRESPASSWORD for the root administrator's password
- --database or POSTGRESDATABASE for both the database name and the new user with privileges to that database
    (e.g., if you provide nautilus as the value, a new user named nautilus will be created with the password from POSTGRESPASSWORD, and the nautilus database will be bootstrapped with this user as the owner).

Example of .env file

List of commands are:

1. nautilus database init: Will bootstrap schema, roles and all sql files located in schema root directory (like tables.sql).
2. nautilus database drop: Will drop all tables, roles and data in target Postgres database.

Rust analyzer settings

Rust analyzer is a popular language server for Rust and has integrations for many IDEs. It is recommended to configure rust analyzer to have same environment variables as make build-debug for faster compile times. Below tested configurations for VSCode and Astro Nvim are provided. For more information see [PR](https://github.com/nautechsystems/nautilustrader/pull/2524) or rust analyzer [config docs](https://rust-analyzer.github.io/book/configuration.html).

VSCode

You can add the following settings to your VSCode settings.json file:

Astro Nvim (Neovim + AstroLSP)

You can add the following to your astro lsp config file:

developer_guide/index.md

Developer Guide

Welcome to the developer guide for NautilusTrader!

Here you'll find guidance on developing and extending NautilusTrader to meet your trading needs or to contribute improvements back to the project.

We believe in using the right tool for the job. The overall design philosophy is to fully utilize the high level power of Python, with its rich eco-system of frameworks and libraries, whilst overcoming some of its inherent shortcomings in performance and lack of built-in type safety (with it being an interpreted dynamic language).

One of the advantages of Cython is that allocation and freeing of memory is handled by the C code generator during the 'cythonization' step of the build (unless you're specifically utilizing some of its lower level features).

This approach combines Python's simplicity with near-native C performance via compiled extensions.

The main development and runtime environment we are working in is of course Python. With the introduction of Cython syntax throughout the production codebase in .pyx and .pxd files - it's important to be aware of how the CPython implementation of Python interacts with the underlying CPython API, and the NautilusTrader C extension modules which Cython produces.

We recommend a thorough review of the [Cython docs](https://cython.readthedocs.io/en/latest/) to familiarize yourself with some of its core
concepts, and where C typing is being introduced.

It's not necessary to become a C language expert, however it's helpful to understand how Cython C syntax is used in function and method definitions, in local code blocks, and the common primitive C types and how these map to their corresponding PyObject types.

Contents

- [Environment Setup](environmentsetup.md)
- [Coding Standards](codingstandards.md)
- [Cython](cython.md)
- [Rust](rust.md)
- [Docs](docs.md)
- [Testing](testing.md)
- [Adapters](adapters.md)
- [Benchmarking](benchmarking.md)
- [Packaged Data](packageddata.md)

developer_guide/packaged_data.md

Packaged Data

Various data is contained internally in the tests/testkit/data folder.

Libor Rates

The libor rates for 1 month USD can be updated by downloading the CSV data from <https://fred.stlouisfed.org/series/USD1MTD156N>.

Ensure you select Max for the time window.

Short Term Interest Rates

The interbank short term interest rates can be updated by downloading the CSV data at <https://data.oecd.org/interest/short-term-interest-rates.htm>.

Economic Events

The economic events can be updated from downloading the CSV data from fxstreet <https://www.fxstreet.com/economic-calendar>.

Ensure timezone is set to GMT.

A maximum 3 month range can be filtered and so yearly quarters must be downloaded manually and stitched together into a single CSV.
Use the calendar icon to filter the data in the following way;

- 01/01/xx - 31/03/xx
- 01/04/xx - 30/06/xx
- 01/07/xx - 30/09/xx
- 01/10/xx - 31/12/xx

Download each CSV.

developer_guide/rust.md

Rust Style Guide

The [Rust](https://www.rust-lang.org/learn) programming language is an ideal fit for implementing the mission-critical core of the platform and systems. Its strong type system, ownership model, and compile-time checks eliminate memory errors and data races by construction, while zero-cost abstractions and the absence of a garbage collector deliver C-like performance-critical for high-frequency trading workloads.

Python Bindings

Python bindings are provided via Cython and [PyO3](https://pyo3.rs), allowing users to import NautilusTrader crates directly in Python without a Rust toolchain.

Code Style and Conventions

File Header Requirements

All Rust files must include the standardized copyright header:

Code Formatting

Import formatting is automatically handled by rustfmt when running make format.
The tool organizes imports into groups (standard library, external crates, local imports) and sorts them alphabetically within each group.

Function spacing

- Leave one blank line between functions (including tests) - this improves readability and mirrors the default behavior of rustfmt.
- Leave one blank line above every doc comment (/// or //!) so that the comment is clearly detached from the previous code block.

PyO3 naming convention

When exposing Rust functions to Python via PyO3:

1. The Rust symbol must be prefixed with py to make its purpose explicit inside the Rust codebase.
2. Use the #[pyo3(name = "...")] attribute to publish the Python name without the py prefix so the Python API remains clean.

Error Handling

Use structured error handling patterns consistently:

1. Primary Pattern: Use anyhow::Result<T> for fallible functions:

2. Custom Error Types: Use thiserror for domain-specific errors:

3. Error Propagation: Use the ? operator for clean error propagation.

4. Error Creation: Prefer anyhow::bail! for early returns with errors:

Note: Use anyhow::bail! for early returns, but anyhow::anyhow! in closure contexts like okorelse() where early returns aren't possible.

5. Error Message Formatting: Prefer inline format strings over positional arguments:

This makes error messages more readable and self-documenting, especially when there are multiple variables.

Attribute Patterns

Consistent attribute usage and ordering:

For enums with extensive derive attributes:

Constructor Patterns

Use the new() vs newchecked() convention consistently:

Always use the FAILED constant for .expect() messages related to correctness checks:

Constants and Naming Conventions

Use SCREAMINGSNAKECASE for constants with descriptive names:

Re-export Patterns

Organize re-exports alphabetically and place at the end of lib.rs files:

Documentation Standards

Module-Level Documentation

All modules must have module-level documentation starting with a brief description:

For modules with feature flags, document them clearly:

Field Documentation

All struct and enum fields must have documentation with terminating periods:

Function Documentation

Document all public functions with:

- Purpose and behavior
- Explanation of input argument usage
- Error conditions (if applicable)
- Panic conditions (if applicable)

Errors and Panics Documentation Format

For single line errors and panics documentation, use sentence case with the following convention:

For multi-line errors and panics documentation, use sentence case with bullets and terminating periods:

Safety Documentation Format

For Safety documentation, use the SAFETY: prefix followed by a short description explaining why the unsafe operation is valid:

For inline unsafe blocks, use the SAFETY: comment directly above the unsafe code:

Testing Conventions

Test Organization

Use consistent test module structure with section separators:

Parameterized Testing

Use the rstest attribute consistently, and for parameterized tests:

Test Naming

Use descriptive test names that explain the scenario:

Unsafe Rust

It will be necessary to write unsafe Rust code to be able to achieve the value
of interoperating between Cython and Rust. The ability to step outside the boundaries of safe Rust is what makes it possible to
implement many of the most fundamental features of the Rust language itself, just as C and C++ are used to implement
their own standard libraries.

Great care will be taken with the use of Rusts unsafe facility - which just enables a small set of additional language features, thereby changing
the contract between the interface and caller, shifting some responsibility for guaranteeing correctness
from the Rust compiler, and onto us. The goal is to realize the advantages of the unsafe facility, whilst avoiding any undefined behavior.
The definition for what the Rust language designers consider undefined behavior can be found in the [language reference](https://doc.rust-lang.org/stable/reference/behavior-considered-undefined.html).

Safety Policy

To maintain correctness, any use of unsafe Rust must follow our policy:

- If a function is unsafe to call, there must be a Safety section in the documentation explaining why the function is unsafe.
and covering the invariants which the function expects the callers to uphold, and how to meet their obligations in that contract.
- Document why each function is unsafe in its doc comment's Safety section, and cover all unsafe blocks with unit tests.
- Always include a SAFETY: comment explaining why the unsafe operation is valid:

Tooling Configuration

The project uses several tools for code quality:

- rustfmt: Automatic code formatting (see rustfmt.toml).
- clippy: Linting and best practices (see clippy.toml).
- cbindgen: C header generation for FFI.

Resources

- [The Rustonomicon](https://doc.rust-lang.org/nomicon/) - The Dark Arts of Unsafe Rust.
- [The Rust Reference - Unsafety](https://doc.rust-lang.org/stable/reference/unsafety.html).
- [Safe Bindings in Rust - Russell Johnston](https://www.abubalay.com/blog/2020/08/22/safe-bindings-in-rust).
- [Google - Rust and C interoperability](https://www.chromium.org/Home/chromium-security/memory-safety/rust-and-c-interoperability/).

developer_guide/testing.md

Testing

The test suite is divided into broad categories of tests including:

- Unit tests
- Integration tests
- Acceptance tests
- Performance tests
- Memory leak tests

The performance tests exist to aid development of performance-critical components.

Tests can be run using [pytest](https://docs.pytest.org), which is our primary test runner. We recommend using parametrized tests and fixtures (e.g., @pytest.mark.parametrize) to avoid repetitive code and improve clarity.

Running Tests

Python Tests

From the repository root:

For performance tests:

Rust Tests

IDE Integration

- PyCharm: Right-click on tests folder or file "Run pytest"
- VS Code: Use the Python Test Explorer extension

Mocks

Unit tests will often include other components acting as mocks. The intent of this is to simplify the test suite to avoid extensive use of a mocking framework, although MagicMock objects are currently used in particular cases.

Code Coverage

Code coverage output is generated using coverage and reported using [codecov](https://about.codecov.io/).

High test coverage is a goal for the project however not at the expense of appropriate error handling, or causing "test induced damage" to the architecture.

There are currently areas of the codebase which are impossible to test unless there is a change to the production code. For example the last condition check of an if-else block which would catch an unrecognized value, these should be left in place in case there is a change to the production code - which these checks could then catch.

Other design-time exceptions may also be impossible to test for, and so 100% test coverage is not the ultimate goal.

Style guidance

- Group assertions where possible - perform all setup/act steps first, then assert expectations together at
  the end of the test to avoid the act-assert-act smell.

- Using unwrap, expect, or direct panic!/assert calls inside tests is acceptable. The clarity and conciseness of the test suite outweigh defensive error-handling that is required in production code.

Excluded code coverage

The pragma: no cover comments found throughout the codebase [exclude code from test coverage](https://coverage.readthedocs.io/en/coverage-4.3.3/excluding.html).
The reason for their use is to reduce redundant/needless tests just to keep coverage high, such as:

- Asserting an abstract method raises NotImplementedError when called.
- Asserting the final condition check of an if-else block when impossible to test (as above).

These tests are expensive to maintain (as they must be kept in line with any refactorings), and offer little to no benefit in return. The intention is for all abstract method implementations to be fully covered by tests. Therefore pragma: no cover should be judiciously removed when no longer appropriate, and its use restricted to the above cases.

Debugging Rust Tests

Rust tests can be debugged using the default test configuration.

If you want to run all tests while compiling with debug symbols for later debugging some tests individually,
run make cargo-test-debug instead of make cargo-test.

In IntellijIdea, to debug parametrised tests starting with #[rstest] with arguments defined in the header of the test
you need to modify the run configuration of the test so it looks like
test --package nautilus-model --lib data::bar::tests::testgettimebarstart::case1
(remove -- --exact at the end of the string and append ::casen where n is an integer corresponding to the n-th parametrised test starting at 1).
The reason for this is [here](https://github.com/rust-lang/rust-analyzer/issues/8964#issuecomment-871592851)
(the test is expanded into a module with several functions named casen).
In VSCode, it is possible to directly select which test case to debug.

getting_started/backtest_high_level.md

Backtest (high-level API)

This guide is generated from the Jupyter notebook backtesthighlevel.ipynb.

getting_started/backtest_low_level.md

Backtest (low-level API)

This guide is generated from the Jupyter notebook backtestlowlevel.ipynb.

getting_started/index.md

Getting Started

To get started with NautilusTrader, you will need:

- A Python 3.11-3.13 environment with the nautilustrader package installed.
- A way to run Python scripts or Jupyter notebooks for backtesting and/or live trading.

[Installation](installation.md)

The Installation guide will help to ensure that NautilusTrader is properly installed on your machine.

[Quickstart](quickstart.md)

The Quickstart provides a step-by-step walk through for setting up your first backtest.

Examples in repository

The [online documentation](https://nautilustrader.io/docs/latest/) shows just a subset of examples. For the full set, see this repository on GitHub.

The following table lists example locations ordered by recommended learning progression:

Directory	Contains
[examples/](https://github.com/nautechsystems/nautilustrader/tree/develop/examples)	Fully runnable, self-contained Python examples.
[docs/tutorials/](tutorials/)	Jupyter notebook tutorials demonstrating common workflows.
[docs/concepts/](concepts/)	Concept guides with concise code snippets illustrating key features.
[nautilustrader/examples/](../nautilustrader/examples/)	Pure-Python examples of basic strategies, indicators, and execution algorithms.
[tests/unittests/](../../tests/unittests/)	Unit tests covering core functionality and edge cases.

Backtesting API levels

NautilusTrader provides two different API levels for backtesting:

API Level	Description	Characteristics
High-Level API	Uses BacktestNode and TradingNode	Recommended for production: easier transition to live trading; requires a Parquet-based data catalog.
Low-Level API	Uses BacktestEngine	Intended for library development: no live-trading path; direct component access; may encourage non-live-compatible patterns.

Backtesting involves running simulated trading systems on historical data.

To get started backtesting with NautilusTrader you need to first understand the two different API

levels which are provided, and which one may be more suitable for your intended use case.

:::info
For more information on which API level to choose, refer to the [Backtesting](../concepts/backtesting.md) guide.
:::

[Backtest (low-level API)](backtestlowlevel.md)

This tutorial runs through how to load raw data (external to Nautilus) using data loaders and wranglers, and then use this data with a BacktestEngine to run a single backtest.

[Backtest (high-level API)](backtesthighlevel.md)

This tutorial runs through how to load raw data (external to Nautilus) into the data catalog,
and then use this data with a BacktestNode to run a single backtest.

Running in docker

Alternatively, a self-contained dockerized Jupyter notebook server is available for download, which does not require any setup or
installation. This is the fastest way to get up and running to try out NautilusTrader. Bear in mind that any data will be
deleted when the container is deleted.

- To get started, install docker:
  - Go to [docker.com](https://docs.docker.com/get-docker/) and follow the instructions
- From a terminal, download the latest image
  - docker pull ghcr.io/nautechsystems/jupyterlab:nightly --platform linux/amd64
- Run the docker container, exposing the jupyter port:
  - docker run -p 8888:8888 ghcr.io/nautechsystems/jupyterlab:nightly
- Open your web browser to localhost:{port}
  - <http://localhost:8888>

:::info
NautilusTrader currently exceeds the rate limit for Jupyter notebook logging (stdout output),
this is why loglevel in the examples is set to ERROR. If you lower this level to see
more logging then the notebook will hang during cell execution. A fix is currently
being investigated which involves either raising the configured rate limits for
Jupyter, or throttling the log flushing from Nautilus.

- <https://github.com/jupyterlab/jupyterlab/issues/12845>
- <https://github.com/deshaw/jupyterlab-limit-output>

:::

getting_started/installation.md

Installation

NautilusTrader is officially supported for Python 3.11-3.13 on the following 64-bit platforms:

Operating System	Supported Versions	CPU Architecture
Linux (Ubuntu)	22.04 and later	x8664
Linux (Ubuntu)	22.04 and later	ARM64
macOS	14.7 and later	ARM64
Windows Server	2022 and later	x8664

\ Windows builds are currently pinned to CPython 3.13.2 because later
3.13.x Windows binaries were produced with the per-interpreter GIL enabled
but without exporting a handful of private C-API symbols. These missing
exports break linking of our Cython extensions. The pin can be removed
if/when an upstream CPython release restores the exports.

:::note
NautilusTrader may work on other platforms, but only those listed above are regularly used by developers and tested in CI.
:::

We recommend using the latest supported version of Python and installing [nautilustrader](https://pypi.org/project/nautilustrader/) inside a virtual environment to isolate dependencies.

There are two supported ways to install:

1. Pre-built binary wheel from PyPI or the Nautech Systems package index.
2. Build from source.

:::tip
We highly recommend installing using the [uv](https://docs.astral.sh/uv) package manager with a "vanilla" CPython.

Conda and other Python distributions may work but aren't officially supported.
:::

From PyPI

To install the latest [nautilustrader](https://pypi.org/project/nautilustrader/) binary wheel (or sdist package) from PyPI using Pythons pip package manager:

Extras

Install optional dependencies as 'extras' for specific integrations:

- betfair: Betfair adapter (integration) dependencies.
- docker: Needed for Docker when using the IB gateway (with the Interactive Brokers adapter).
- dydx: dYdX adapter (integration) dependencies.
- ib: Interactive Brokers adapter (integration) dependencies.
- polymarket: Polymarket adapter (integration) dependencies.

To install with specific extras using pip:

From the Nautech Systems package index

The Nautech Systems package index (packages.nautechsystems.io) is [PEP-503](https://peps.python.org/pep-0503/) compliant and hosts both stable and development binary wheels for nautilustrader.
This enables users to install either the latest stable release or pre-release versions for testing.

Stable wheels

Stable wheels correspond to official releases of nautilustrader on PyPI, and use standard versioning.

To install the latest stable release:

Development wheels

Development wheels are published from both the nightly and develop branches,
allowing users to test features and fixes ahead of stable releases.

Note: Wheels from the develop branch are only built for the Linux x8664 platform to save time
and compute resources, while nightly wheels support additional platforms as shown below.

Platform	Nightly	Develop
Linux (x8664)		
Linux (ARM64)		-
macOS (ARM64)		-
Windows (x8664)		-

This process also helps preserve compute resources and ensures easy access to the exact binaries tested in CI pipelines,
while adhering to [PEP-440](https://peps.python.org/pep-0440/) versioning standards:

- develop wheels use the version format dev{date}+{buildnumber} (e.g., 1.208.0.dev20241212+7001).
- nightly wheels use the version format a{date} (alpha) (e.g., 1.208.0a20241212).

:::warning
We don't recommend using development wheels in production environments, such as live trading controlling real capital.
:::

Installation commands

By default, pip installs the latest stable release. Adding the --pre flag ensures that pre-release versions, including development wheels, are considered.

To install the latest available pre-release (including development wheels):

To install a specific development wheel (e.g., 1.208.0a20241212 for December 12, 2024):

Available versions

You can view all available versions of nautilustrader on the [package index](https://packages.nautechsystems.io/simple/nautilus-trader/index.html).

To programmatically request and list available versions:

Branch updates

- develop branch wheels (.dev): Are built and published continuously with every merged commit.
- nightly branch wheels (a): Are built and published daily when develop branch is automatically merged at 14:00 UTC (if there are changes).

Retention policies

- develop branch wheels (.dev): Only the most recent wheel build is retained.
- nightly branch wheels (a): Only the 10 most recent wheel builds are retained.

From Source

It's possible to install from source using pip if you first install the build dependencies as specified in the pyproject.toml.

1. Install [rustup](https://rustup.rs/) (the Rust toolchain installer):
   - Linux and macOS:

- Windows:
  - Download and install [rustup-init.exe](https://win.rustup.rs/x8664)
      - Install "Desktop development with C++" with [Build Tools for Visual Studio 2019](https://visualstudio.microsoft.com/thank-you-downloading-visual-studio/?sku=BuildTools&rel=16)
- Verify (any system):
   from a terminal session run: rustc --version

2. Enable cargo in the current shell:
   - Linux and macOS:

- Windows:
  - Start a new PowerShell

1. Install [clang](https://clang.llvm.org/) (a C language frontend for LLVM):
- Linux:

- Windows:

1. Add Clang to your [Build Tools for Visual Studio 2019](https://visualstudio.microsoft.com/thank-you-downloading-visual-studio/?sku=BuildTools&rel=16):

- Start | Visual Studio Installer | Modify | C++ Clang tools for Windows (12.0.0 - x64...) = checked | Modify

2. Enable clang in the current shell:

- Verify (any system):
  from a terminal session run:

3. Install uv (see the [uv installation guide](https://docs.astral.sh/uv/getting-started/installation) for more details):

4. Clone the source with git, and install from the project's root directory:

:::note
The --depth 1 flag fetches just the latest commit for a faster, lightweight clone.
:::

5. Set environment variables for PyO3 compilation (Linux and macOS only):

:::note
Adjust the Python version and architecture in the LDLIBRARYPATH to match your system.
Use uv python list to find the exact path for your Python installation.
:::

From GitHub Release

To install a binary wheel from GitHub, first navigate to the [latest release](https://github.com/nautechsystems/nautilustrader/releases/latest).
Download the appropriate .whl for your operating system and Python version, then run:

Versioning and releases

NautilusTrader is still under active development. Some features may be incomplete, and while
the API is becoming more stable, breaking changes can occur between releases.
We strive to document these changes in the release notes on a best-effort basis.

We aim to follow a weekly release schedule, though experimental or larger features may cause delays.

Use NautilusTrader only if you are prepared to adapt to these changes.

Redis

Using [Redis](https://redis.io) with NautilusTrader is optional and only required if configured as the backend for a cache database or [message bus](../concepts/messagebus.md).

:::info
The minimum supported Redis version is 6.2 (required for [streams](https://redis.io/docs/latest/develop/data-types/streams/) functionality).
:::

For a quick setup, we recommend using a [Redis Docker container](https://hub.docker.com//redis/).
You can find an example setup in the .docker directory,
or run the following command to start a container:

This command will:

- Pull the latest version of Redis from Docker Hub if it's not already downloaded.
- Run the container in detached mode (-d).
- Name the container redis for easy reference.
- Expose Redis on the default port 6379, making it accessible to NautilusTrader on your machine.

To manage the Redis container:

- Start it with docker start redis
- Stop it with docker stop redis

:::tip
We recommend using [Redis Insight](https://redis.io/insight/) as a GUI to visualize and debug Redis data efficiently.
:::

Precision mode

NautilusTrader supports two precision modes for its core value types (Price, Quantity, Money),
which differ in their internal bit-width and maximum decimal precision.

- High-precision: 128-bit integers with up to 16 decimals of precision, and a larger value range.
- Standard-precision: 64-bit integers with up to 9 decimals of precision, and a smaller value range.

:::note
By default, the official Python wheels ship in high-precision (128-bit) mode on Linux and macOS.
On Windows, only standard-precision (64-bit) is available due to the lack of native 128-bit integer support.

For the Rust crates, the default is standard-precision unless you explicitly enable the high-precision feature flag.
:::

The performance tradeoff is that standard-precision is ~3-5% faster in typical backtests,
but has lower decimal precision and a smaller representable value range.

:::note
Performance benchmarks comparing the modes are pending.
:::

Build configuration

The precision mode is determined by:

- Setting the HIGHPRECISION environment variable during compilation, and/or
- Enabling the high-precision Rust feature flag explicitly.

High-precision mode (128-bit)

Standard-precision mode (64-bit)

Rust feature flag

To enable high-precision (128-bit) mode in Rust, add the high-precision feature to your Cargo.toml:

:::info
See the [Value Types](../concepts/overview.md#value-types) specifications for more details.
:::

getting_started/quickstart.md

Quickstart

This guide is generated from the Jupyter notebook quickstart.ipynb.

integrations/betfair.md

Betfair

Founded in 2000, Betfair operates the world's largest online betting exchange,
with its headquarters in London and satellite offices across the globe.

NautilusTrader provides an adapter for integrating with the Betfair REST API and
Exchange Streaming API.

Installation

Install NautilusTrader with Betfair support via pip:

To build from source with Betfair extras:

Examples

You can find live example scripts [here](https://github.com/nautechsystems/nautilustrader/tree/develop/examples/live/betfair/).

Betfair documentation

For API details and troubleshooting, see the official [Betfair Developer Documentation](https://developer.betfair.com/en/get-started/).

Application Keys

Betfair requires an Application Key to authenticate API requests. After registering and funding your account, obtain your key using the [API-NG Developer AppKeys Tool](https://apps.betfair.com/visualisers/api-ng-account-operations/).

:::info
See also the [Betfair Getting Started - Application Keys](https://betfair-developer-docs.atlassian.net/wiki/spaces/1smk3cen4v3lu3yomq5qye0ni/pages/2687105/Application+Keys) guide.
:::

API credentials

Supply your Betfair credentials via environment variables or client configuration:

:::tip
We recommend using environment variables to manage your credentials.
:::

Overview

The Betfair adapter provides three primary components:

- BetfairInstrumentProvider: loads Betfair markets and converts them into Nautilus instruments.
- BetfairDataClient: streams real-time market data from the Exchange Streaming API.
- BetfairExecutionClient: submits orders (bets) and tracks execution status via the REST API.

Capability Matrix

Betfair operates as a betting exchange with unique characteristics compared to traditional financial exchanges:

Order Types

Order Type	Betfair	Notes
MARKET	-	Not applicable to betting exchange.
LIMIT		Orders placed at specific odds.
STOPMARKET	-	Not supported.
STOPLIMIT	-	Not supported.
MARKETIFTOUCHED	-	Not supported.
LIMITIFTOUCHED	-	Not supported.
TRAILINGSTOPMARKET	-	Not supported.

Execution Instructions

Instruction	Betfair	Notes
postonly	-	Not applicable to betting exchange.
reduceonly	-	Not applicable to betting exchange.

Time-in-Force Options

Time-in-Force	Betfair	Notes
GTC	-	Betting exchange uses different model.
GTD	-	Betting exchange uses different model.
FOK	-	Betting exchange uses different model.
IOC	-	Betting exchange uses different model.

Advanced Order Features

Feature	Betfair	Notes
Order Modification		Limited to non-exposure changing fields.
Bracket/OCO Orders	-	Not supported.
Iceberg Orders	-	Not supported.

Configuration Options

The following execution client configuration options affect order behavior:

Option	Default	Description
calculateaccountstate	True	If True, calculates account state from events.
requestaccountstatesecs	300	Interval for account state checks in seconds (0 disables).
reconcilemarketidsonly	False	If True, only reconciles orders for configured market IDs.
ignoreexternalorders	False	If True, silently ignores orders not found in cache.

Configuration

Here is a minimal example showing how to configure a live TradingNode with Betfair clients:

integrations/binance.md

Binance

Founded in 2017, Binance is one of the largest cryptocurrency exchanges in terms of daily trading volume, and open interest of crypto assets and crypto derivative products. This integration supports live market data ingest and order execution with Binance.

Installation

To install NautilusTrader with Binance support:

To build from source with all extras (including Binance):

Examples

You can find live example scripts [here](https://github.com/nautechsystems/nautilustrader/tree/develop/examples/live/binance/).

Overview

The Binance adapter supports the following product types:

- Spot markets (including Binance US)
- USDT-Margined Futures (perpetual and delivery)
- Coin-Margined Futures

:::note
Margin trading (cross & isolated) is not implemented at this time.
Contributions via [GitHub issue #2631](https://github.com/nautechsystems/nautilustrader/issues/#2631) or pull requests to add margin trading functionality are welcome.
:::

Product Support Matrix

Product Type	Supported	Notes
Spot Markets (incl. Binance US)		
Margin Accounts (Cross & Isolated)		Margin trading not implemented
USDT-Margined Futures (PERP & Delivery)		
Coin-Margined Futures		

This guide assumes a trader is setting up for both live market data feeds, and trade execution.
The Binance adapter includes multiple components, which can be used together or separately depending
on the use case.

- BinanceHttpClient: Low-level HTTP API connectivity.
- BinanceWebSocketClient: Low-level WebSocket API connectivity.
- BinanceInstrumentProvider: Instrument parsing and loading functionality.
- BinanceSpotDataClient/BinanceFuturesDataClient: A market data feed manager.
- BinanceSpotExecutionClient/BinanceFuturesExecutionClient: An account management and trade execution gateway.
- BinanceLiveDataClientFactory: Factory for Binance data clients (used by the trading node builder).
- BinanceLiveExecClientFactory: Factory for Binance execution clients (used by the trading node builder).

:::note
Most users will simply define a configuration for a live trading node (as below),
and won't need to necessarily work with these lower level components directly.
:::

Data Types

To provide complete API functionality to traders, the integration includes several
custom data types:

- BinanceTicker: Represents data returned for Binance 24-hour ticker subscriptions, including comprehensive price and statistical information.
- BinanceBar: Represents data for historical requests or real-time subscriptions to Binance bars, with additional volume metrics.
- BinanceFuturesMarkPriceUpdate: Represents mark price updates for Binance Futures subscriptions.

See the Binance [API Reference](../apireference/adapters/binance.md) for full definitions.

Symbology

As per the Nautilus unification policy for symbols, the native Binance symbols are used where possible including for
spot assets and futures contracts. Because NautilusTrader is capable of multi-venue + multi-account
trading, it's necessary to explicitly clarify the difference between BTCUSDT as the spot and margin traded
pair, and the BTCUSDT perpetual futures contract (this symbol is used for both natively by Binance).

Therefore, Nautilus appends the suffix -PERP to all perpetual symbols.
E.g. for Binance Futures, the BTCUSDT perpetual futures contract symbol would be BTCUSDT-PERP within the Nautilus system boundary.

Capability Matrix

The following tables detail the order types, execution instructions, and time-in-force options supported across different Binance account types:

Order Types

Order Type	Spot	Margin	USDT Futures	Coin Futures	Notes
MARKET	✓	✓	✓	✓	
LIMIT	✓	✓	✓	✓	
STOPMARKET	-	✓	✓	✓	Not supported for Spot.
STOPLIMIT	✓	✓	✓	✓	
MARKETIFTOUCHED	-	-	✓	✓	Futures only.
LIMITIFTOUCHED	✓	✓	✓	✓	
TRAILINGSTOPMARKET	-	-	✓	✓	Futures only.

Execution Instructions

Instruction	Spot	Margin	USDT Futures	Coin Futures	Notes
postonly	✓	✓	✓	✓	See restrictions below.
reduceonly	-	-	✓	✓	Futures only; disabled in Hedge Mode.

Post-Only Restrictions

Only limit order types support postonly.

Order Type	Spot	Margin	USDT Futures	Coin Futures	Notes
LIMIT					Uses LIMITMAKER for Spot/Margin, GTX TIF for Futures.
STOPLIMIT	-	-			Not supported for Spot/Margin.

Time-in-Force Options

Time-in-Force	Spot	Margin	USDT Futures	Coin Futures	Notes
GTC					Good Till Canceled.
GTD					Converted to GTC for Spot/Margin with warning.
FOK					Fill or Kill.
IOC					Immediate or Cancel.

Advanced Order Features

Feature	Spot	Margin	USDT Futures	Coin Futures	Notes
Order Modification					Price and quantity for LIMIT orders only.
Bracket/OCO Orders					One-Cancels-Other for stop loss/take profit.
Iceberg Orders					Large orders split into visible portions.

Configuration Options

The following execution client configuration options affect order behavior:

Option	Default	Description
usegtd	True	If True, uses Binance GTD TIF; if False, remaps GTD to GTC for local management.
usereduceonly	True	If True, sends reduceonly instruction to exchange; if False, always sends False.
usepositionids	True	If True, uses Binance Futures hedging position IDs; if False, enables virtual positions.
treatexpiredascanceled	False	If True, treats EXPIRED execution type as CANCELED for consistent handling.
futuresleverages	None	Dict to set initial leverage per symbol for Futures accounts.
futuresmargintypes	None	Dict to set margin type (isolated/cross) per symbol for Futures accounts.

Trailing Stops

Binance uses the concept of an activation price for trailing stops, as detailed in their [documentation](https://www.binance.com/en/support/faq/what-is-a-trailing-stop-order-360042299292). This approach is somewhat unconventional. For trailing stop orders to function on Binance, the activation price should be set using the activationprice parameter.

Note that the activation price is not the same as the trigger/STOP price. Binance will always calculate the trigger price for the order based on the current market price and the callback rate provided by trailingoffset.
The activation price is simply the price at which the order will begin trailing based on the callback rate.

:::warning
For Binance trailing stop orders, you must use activationprice instead of triggerprice. Using triggerprice will result in an order rejection.

:::

When submitting trailing stop orders from your strategy, you have two options:

1. Use the activationprice to manually set the activation price.
2. Leave the activationprice as None, activating the trailing mechanism immediately.

You must also have at least one of the following:

- The activationprice for the order is set.
- (or) you have subscribed to quotes for the instrument you're submitting the order for (used to infer activation price).
- (or) you have subscribed to trades for the instrument you're submitting the order for (used to infer activation price).

Configuration

The most common use case is to configure a live TradingNode to include Binance data and execution clients. To achieve this, add a BINANCE section to your client configuration(s):

Then, create a TradingNode and add the client factories:

API Credentials

There are two options for supplying your credentials to the Binance clients.
Either pass the corresponding apikey and apisecret values to the configuration objects, or set the following environment variables:

For Binance Spot/Margin live clients, you can set:

- BINANCEAPIKEY
- BINANCEAPISECRET

For Binance Spot/Margin testnet clients, you can set:

- BINANCETESTNETAPIKEY
- BINANCETESTNETAPISECRET

For Binance Futures live clients, you can set:

- BINANCEFUTURESAPIKEY
- BINANCEFUTURESAPISECRET

For Binance Futures testnet clients, you can set:

- BINANCEFUTURESTESTNETAPIKEY
- BINANCEFUTURESTESTNETAPISECRET

When starting the trading node, you'll receive immediate confirmation of whether your credentials are valid and have trading permissions.

Account Type

All the Binance account types will be supported for live trading. Set the accounttype using the BinanceAccountType enum. The account type options are:

- SPOT
- MARGIN (Margin shared between open positions)
- ISOLATEDMARGIN (Margin assigned to a single position)
- USDTFUTURE (USDT or BUSD stablecoins as collateral)

- COINFUTURE (other cryptocurrency as collateral)

:::tip
We recommend using environment variables to manage your credentials.
:::

Base URL Overrides

It's possible to override the default base URLs for both HTTP Rest and
WebSocket APIs. This is useful for configuring API clusters for performance reasons,
or when Binance has provided you with specialized endpoints.

Binance US

There is support for Binance US accounts by setting the us option in the configs
to True (this is False by default). All functionality available to US accounts
should behave identically to standard Binance.

Testnets

It's also possible to configure one or both clients to connect to the Binance testnet.
Simply set the testnet option to True (this is False by default):

Aggregated Trades

Binance provides aggregated trade data endpoints as an alternative source of trades.
In comparison to the default trade endpoints, aggregated trade data endpoints can return all
ticks between a starttime and endtime.

To use aggregated trades and the endpoint features, set the useaggtradeticks option
to True (this is False by default.)

Parser Warnings

Some Binance instruments are unable to be parsed into Nautilus objects if they
contain enormous field values beyond what can be handled by the platform.
In these cases, a warn and continue approach is taken (the instrument will not
be available).

These warnings may cause unnecessary log noise, and so it's possible to
configure the provider to not log the warnings, as per the client configuration
example below:

Futures Hedge Mode

Binance Futures Hedge mode is a position mode where a trader opens positions in both long and short
directions to mitigate risk and potentially profit from market volatility.

To use Binance Future Hedge mode, you need to follow the three items below:

- 1. Before starting the strategy, ensure that hedge mode is configured on Binance.
- 2. Set the usereduceonly option to False in BinanceExecClientConfig (this is True by default).

- 3. When submitting an order, use a suffix (LONG or SHORT) in the positionid to indicate the position direction.

Order books

Order books can be maintained at full or partial depths depending on the

subscription. WebSocket stream throttling is different between Spot and Futures exchanges, Nautilus will use the highest streaming rate possible:

Order books can be maintained at full or partial depths based on the subscription settings. WebSocket stream update rates differ between Spot and Futures exchanges, with Nautilus using the highest available streaming rate:

- Spot: 100ms
- Futures: 0ms (unthrottled)

There is a limitation of one order book per instrument per trader instance.
As stream subscriptions may vary, the latest order book data (deltas or snapshots) subscription will be used by the Binance data client.

Order book snapshot rebuilds will be triggered on:

- Initial subscription of the order book data.
- Data websocket reconnects.

The sequence of events is as follows:

- Deltas will start buffered.
- Snapshot is requested and awaited.
- Snapshot response is parsed to OrderBookDeltas.
- Snapshot deltas are sent to the DataEngine.
- Buffered deltas are iterated, dropping those where the sequence number is not greater than the last delta in the snapshot.
- Deltas will stop buffering.
- Remaining deltas are sent to the DataEngine.

Binance data differences

The tsevent field value for QuoteTick objects will differ between Spot and Futures exchanges, where the former does not provide an event timestamp, so the tsinit is used (which means tsevent and tsinit are identical).

Binance specific data

It's possible to subscribe to Binance specific data streams as they become available to the adapter over time.

:::note
Bars are not considered 'Binance specific' and can be subscribed to in the normal way.
As more adapters are built out which need for example mark price and funding rate updates, then these methods may eventually become first-class (not requiring custom/generic subscriptions as below).
:::

BinanceFuturesMarkPriceUpdate

You can subscribe to BinanceFuturesMarkPriceUpdate (including funding rating info) data streams by subscribing in the following way from your actor or strategy:

This will result in your actor/strategy passing these received BinanceFuturesMarkPriceUpdate objects to your ondata method. You will need to check the type, as this method acts as a flexible handler for all custom/generic data.

integrations/bybit.md

Bybit

:::info
We are currently working on this integration guide.
:::

Founded in 2018, Bybit is one of the largest cryptocurrency exchanges in terms of daily trading volume, and open interest of crypto assets and crypto derivative products. This integration supports live market data ingest and order execution with Bybit.

Installation

To install NautilusTrader with Bybit support:

To build from source with all extras (including Bybit):

Examples

You can find live example scripts [here](https://github.com/nautechsystems/nautilustrader/tree/develop/examples/live/bybit/).

Overview

This guide assumes a trader is setting up for both live market data feeds, and trade execution.
The Bybit adapter includes multiple components, which can be used together or separately depending on the use case.

- BybitHttpClient: Low-level HTTP API connectivity.
- BybitWebSocketClient: Low-level WebSocket API connectivity.
- BybitInstrumentProvider: Instrument parsing and loading functionality.
- BybitDataClient: A market data feed manager.
- BybitExecutionClient: An account management and trade execution gateway.
- BybitLiveDataClientFactory: Factory for Bybit data clients (used by the trading node builder).
- BybitLiveExecClientFactory: Factory for Bybit execution clients (used by the trading node builder).

:::note
Most users will simply define a configuration for a live trading node (as below),
and won't need to necessarily work with these lower level components directly.
:::

Bybit documentation

Bybit provides extensive documentation for users which can be found in the [Bybit help center](https://www.bybit.com/en/help-center).
It's recommended you also refer to the Bybit documentation in conjunction with this NautilusTrader integration guide.

Products

A product is an umbrella term for a group of related instrument types.

:::note
Product is also referred to as category in the Bybit v5 API.
:::

The following product types are supported on Bybit:

- Spot cryptocurrencies
- Perpetual contracts
- Perpetual inverse contracts
- Futures contracts
- Futures inverse contracts

Options contracts are not currently supported (will be implemented in a future version)

Symbology

To distinguish between different product types on Bybit, Nautilus uses specific product category suffixes for symbols:

- -SPOT: Spot cryptocurrencies
- -LINEAR: Perpetual and futures contracts
- -INVERSE: Inverse perpetual and inverse futures contracts
- -OPTION: Options contracts (not currently supported)

These suffixes must be appended to the Bybit raw symbol string to identify the specific product type for the instrument ID. For example:

- The Ether/Tether spot currency pair is identified with -SPOT, such as ETHUSDT-SPOT.
- The BTCUSDT perpetual futures contract is identified with -LINEAR, such as BTCUSDT-LINEAR.
- The BTCUSD inverse perpetual futures contract is identified with -INVERSE, such as BTCUSD-INVERSE.

Capability Matrix

Bybit offers a flexible combination of trigger types, enabling a broader range of Nautilus orders.
All the order types listed below can be used as either entries or exits, except for trailing stops (which utilize a position-related API).

Order Types

Order Type	Spot	Linear	Inverse	Notes
MARKET				
LIMIT				
STOPMARKET				
STOPLIMIT				
MARKETIFTOUCHED				
LIMITIFTOUCHED				
TRAILINGSTOPMARKET	-			Not supported for Spot.

Execution Instructions

Instruction	Spot	Linear	Inverse	Notes
postonly				Only supported on LIMIT orders.
reduceonly	-			Not supported for Spot products.

Time-in-Force Options

Time-in-Force	Spot	Linear	Inverse	Notes
GTC				Good Till Canceled.
GTD	-	-	-	Not supported.
FOK				Fill or Kill.
IOC				Immediate or Cancel.

Advanced Order Features

Feature	Spot	Linear	Inverse	Notes
Order Modification				Price and quantity modification.
Bracket/OCO Orders				UI only; API users implement manually.
Iceberg Orders				Max 10 per account, 1 per symbol.

Configuration Options

The following execution client configuration options affect order behavior:

Option	Default	Description
usegtd	False	GTD is not supported; orders are remapped to GTC for local management.
usewstradeapi	False	If True, uses WebSocket for order requests instead of HTTP.
usehttpbatchapi	False	If True, uses HTTP batch API when WebSocket trading is enabled.
futuresleverages	None	Dict to set leverage for futures symbols.
positionmode	None	Dict to set position mode for USDT perpetual and inverse futures.
marginmode	None	Sets margin mode for the account.

Product-Specific Limitations

The following limitations apply to SPOT products, as positions are not tracked on the venue side:

- reduceonly orders are not supported.
- Trailing stop orders are not supported.

Trailing Stops

Trailing stops on Bybit do not have a client order ID on the venue side (though there is a venueorderid). This is because trailing stops are associated with a netted position for an instrument.
Consider the following points when using trailing stops on Bybit:

- reduceonly instruction is available
- When the position associated with a trailing stop is closed, the trailing stop is automatically "deactivated" (closed) on the venue side
- You cannot query trailing stop orders that are not already open (the venueorderid is unknown until then)
- You can manually adjust the trigger price in the GUI, which will update the Nautilus order

Configuration

The product types for each client must be specified in the configurations.

Data Clients

If no product types are specified then all product types will be loaded and available.

Execution Clients

Because Nautilus does not support a "unified" account, the account type must be either cash or margin. This means there is a limitation that you cannot specify SPOT with any of the other derivative product types.

- CASH account type will be used for SPOT products.
- MARGIN account type will be used for all other derivative products.

The most common use case is to configure a live TradingNode to include Bybit

data and execution clients. To achieve this, add a BYBIT section to your client configuration(s):

Then, create a TradingNode and add the client factories:

API Credentials

There are two options for supplying your credentials to the Bybit clients.
Either pass the corresponding apikey and apisecret values to the configuration objects, or set the following environment variables:

For Bybit live clients, you can set:

- BYBITAPIKEY
- BYBITAPISECRET

For Bybit demo clients, you can set:

- BYBITDEMOAPIKEY
- BYBITDEMOAPISECRET

For Bybit testnet clients, you can set:

- BYBITTESTNETAPIKEY
- BYBITTESTNETAPISECRET

:::tip
We recommend using environment variables to manage your credentials.
:::

When starting the trading node, you'll receive immediate confirmation of whether your credentials are valid and have trading permissions.

integrations/coinbase_intx.md

Coinbase International

This guide will walk you through using Coinbase International with NautilusTrader for data ingest and/or live trading.

:::warning
The Coinbase International integration is currently in a beta testing phase.
Exercise caution and report any issues on GitHub.
:::

[Coinbase International Exchange](https://www.coinbase.com/en/international-exchange) provides non-US institutional clients with access to cryptocurrency perpetual futures and spot markets.
The exchange serves European and international traders by providing leveraged crypto derivatives, often restricted or unavailable in these regions.

Coinbase International brings a high standard of customer protection, a robust risk management framework and high-performance trading technology, including:

- Real-time 24/7/365 risk management.
- Liquidity from external market makers (no proprietary trading).
- Dynamic margin requirements and collateral assessments.
- Liquidation framework that meets rigorous compliance standards.
- Well-capitalized exchange to support tail market events.
- Collaboration with top-tier global regulators.

:::info
See the [Introducing Coinbase International Exchange](https://www.coinbase.com/en-au/blog/introducing-coinbase-international-exchange) blog article for more details.
:::

Installation

:::note
No additional coinbaseintx installation is required; the adapter's core components, written in Rust, are automatically compiled and linked during the build.
:::

Examples

You can find live example scripts [here](https://github.com/nautechsystems/nautilustrader/tree/develop/examples/live/coinbaseintx).
These examples demonstrate how to set up live market data feeds and execution clients for trading on Coinbase International.

Overview

The following products are supported on the Coinbase International exchange:

- Perpetual Futures contracts
- Spot cryptocurrencies

This guide assumes a trader is setting up for both live market data feeds, and trade execution.
The Coinbase International adapter includes multiple components, which can be used together or separately depending on the use case. These components work together to connect to Coinbase International's APIs

for market data and execution.

- CoinbaseIntxHttpClient: REST API connectivity.
- CoinbaseIntxWebSocketClient: WebSocket API connectivity.
- CoinbaseIntxInstrumentProvider: Instrument parsing and loading functionality.
- CoinbaseIntxDataClient: A market data feed manager.
- CoinbaseIntxExecutionClient: An account management and trade execution gateway.
- CoinbaseIntxLiveDataClientFactory: Factory for Coinbase International data clients.
- CoinbaseIntxLiveExecClientFactory: Factory for Coinbase International execution clients.

:::note
Most users will simply define a configuration for a live trading node (described below),
and won't necessarily need to work with the above components directly.
:::

Coinbase documentation

Coinbase International provides extensive API documentation for users which can be found in the [Coinbase Developer Platform](https://docs.cdp.coinbase.com/intx/docs/welcome).
We recommend also referring to the Coinbase International documentation in conjunction with this NautilusTrader integration guide.

Data

Instruments

On startup, the adapter automatically loads all available instruments from the Coinbase International REST API
and subscribes to the INSTRUMENTS WebSocket channel for updates. This ensures that the cache and clients requiring
up-to-date definitions for parsing always have the latest instrument data.

Available instrument types include:

- CurrencyPair (Spot cryptocurrencies)
- CryptoPerpetual

:::note
Index products have not yet been implemented.
:::

The following data types are available:

- OrderBookDelta (L2 market-by-price)
- QuoteTick (L1 top-of-book best bid/ask)
- TradeTick
- Bar
- MarkPriceUpdate
- IndexPriceUpdate

:::note
Historical data requests have not yet been implemented.
:::

WebSocket market data

The data client connects to Coinbase International's WebSocket feed to stream real-time market data.
The WebSocket client handles automatic reconnection and re-subscribes to active subscriptions upon reconnecting.

Execution

The adapter is designed to trade one Coinbase International portfolio per execution client.

Selecting a Portfolio

To identify your available portfolios and their IDs, use the REST client by running the following script:

This will output a list of portfolio details, similar to the example below:

Configuring the Portfolio

To specify a portfolio for trading, set the COINBASEINTXPORTFOLIOID environment variable to the desired portfolioid. If you're using multiple execution clients, you can alternatively define the portfolioid in the execution configuration for each client.

Capability Matrix

Coinbase International offers market, limit, and stop order types, enabling a broad range of strategies.

Order Types

Order Type	Derivatives	Spot	Notes
MARKET			Must use IOC or FOK time-in-forc
LIMIT			
STOPMARKET			
STOPLIMIT			
MARKETIFTOUCHED	-	-	Not supported.
LIMITIFTOUCHED	-	-	Not supported.
TRAILINGSTOPMARKET	-	-	Not supported.

Execution Instructions

Instruction	Derivatives	Spot	Notes
postonly			Ensures orders only provide liquidity.
reduceonly			Ensures orders only reduce existing positions.

Time-in-Force Options

Time-in-Force	Derivatives	Spot	Notes
GTC			Good Till Canceled.
GTD			Good Till Date.
FOK			Fill or Kill.
IOC			Immediate or Cancel.

Advanced Order Features

Feature	Derivatives	Spot	Notes
Order Modification			Price and quantity modification.
Bracket/OCO Orders	?	?	Requires further investigation.
Iceberg Orders			Available via FIX protocol.

Configuration Options

The following execution client configuration options are available:

| Option | Default | Description |

|---|---|---|
| portfolioid | None | Specifies the Coinbase International portfolio to trade. Required for execution. |
| httptimeoutsecs | 60 | Default timeout for HTTP requests in seconds. |

FIX Drop Copy Integration

The Coinbase International adapter includes a FIX (Financial Information eXchange) [drop copy](https://docs.cdp.coinbase.com/intx/docs/fix-msg-drop-copy) client.
This provides reliable, low-latency execution updates directly from Coinbase's matching engine.

:::note
This approach is necessary because execution messages are not provided over the WebSocket feed, and delivers faster and more reliable order execution updates than polling the REST API.
:::

The FIX client:

- Establishes a secure TCP/TLS connection and logs on automatically when the trading node starts.
- Handles connection monitoring and automatic reconnection and logon if the connection is interrupted.
- Properly logs out and closes the connection when the trading node stops.

The client processes several types of execution messages:

- Order status reports (canceled, expired, triggered).
- Fill reports (partial and complete fills).

The FIX credentials are automatically managed using the same API credentials as the REST and WebSocket clients.
No additional configuration is required beyond providing valid API credentials.

:::note
The REST client handles processing REJECTED and ACCEPTED status execution messages on order submission.
:::

Account and Position Management

On startup, the execution client requests and loads your current account and execution state including:

- Available balances across all assets.
- Open orders.
- Open positions.

This provides your trading strategies with a complete picture of your account before placing new orders.

Configuration

Strategies

:::warning
Coinbase International has a strict specification for client order IDs.
Nautilus can meet the spec by using UUID4 values for client order IDs.
To comply, set the useuuidclientorderids=True config option in your strategy configuration (otherwise, order submission will trigger an API error).

See the Coinbase International [Create order](https://docs.cdp.coinbase.com/intx/reference/createorder) REST API documentation for further details.
:::

An example configuration could be:

Then, create a TradingNode and add the client factories:

API Credentials

Provide credentials to the clients using one of the following methods.

Either pass values for the following configuration options:

- apikey
- apisecret
- apipassphrase
- portfolioid

Or, set the following environment variables:

- COINBASEINTXAPIKEY
- COINBASEINTXAPISECRET
- COINBASEINTXAPIPASSPHRASE
- COINBASEINTXPORTFOLIOID

:::tip
We recommend using environment variables to manage your credentials.
:::

When starting the trading node, you'll receive immediate confirmation of whether your credentials are valid and have trading permissions.

Implementation notes

- Heartbeats: The adapter maintains heartbeats on both the WebSocket and FIX connections to ensure reliable connectivity.
- Rate Limits: The REST API client is configured to limit requests to the 40/second, as specified by Coinbase International.
- Graceful Shutdown: The adapter properly handles graceful shutdown, ensuring all pending messages are processed before disconnecting.
- Thread Safety: All adapter components are thread-safe, allowing them to be used from multiple threads concurrently.
- Execution Model: The adapter can be configured with a single Coinbase International portfolio per execution client. For trading multiple portfolios, you can create multiple execution clients.

integrations/databento.md

Databento

NautilusTrader provides an adapter for integrating with the Databento API and [Databento Binary Encoding (DBN)](https://databento.com/docs/standards-and-conventions/databento-binary-encoding) format data.
As Databento is purely a market data provider, there is no execution client provided - although a sandbox environment with simulated execution could still be set up.
It's also possible to match Databento data with Interactive Brokers execution, or to calculate traditional asset class signals for crypto trading.

The capabilities of this adapter include:

- Loading historical data from DBN files and decoding into Nautilus objects for backtesting or writing to the data catalog.
- Requesting historical data which is decoded to Nautilus objects to support live trading and backtesting.
- Subscribing to real-time data feeds which are decoded to Nautilus objects to support live trading and sandbox environments.

:::tip
[Databento](https://databento.com/signup) currently offers 125 USD in free data credits (historical data only) for new account sign-ups.

With careful requests, this is more than enough for testing and evaluation purposes.
We recommend you make use of the [/metadata.getcost](https://databento.com/docs/api-reference-historical/metadata/metadata-get-cost) endpoint.
:::

Overview

The adapter implementation takes the [databento-rs](https://crates.io/crates/databento) crate as a dependency,
which is the official Rust client library provided by Databento.

:::info
There is no need for an optional extra installation of databento, as the core components of the adapter are compiled as static libraries and linked automatically during the build process.
:::

The following adapter classes are available:

- DatabentoDataLoader: Loads Databento Binary Encoding (DBN) data from files.
- DatabentoInstrumentProvider: Integrates with the Databento API (HTTP) to provide latest or historical instrument definitions.
- DatabentoHistoricalClient: Integrates with the Databento API (HTTP) for historical market data requests.
- DatabentoLiveClient: Integrates with the Databento API (raw TCP) for subscribing to real-time data feeds.
- DatabentoDataClient: Provides a LiveMarketDataClient implementation for running a trading node in real time.

:::info
As with the other integration adapters, most users will simply define a configuration for a live trading node (covered below),

and won't need to necessarily work with these lower level components directly.
:::

Examples

You can find live example scripts [here](https://github.com/nautechsystems/nautilustrader/tree/develop/examples/live/databento/).

Databento documentation

Databento provides extensive documentation for new users which can be found in the [Databento new users guide](https://databento.com/docs/quickstart/new-user-guides).
We recommend also referring to the Databento documentation in conjunction with this NautilusTrader integration guide.

Databento Binary Encoding (DBN)

Databento Binary Encoding (DBN) is an extremely fast message encoding and storage format for normalized market data.
The [DBN specification](https://databento.com/docs/standards-and-conventions/databento-binary-encoding) includes a simple, self-describing metadata header and a fixed set of struct definitions,
which enforce a standardized way to normalize market data.

The integration provides a decoder which can convert DBN format data to Nautilus objects.

The same Rust implemented Nautilus decoder is used for:

- Loading and decoding DBN files from disk
- Decoding historical and live data in real time

Supported schemas

The following Databento schemas are supported by NautilusTrader:

Databento schema	Nautilus data type
MBO	OrderBookDelta
MBP1	(QuoteTick, Option<TradeTick>)
MBP10	OrderBookDepth10
BBO1S	QuoteTick
BBO1M	QuoteTick
TBBO	(QuoteTick, TradeTick)
TRADES	TradeTick
OHLCV1S	Bar
OHLCV1M	Bar
OHLCV1H	Bar
OHLCV1D	Bar
DEFINITION	Instrument (various types)
IMBALANCE	DatabentoImbalance
STATISTICS	DatabentoStatistics
STATUS	InstrumentStatus

:::warning
NautilusTrader no longer supports Databento DBN v1 schema decoding.
You will need to migrate historical DBN v1 data to v2 or v3 for loading.
:::

:::info

See also the Databento [Schemas and data formats](https://databento.com/docs/schemas-and-data-formats) guide.
:::

Instrument IDs and symbology

Databento market data includes an instrumentid field which is an integer assigned
by either the original source venue, or internally by Databento during normalization.

It's important to realize that this is different to the Nautilus InstrumentId
which is a string made up of a symbol + venue with a period separator i.e. "{symbol}.{venue}".

The Nautilus decoder will use the Databento rawsymbol for the Nautilus symbol and an [ISO 10383 MIC](https://www.iso20022.org/market-identifier-codes) (Market Identifier Code)
from the Databento instrument definition message for the Nautilus venue.

Databento datasets are identified with a dataset code which is not the same
as a venue identifier. You can read more about Databento dataset naming conventions [here](https://databento.com/docs/api-reference-historical/basics/datasets).

Of particular note is for CME Globex MDP 3.0 data (GLBX.MDP3 dataset code), the following
exchanges are all grouped under the GLBX venue. These mappings can be determined from the
instruments exchange field:

- CBCM: XCME-XCBT inter-exchange spread
- NYUM: XNYM-DUMX inter-exchange spread
- XCBT: Chicago Board of Trade (CBOT)
- XCEC: Commodities Exchange Center (COMEX)
- XCME: Chicago Mercantile Exchange (CME)
- XFXS: CME FX Link spread
- XNYM: New York Mercantile Exchange (NYMEX)

:::info
Other venue MICs can be found in the venue field of responses from the [metadata.listpublishers](https://databento.com/docs/api-reference-historical/metadata/metadata-list-publishers) endpoint.
:::

Timestamps

Databento data includes various timestamp fields including (but not limited to):

- tsevent: The matching-engine-received timestamp expressed as the number of nanoseconds since the UNIX epoch.
- tsindelta: The matching-engine-sending timestamp expressed as the number of nanoseconds before tsrecv.
- tsrecv: The capture-server-received timestamp expressed as the number of nanoseconds since the UNIX epoch.
- tsout: The Databento sending timestamp.

Nautilus data includes at least two timestamps (required by the Data contract):

- tsevent: UNIX timestamp (nanoseconds) when the data event occurred.
- tsinit: UNIX timestamp (nanoseconds) when the data object was created.

When decoding and normalizing Databento to Nautilus we generally assign the Databento tsrecv value to the Nautilus
tsevent field, as this timestamp is much more reliable and consistent, and is guaranteed to be monotonically increasing per instrument.

The exception to this are the DatabentoImbalance and DatabentoStatistics data types, which have fields for all timestamps as these types are defined specifically for the adapter.

:::info
See the following Databento docs for further information:

- [Databento standards and conventions - timestamps](https://databento.com/docs/standards-and-conventions/common-fields-enums-types#timestamps)
- [Databento timestamping guide](https://databento.com/docs/architecture/timestamping-guide)

:::

Data types

The following section discusses Databento schema -> Nautilus data type equivalence and considerations.

:::info
See Databento [schemas and data formats](https://databento.com/docs/schemas-and-data-formats).
:::

Instrument definitions

Databento provides a single schema to cover all instrument classes, these are decoded to the appropriate Nautilus Instrument types.

The following Databento instrument classes are supported by NautilusTrader:

Databento instrument class	Code	Nautilus instrument type
Stock	K	Equity
Future	F	FuturesContract
Call	C	OptionContract
Put	P	OptionContract
Future spread	S	FuturesSpread
Option spread	T	OptionSpread
Mixed spread	M	OptionSpread
FX spot	X	CurrencyPair
Bond	B	Not yet available

MBO (market by order)

This schema is the highest granularity data offered by Databento, and represents full order book depth. Some messages also provide trade information, and so when decoding MBO messages Nautilus will produce an OrderBookDelta and optionally a TradeTick.

The Nautilus live data client will buffer MBO messages until an FLAST flag is seen. A discrete OrderBookDeltas container object will then be passed to the registered handler.

Order book snapshots are also buffered into a discrete OrderBookDeltas container object, which occurs during the replay startup sequence.

MBP-1 (market by price, top-of-book)

This schema represents the top-of-book only (quotes and trades). Like with MBO messages, some messages carry trade information, and so when decoding MBP-1 messages Nautilus

will produce a QuoteTick and also a TradeTick if the message is a trade.

OHLCV (bar aggregates)

The Databento bar aggregation messages are timestamped at the open of the bar interval.
The Nautilus decoder will normalize the tsevent timestamps to the close of the bar
(original tsevent + bar interval).

Imbalance & Statistics

The Databento imbalance and statistics schemas cannot be represented as a built-in Nautilus data types,
and so they have specific types defined in Rust DatabentoImbalance and DatabentoStatistics.
Python bindings are provided via pyo3 (Rust) so the types behave a little differently to a built-in Nautilus data types, where all attributes are pyo3 provided objects and not directly compatible
with certain methods which may expect a Cython provided type. There are pyo3 -> legacy Cython object conversion methods available, which can be found in the API reference.

Here is a general pattern for converting a pyo3 Price to a Cython Price:

Additionally requesting for and subscribing to these data types requires the use of the
lower level generic methods for custom data types. The following example subscribes to the imbalance schema for the AAPL.XNAS instrument (Apple Inc trading on the Nasdaq exchange):

Or requesting the previous days statistics schema for the ES.FUT parent symbol (all active E-mini S&P 500 futures contracts on the CME Globex exchange):

Performance considerations

When backtesting with Databento DBN data, there are two options:

- Store the data in DBN (.dbn.zst) format files and decode to Nautilus objects on every run
- Convert the DBN files to Nautilus objects and then write to the data catalog once (stored as Nautilus Parquet format on disk)

Whilst the DBN -> Nautilus decoder is implemented in Rust and has been optimized,
the best performance for backtesting will be achieved by writing the Nautilus
objects to the data catalog, which performs the decoding step once.

[DataFusion](https://arrow.apache.org/datafusion/) provides a query engine backend to efficiently load and stream
the Nautilus Parquet data from disk, which achieves extremely high through-put (at least an order of magnitude faster
than converting DBN -> Nautilus on the fly for every backtest run).

:::note
Performance benchmarks are currently under development.
:::

Loading DBN data

You can load DBN files and convert the records to Nautilus objects using the
DatabentoDataLoader class. There are two main purposes for doing so:

- Pass the converted data to BacktestEngine.adddata directly for backtesting.
- Pass the converted data to ParquetDataCatalog.writedata for later streaming use with a BacktestNode.

DBN data to a BacktestEngine

This code snippet demonstrates how to load DBN data and pass to a BacktestEngine.
Since the BacktestEngine needs an instrument added, we'll use a test instrument
provided by the TestInstrumentProvider (you could also pass an instrument object
which was parsed from a DBN file too).
The data is a month of TSLA (Tesla Inc) trades on the Nasdaq exchange:

DBN data to a ParquetDataCatalog

This code snippet demonstrates how to load DBN data and write to a ParquetDataCatalog.
We pass a value of false for the aslegacycython flag, which will ensure the
DBN records are decoded as pyo3 (Rust) objects. It's worth noting that legacy Cython
objects can also be passed to writedata, but these need to be converted back to
pyo3 objects under the hood (so passing pyo3 objects is an optimization).

:::info
See also the [Data concepts guide](../concepts/data.md).
:::

Real-time client architecture

The DatabentoDataClient is a Python class which contains other Databento adapter classes.
There are two DatabentoLiveClients per Databento dataset:

- One for MBO (order book deltas) real-time feeds
- One for all other real-time feeds

:::warning
There is currently a limitation that all MBO (order book deltas) subscriptions for a dataset have to be made at
node startup, to then be able to replay data from the beginning of the session. If subsequent subscriptions
arrive after start, then an error will be logged (and the subscription ignored).

There is no such limitation for any of the other Databento schemas.
:::

A single DatabentoHistoricalClient instance is reused between the DatabentoInstrumentProvider and DatabentoDataClient,
which makes historical instrument definitions and data requests.

Configuration

The most common use case is to configure a live TradingNode to include a
Databento data client. To achieve this, add a DATABENTO section to your client
configuration(s):

Then, create a TradingNode and add the client factory:

Configuration parameters

- apikey: The Databento API secret key. If None then will source the DATABENTOAPIKEY environment variable.
- httpgateway: The historical HTTP client gateway override (useful for testing and typically not needed by most users).
- livegateway: The raw TCP real-time client gateway override (useful for testing and typically not needed by most users).
- parentsymbols: The Databento parent symbols to subscribe to instrument definitions for on start. This is a map of Databento dataset keys -> to a sequence of the parent symbols, e.g. {'GLBX.MDP3', ['ES.FUT', 'ES.OPT']} (for all E-mini S&P 500 futures and options products).

- instrumentids: The instrument IDs to request instrument definitions for on start.
- timeoutinitialload: The timeout (seconds) to wait for instruments to load (concurrently per dataset).
- mbosubscriptionsdelay: The timeout (seconds) to wait for MBO/L3 subscriptions (concurrently per dataset). After the timeout the MBO order book feed will start and replay messages from the initial snapshot and then all deltas.

:::tip
We recommend using environment variables to manage your credentials.
:::

integrations/dydx.md

dYdX

:::info
We are currently working on this integration guide.
:::

dYdX is one of the largest decentralized cryptocurrency exchanges in terms of daily trading volume for crypto derivative products. dYdX runs on smart contracts on the Ethereum blockchain, and allows users to trade with no intermediaries. This integration supports live market data ingestion and order execution with dYdX v4, which is the first version of the protocol to be fully decentralized with no central components.

Installation

To install NautilusTrader with dYdX support:

To build from source with all extras (including dYdX):

Examples

You can find live example scripts [here](https://github.com/nautechsystems/nautilustrader/tree/develop/examples/live/dydx/).

Overview

This guide assumes a trader is setting up for both live market data feeds, and trade execution.
The dYdX adapter includes multiple components, which can be used together or separately depending on the use case.

- DYDXHttpClient: Low-level HTTP API connectivity.
- DYDXWebSocketClient: Low-level WebSocket API connectivity.
- DYDXAccountGRPCAPI: Low-level gRPC API connectivity for account updates.
- DYDXInstrumentProvider: Instrument parsing and loading functionality.
- DYDXDataClient: A market data feed manager.
- DYDXExecutionClient: An account management and trade execution gateway.
- DYDXLiveDataClientFactory: Factory for dYdX data clients (used by the trading node builder).
- DYDXLiveExecClientFactory: Factory for dYdX execution clients (used by the trading node builder).

:::note
Most users will simply define a configuration for a live trading node (as below),
and won't need to necessarily work with these lower level components directly.
:::

Symbology

Only perpetual contracts are available on dYdX. To be consistent with other adapters and to be futureproof in case other products become available on dYdX, NautilusTrader appends -PERP for all available perpetual symbols. For example, the Bitcoin/USD-C perpetual futures contract is identified as BTC-USD-PERP. The quote currency for all markets is USD-C. Therefore, dYdX abbreviates it to USD.

Short-term and long-term orders

dYdX makes a distinction between short-term orders and long-term orders (or stateful orders).
Short-term orders are meant to be placed immediately and belongs in the same block the order was received.

These orders stay in-memory up to 20 blocks, with only their fill amount and expiry block height being committed to state.
Short-term orders are mainly intended for use by market makers with high throughput or for market orders.

By default, all orders are sent as short-term orders. To construct long-term orders, you can attach a tag to
an order like this:

To specify the number of blocks that an order is active:

Market orders

Market orders require specifying a price to for price slippage protection and use hidden orders.
By setting a price for a market order, you can limit the potential price slippage. For example,
if you set the price of $100 for a market buy order, the order will only be executed if the market price is at or below $100. If the market price is above $100, the order will not be executed.

Some exchanges, including dYdX, support hidden orders. A hidden order is an order that is not visible to other market participants, but is still executable. By setting a price for a market order, you can create a hidden order that will only be executed if the market price reaches the specified price.

If the market price is not specified, a default value of 0 is used.

To specify the price when creating a market order:

Stop limit and stop market orders

Both stop limit and stop market conditional orders can be submitted. dYdX only supports long-term orders
for conditional orders.

Capability Matrix

dYdX supports perpetual futures trading with a comprehensive set of order types and execution features.

Order Types

Order Type	Perpetuals	Notes
MARKET		Requires price for slippage protection.
LIMIT		
STOPMARKET		Long-term orders only.
STOPLIMIT		Long-term orders only.
MARKETIFTOUCHED	-	Not supported.
LIMITIFTOUCHED	-	Not supported.
TRAILINGSTOPMARKET	-	Not supported.

Execution Instructions

Instruction	Perpetuals	Notes
postonly		Supported on all order types.
reduceonly		Supported on all order types.

Time-in-Force Options

Time-in-Force	Perpetuals	Notes

GTC		Good Till Canceled.
GTD		Good Till Date.
FOK		Fill or Kill.
IOC		Immediate or Cancel.

Advanced Order Features

Feature	Perpetuals	Notes
Order Modification		Short-term orders only; cancel-replace method.
Bracket/OCO Orders	-	Not supported.
Iceberg Orders	-	Not supported.

Configuration Options

The following execution client configuration options are available:

Option	Default	Description
subaccount	0	Subaccount number (venue creates subaccount 0 by default).
walletaddress	None	dYdX wallet address for the account.
mnemonic	None	Mnemonic for generating private key for order signing.
istestnet	False	If True, connects to testnet; if False, connects to mainnet.

Order Classification

dYdX classifies orders as either short-term or long-term orders:

- Short-term orders: Default for all orders; intended for high-frequency trading and market orders.
- Long-term orders: Required for conditional orders; use DYDXOrderTags to specify.

Configuration

The product types for each client must be specified in the configurations.

Execution Clients

The account type must be a margin account to trade the perpetual futures contracts.

The most common use case is to configure a live TradingNode to include dYdX
data and execution clients. To achieve this, add a DYDX section to your client
configuration(s):

Then, create a TradingNode and add the client factories:

API Credentials

There are two options for supplying your credentials to the dYdX clients.
Either pass the corresponding walletaddress and mnemonic values to the configuration objects, or
set the following environment variables:

For dYdX live clients, you can set:

- DYDXWALLETADDRESS
- DYDXMNEMONIC

For dYdX testnet clients, you can set:

- DYDXTESTNETWALLETADDRESS
- DYDXTESTNETMNEMONIC

:::tip
We recommend using environment variables to manage your credentials.
:::

The data client is using the wallet address to determine the trading fees. The trading fees are used during back tests only.

Testnets

It's also possible to configure one or both clients to connect to the dYdX testnet.
Simply set the istestnet option to True (this is False by default):

Parser Warnings

Some dYdX instruments are unable to be parsed into Nautilus objects if they
contain enormous field values beyond what can be handled by the platform.
In these cases, a warn and continue approach is taken (the instrument will not be available).

Order books

Order books can be maintained at full depth or top-of-book quotes depending on the
subscription. The venue does not provide quotes, but the adapter subscribes to order
book deltas and sends new quotes to the DataEngine when there is a top-of-book price or size change.

integrations/ib.md

Interactive Brokers

Interactive Brokers (IB) is a trading platform providing market access across a wide range of financial instruments, including stocks, options, futures, currencies, bonds, funds, and cryptocurrencies. NautilusTrader offers an adapter to integrate with IB using their [Trader Workstation (TWS) API](https://ibkrcampus.com/ibkr-api-page/trader-workstation-api/) through their Python library, [ibapi](https://github.com/nautechsystems/ibapi).

The TWS API serves as an interface to IB's standalone trading applications: TWS and IB Gateway. Both can be downloaded from the IB website. If you haven't installed TWS or IB Gateway yet, refer to the [Initial Setup](https://ibkrcampus.com/ibkr-api-page/trader-workstation-api/#tws-download) guide. In NautilusTrader, you'll establish a connection to one of these applications via the InteractiveBrokersClient.

Alternatively, you can start with a [dockerized version](https://github.com/gnzsnz/ib-gateway-docker) of the IB Gateway, which is particularly useful when deploying trading strategies on a hosted cloud platform. This requires having [Docker](https://www.docker.com/) installed on your machine, along with the [docker](https://pypi.org/project/docker/) Python package, which NautilusTrader conveniently includes as an extra package.

:::note
The standalone TWS and IB Gateway applications require manually inputting username, password, and trading mode (live or paper) at startup. The dockerized version of the IB Gateway handles these steps programmatically.
:::

Installation

To install NautilusTrader with Interactive Brokers (and Docker) support:

To build from source with all extras (including IB and Docker):

:::note
Because IB does not provide wheels for ibapi, NautilusTrader [repackages](https://pypi.org/project/nautilus-ibapi/) it for release on PyPI.
:::

Examples

You can find live example scripts [here](https://github.com/nautechsystems/nautilustrader/tree/develop/examples/live/interactivebrokers/).

Getting Started

Before implementing your trading strategies, please ensure that either TWS (Trader Workstation) or IB Gateway is currently running. You have the option to log in to one of these standalone applications using your personal credentials or alternatively, via DockerizedIBGateway.

Establish Connection to an Existing Gateway or TWS

Should you choose to connect to a pre-existing Gateway or TWS, it is crucial that you specify the host and port parameters in both the InteractiveBrokersDataClientConfig and InteractiveBrokersExecClientConfig to guarantee a successful connection.

Establish Connection to DockerizedIBGateway

In this case, it's essential to supply dockerizedgateway with an instance of DockerizedIBGatewayConfig

in both the InteractiveBrokersDataClientConfig and InteractiveBrokersExecClientConfig. It's important to stress, however, that host and port parameters aren't necessary in this context.
The following example provides a clear illustration of how to establish a connection to a Dockerized Gateway, which is judiciously managed internally by the Factories.

Note: To supply credentials to the Interactive Brokers Gateway, either pass the username and password to the DockerizedIBGatewayConfig, or set the following environment variables:

- TWSUSERNAME
- TWSPASSWORD

Overview

The adapter includes several major components:

- InteractiveBrokersClient: Executes TWS API requests using ibapi.
- HistoricInteractiveBrokersClient: Provides methods for retrieving instruments and historical data, useful for backtesting.
- InteractiveBrokersInstrumentProvider: Retrieves or queries instruments for trading.
- InteractiveBrokersDataClient: Connects to the Gateway for streaming market data.
- InteractiveBrokersExecutionClient: Handles account information and executes trades.

The Interactive Brokers Client

The InteractiveBrokersClient serves as the central component of the IB adapter, overseeing a range of critical functions. These include establishing and maintaining connections, handling API errors, executing trades, and gathering various types of data such as market data, contract/instrument data, and account details.

To ensure efficient management of these diverse responsibilities, the InteractiveBrokersClient is divided into several specialized mixin classes. This modular approach enhances manageability and clarity. The key subcomponents are:

- InteractiveBrokersClientConnectionMixin: This class is dedicated to managing the connection with TWS/Gateway.
- InteractiveBrokersClientErrorMixin: It focuses on addressing all encountered errors and warnings.
- InteractiveBrokersClientAccountMixin: Responsible for handling requests related to account information and positions.
- InteractiveBrokersClientContractMixin: Handles retrieving contracts (instruments) data.
- InteractiveBrokersClientMarketDataMixin: Handles market data requests, subscriptions and data processing.
- InteractiveBrokersClientOrderMixin: Oversees all aspects of order placement and management.

:::tip
To troubleshoot TWS API incoming message issues, consider starting at the InteractiveBrokersClient.processmessage method, which acts as the primary gateway for processing all messages received from the API.
:::

Symbology

The InteractiveBrokersInstrumentProvider supports three methods for constructing InstrumentId instances, which can be configured via the symbologymethod enum in InteractiveBrokersInstrumentProviderConfig.

Simplified Symbology

When symbologymethod is set to IBSIMPLIFIED (the default setting), the system utilizes the following parsing rules for symbology:

- Forex: The format is {symbol}/{currency}.{exchange}, where the currency pair is constructed as EUR/USD.IDEALPRO.
- Stocks: The format is {localSymbol}.{primaryExchange}. Any spaces in localSymbol are replaced with -, e.g., BF-B.NYSE.
- Futures: The format is {localSymbol}.{exchange}. Single digit years are expanded to two digits, e.g., ESM24.CME.
- Options: The format is {localSymbol}.{exchange}, with all spaces removed from localSymbol, e.g., AAPL230217P00155000.SMART.
- Index: The format is ^{localSymbol}.{exchange}, e.g., ^SPX.CBOE.

Raw Symbology

Setting symbologymethod to IBRAW enforces stricter parsing rules that align directly with the fields defined in the ibapi. The format for each security type is as follows:

- CFDs: {localSymbol}={secType}.IBCFD
- Commodities: {localSymbol}={secType}.IBCMDTY
- Default for Other Types: {localSymbol}={secType}.{exchange}

This configuration ensures that the symbology is explicitly defined and matched with the Interactive Brokers API requirements, providing clear and consistent instrument identification.
While this format may lack visual clarity, it is robust and supports instruments from any region,
especially those with non-standard symbology where simplified parsing may fail.

Databento Symbology

Setting symbologymethod to DATABENTO, the system utilized the symbology rules defined by DatabentoInstrumentProvider.
Note that this symbology is only compatible with venues supported by Databento and there is not automatic fall-back to other symbology methods to avoid any conflicts.

Instruments & Contracts

In IB, a NautilusTrader Instrument is equivalent to a [Contract](https://ibkrcampus.com/ibkr-api-page/trader-workstation-api/#contracts). Contracts can be either a [basic contract](https://ibkrcampus.com/ibkr-api-page/trader-workstation-api/#contract-object) or a more [detailed](https://ibkrcampus.com/ibkr-api-page/trader-workstation-api/#contract-details) version (ContractDetails). The adapter models these using IBContract and IBContractDetails classes. The latter includes critical data like order types and trading hours, which are absent in the basic contract. As a result, IBContractDetails can be converted to an Instrument while IBContract cannot.

To search for contract information, use the [IB Contract Information Center](https://pennies.interactivebrokers.com/cstools/contractinfo/).

It's typically suggested to utilize symbologymethod=SymbologyMethod.IBSIMPLIFIED (which is the default setting). This provides a cleaner and more intuitive use of InstrumentId by employing loadids in the InteractiveBrokersInstrumentProviderConfig, following the guidelines established in the Simplified Symbology section.
In order to load multiple Instruments, such as Options Instrument without having to specify each strike explicitly, you would need to utilize loadcontracts with provided instances of IBContract.

> Note: The secType and symbol should be specified for the Underlying Contract.\
> To specify an options exchange other than the underlying's, use the optionschainexchange parameter.

Some more examples of building IBContracts:

Historical Data & Backtesting

When developing strategies with the IB adapter, the first step usually involves acquiring historical data for backtesting. The HistoricInteractiveBrokersClient offers methods to request and save this data.

Here's an example of retrieving and saving instrument and bar data. A more comprehensive example is available
[here](https://github.com/nautechsystems/nautilustrader/blob/master/examples/live/interactivebrokers/historicdownload.py).

Live Trading

Engaging in live or paper trading requires constructing and running a TradingNode.
This node incorporates both InteractiveBrokersDataClient and InteractiveBrokersExecutionClient,
which depend on the InteractiveBrokersInstrumentProvider to operate.

InstrumentProvider

The InteractiveBrokersInstrumentProvider class functions as a bridge for accessing financial instrument data from IB.
Configurable through InteractiveBrokersInstrumentProviderConfig, it enables the customization of various instrument type parameters.
Additionally, this provider offers specialized methods to build and retrieve the entire futures and options chains.

Integration with Databento Data Client

To integrate with DatabentoDataClient, set the symbologymethod in InteractiveBrokersInstrumentProviderConfig
to SymbologyMethod.DATABENTO. This ensures seamless compatibility with Databento symbology, eliminating the need
for manual translations or mappings within your strategy.

When using this configuration:

- InteractiveBrokersInstrumentProvider will not publish instruments to the cache to prevent conflicts.
- Instruments Cache management must be handled exclusively by DatabentoDataClient.

Data Client

InteractiveBrokersDataClient interfaces with IB for streaming and retrieving market data. Upon connection, it configures the [market data type](https://ibkrcampus.com/ibkr-api-page/trader-workstation-api/#delayed-market-data)
and loads instruments based on the settings in InteractiveBrokersInstrumentProviderConfig.
This client can subscribe to and unsubscribe from various market data types, including quotes, trades, and bars.

Configurable through InteractiveBrokersDataClientConfig, it enables adjustments for handling revised bars,
trading hours preferences, and market data types (e.g., IBMarketDataTypeEnum.REALTIME or IBMarketDataTypeEnum.DELAYEDFROZEN).

Execution Client

The InteractiveBrokersExecutionClient facilitates executing trades, accessing account information,
and processing order and trade-related details. It encompasses a range of methods for order management,
including reporting order statuses, placing new orders, and modifying or canceling existing ones.
Additionally, it generates position reports, although fill reports are not yet implemented.

Full Configuration

Setting up a complete trading environment typically involves configuring a TradingNodeConfig, which includes data and execution client configurations. Additional configurations are specified in LiveDataEngineConfig
to accommodate IB-specific requirements. A TradingNode is then instantiated from these configurations,
and factories for creating InteractiveBrokersDataClient and InteractiveBrokersExecutionClient are added.
Finally, the node is built and run.

You can find additional examples here: <https://github.com/nautechsystems/nautilustrader/tree/develop/examples/live/interactivebrokers>

integrations/index.md

Integrations

NautilusTrader uses modular adapters to connect to trading venues and data providers, translating raw APIs into a unified interface and normalized domain model.

The following integrations are currently supported:

Name	ID	Type	Status	Docs
[Betfair](https://betfair.com)	BETFAIR	Sports Betting Exchange	![status](https://img.shields.io/badge/stable-green)	[Guide](integrations/betfair.md)
[Binance](https://binance.com)	BINANCE	Crypto Exchange (CEX)	![status](https://img.shields.io/badge/stable-green)	[Guide](integrations/binance.md)
[Binance US](https://binance.us)	BINANCE	Crypto Exchange (CEX)	![status](https://img.shields.io/badge/stable-green)	[Guide](integrations/binance.md)
[Binance Futures](https://www.binance.com/en/futures)	BINANCE	Crypto Exchange (CEX)	![status](https://img.shields.io/badge/stable-green)	[Guide](integrations/binance.md)
[Bybit](https://www.bybit.com)	BYBIT	Crypto Exchange (CEX)	![status](https://img.shields.io/badge/stable-green)	[Guide](integrations/bybit.md)
[Coinbase International](https://www.coinbase.com/en/international-exchange)	COINBASEINTX	Crypto Exchange (CEX)	![status](https://img.shields.io/badge/stable-green)	[Guide](integrations/coinbaseintx.md)
[Databento](https://databento.com)	DATABENTO	Data Provider	![status](https://img.shields.io/badge/stable-green)	[Guide](integrations/databento.md)
[dYdX](https://dydx.exchange/)	DYDX	Crypto Exchange (DEX)	![status](https://img.shields.io/badge/stable-green)	[Guide](integrations/dydx.md)
[Interactive Brokers](https://www.interactivebrokers.com)	INTERACTIVEBROKERS	Brokerage (multi-venue)	![status](https://img.shields.io/badge/stable-green)	[Guide](integrations/ib.md)
[OKX](https://okx.com)	OKX	Crypto Exchange (CEX)	![status](https://img.shields.io/badge/building-orange)	[Guide](integrations/okx.md)
[Polymarket](https://polymarket.com)	POLYMARKET	Prediction Market (DEX)	![status](https://img.shields.io/badge/stable-green)	[Guide](integrations/polymarket.md)
[Tardis](https://tardis.dev)	TARDIS	Crypto Data Provider	![status](https://img.shields.io/badge/stable-green)	[Guide](integrations/tardis.md)

- ID: The default client ID for the integrations adapter clients.
- Type: The type of integration (often the venue type).

Status

- building: Under construction and likely not in a usable state.
- beta: Completed to a minimally working state and in a 'beta' testing phase.
- stable: Stabilized feature set and API, the integration has been tested by both developers and users to a reasonable level (some bugs may still remain).

Implementation goals

The primary goal of NautilusTrader is to provide a unified trading system for

use with a variety of integrations. To support the widest range of trading strategies, priority will be given to standard functionality:

- Requesting historical market data
- Streaming live market data
- Reconciling execution state
- Submitting standard order types with standard execution instructions
- Modifying existing orders (if possible on an exchange)
- Canceling orders

The implementation of each integration aims to meet the following criteria:

- Low-level client components should match the exchange API as closely as possible.
- The full range of an exchange's functionality (where applicable to NautilusTrader) should eventually be supported.
- Exchange specific data types will be added to support the functionality and return types which are reasonably expected by a user.
- Actions unsupported by an exchange or NautilusTrader will be logged as a warning or error when invoked.

API unification

All integrations must conform to NautilusTrader's system API, requiring normalization and standardization:

- Symbols should use the venue's native symbol format unless disambiguation is required (e.g., Binance Spot vs. Binance Futures).
- Timestamps must use UNIX epoch nanoseconds. If milliseconds are used, field/property names should explicitly end with ms.

integrations/okx.md

OKX

:::warning
The OKX integration is still under development and only supports linear perpetual instruments.
:::

:::info
We are currently working on this integration guide.
:::

integrations/polymarket.md

Polymarket

Founded in 2020, Polymarket is the world's largest decentralized prediction market platform,
enabling traders to speculate on the outcomes of world events by buying and selling binary option contracts using cryptocurrency.

NautilusTrader provides a venue integration for data and execution via Polymarket's Central Limit Order Book (CLOB) API.
The integration leverages the [official Python CLOB client library](https://github.com/Polymarket/py-clob-client)
to facilitate interaction with the Polymarket platform.

NautilusTrader is designed to work with Polymarket's signature type 0, supporting EIP712 signatures from externally owned accounts (EOA).
This integration ensures that traders can execute orders securely and efficiently, using the most common on-chain signature method,
while NautilusTrader abstracts the complexity of signing and preparing orders for seamless execution.

Installation

To install NautilusTrader with Polymarket support:

To build from source with all extras (including Polymarket):

Examples

You can find live example scripts [here](https://github.com/nautechsystems/nautilustrader/tree/develop/examples/live/polymarket/).

Binary options

A [binary option](https://en.wikipedia.org/wiki/Binaryoption) is a type of financial exotic option contract in which traders bet on the outcome of a yes-or-no proposition.
If the prediction is correct, the trader receives a fixed payout; otherwise, they receive nothing.

All assets traded on Polymarket are quoted and settled in USDC.e (PoS), [see below](#usdce-pos) for more information.

Polymarket documentation

Polymarket offers comprehensive resources for different audiences:

- [Polymarket Learn](https://learn.polymarket.com/): Educational content and guides for users to understand the platform and how to engage with it.
- [Polymarket CLOB API](https://docs.polymarket.com/#introduction): Technical documentation for developers interacting with the Polymarket CLOB API.

Overview

This guide assumes a trader is setting up for both live market data feeds, and trade execution.
The Polymarket integration adapter includes multiple components, which can be used together or separately depending on the use case.

- PolymarketWebSocketClient: Low-level WebSocket API connectivity (built on top of the Nautilus WebSocketClient written in Rust).
- PolymarketInstrumentProvider: Instrument parsing and loading functionality for BinaryOption instruments.

- PolymarketDataClient: A market data feed manager.
- PolymarketExecutionClient: A trade execution gateway.
- PolymarketLiveDataClientFactory: Factory for Polymarket data clients (used by the trading node builder).
- PolymarketLiveExecClientFactory: Factory for Polymarket execution clients (used by the trading node builder).

:::note
Most users will simply define a configuration for a live trading node (as below),
and won't need to necessarily work with these lower level components directly.
:::

USDC.e (PoS)

USDC.e is a bridged version of USDC from Ethereum to the Polygon network, operating on Polygon's Proof of Stake (PoS) chain.
This enables faster, more cost-efficient transactions on Polygon while maintaining backing by USDC on Ethereum.

The contract address is [0x2791bca1f2de4661ed88a30c99a7a9449aa84174](https://polygonscan.com/address/0x2791bca1f2de4661ed88a30c99a7a9449aa84174) on the Polygon blockchain.
More information can be found in this [blog](https://polygon.technology/blog/phase-one-of-native-usdc-migration-on-polygon-pos-is-underway).

Wallets and accounts

To interact with Polymarket via NautilusTrader, you'll need a Polygon-compatible wallet (such as MetaMask).
The integration uses Externally Owned Account (EOA) signature types compatible with EIP712, meaning the wallet is directly owned by the trader/user.
This contrasts with the signature types used for Polymarket-administered wallets (such as those accessed via their user interface).

A single wallet address is supported per trader instance when using environment variables,
or multiple wallets could be configured with multiple PolymarketExecutionClient instances.

:::info
Ensure your wallet is funded with USDC.e, otherwise you will encounter the "not enough balance / allowance" API error when submitting orders.
:::

Setting Allowances for Polymarket Contracts

Before you can start trading, you need to ensure that your wallet has allowances set for Polymarket's smart contracts.
You can do this by running the provided script located at /adapters/polymarket/scripts/setallowances.py.

This script is adapted from a [gist](https://gist.github.com/poly-rodr/44313920481de58d5a3f6d1f8226bd5e) created by @poly-rodr.

:::note
You only need to run this script once per EOA wallet that you intend to use for trading on Polymarket.
:::

This script automates the process of approving the necessary allowances for the Polymarket contracts.
It sets approvals for the USDC token and Conditional Token Framework (CTF) contract to allow the

Polymarket CLOB Exchange to interact with your funds.

Before running the script, ensure the following prerequisites are met:

- Install the web3 Python package: pip install --upgrade web3==5.28
- Have a Polygon-compatible wallet funded with some MATIC (used for gas fees).
- Set the following environment variables in your shell:
  - POLYGONPRIVATEKEY: Your private key for the Polygon-compatible wallet.
  - POLYGONPUBLICKEY: Your public key for the Polygon-compatible wallet.

Once you have these in place, the script will:

- Approve the maximum possible amount of USDC (using the MAXINT value) for the Polymarket USDC token contract.
- Set the approval for the CTF contract, allowing it to interact with your account for trading purposes.

:::note
You can also adjust the approval amount in the script instead of using MAXINT,
with the amount specified in fractional units of USDC.e, though this has not been tested.
:::

Ensure that your private key and public key are correctly stored in the environment variables before running the script.
Here's an example of how to set the variables in your terminal session:

Run the script using:

Script Breakdown

The script performs the following actions:

- Connects to the Polygon network via an RPC URL (<https://polygon-rpc.com/>).
- Signs and sends a transaction to approve the maximum USDC allowance for Polymarket contracts.
- Sets approval for the CTF contract to manage Conditional Tokens on your behalf.
- Repeats the approval process for specific addresses like the Polymarket CLOB Exchange and Neg Risk Adapter.

This allows Polymarket to interact with your funds when executing trades and ensures smooth integration with the CLOB Exchange.

API keys

To trade with Polymarket using an EOA wallet, follow these steps to generate your API keys:

1. Ensure the following environment variables are set:

- POLYMARKETPK: Your private key for signing transactions.
- POLYMARKETFUNDER: The wallet address (public key) on the Polygon network used for funding trades on Polymarket.

1. Run the script using:

The script will generate and print API credentials, which you should save to the following environment variables:

- POLYMARKETAPIKEY
- POLYMARKETAPISECRET
- POLYMARKETPASSPHRASE

These can then be used for Polymarket client configurations:

- PolymarketDataClientConfig
- PolymarketExecClientConfig

Configuration

When setting up NautilusTrader to work with Polymarket, it's crucial to properly configure the necessary parameters, particularly the private key.

Key parameters

- privatekey: This is the private key for your external EOA wallet (not the Polymarket wallet accessed through their GUI). This private key allows the system to sign and send transactions on behalf of the external account interacting with Polymarket. If not explicitly provided in the configuration, it will automatically source the POLYMARKETPK environment variable.
- funder: This refers to the USDC.e wallet address used for funding trades. If not provided, will source the POLYMARKETFUNDER environment variable.
- API credentials: You will need to provide the following API credentials to interact with the Polymarket CLOB:
  - apikey: If not provided, will source the POLYMARKETAPIKEY environment variable.
  - apisecret: If not provided, will source the POLYMARKETAPISECRET environment variable.
  - passphrase: If not provided, will source the POLYMARKETPASSPHRASE environment variable.

:::tip
We recommend using environment variables to manage your credentials.
:::

Capability Matrix

Polymarket operates as a prediction market with limited order complexity compared to traditional exchanges.

Order Types

Order Type	Binary Options	Notes
MARKET		Executed as marketable limit order.
LIMIT		
STOPMARKET	-	Not supported.
STOPLIMIT	-	Not supported.
MARKETIFTOUCHED	-	Not supported.
LIMITIFTOUCHED	-	Not supported.
TRAILINGSTOPMARKET	-	Not supported.

Execution Instructions

Instruction	Binary Options	Notes
postonly	-	Not supported.
reduceonly	-	Not supported.

Time-in-Force Options

Time-in-Force	Binary Options	Notes
GTC		Good Till Canceled.
GTD		Good Till Date.
FOK		Fill or Kill.
IOC		Immediate or Cancel (maps to FAK).

Advanced Order Features

Feature	Binary Options	Notes
Order Modification	-	Cancellation functionality only.
Bracket/OCO Orders	-	Not supported.
Iceberg Orders	-	Not supported.

Configuration Options

The following execution client configuration options are available:

Option	Default	Description
signaturetype	0	Polymarket signature type (EOA).
funder	None	Wallet address for funding USDC transactions.
generateorderhistoryfromtrades	False	Experimental feature to generate order reports from trade history (not recommended).
lograwwsmessages	False	If True, logs raw WebSocket messages (performance penalty from pretty JSON formatting).

Trades

Trades on Polymarket can have the following statuses:

- MATCHED: Trade has been matched and sent to the executor service by the operator, the executor service submits the trade as a transaction to the Exchange contract.
- MINED: Trade is observed to be mined into the chain, and no finality threshold is established.
- CONFIRMED: Trade has achieved strong probabilistic finality and was successful.
- RETRYING: Trade transaction has failed (revert or reorg) and is being retried/resubmitted by the operator.
- FAILED: Trade has failed and is not being retried.

Once a trade is initially matched, subsequent trade status updates will be received via the WebSocket.
NautilusTrader records the initial trade details in the info field of the OrderFilled event,
with additional trade events stored in the cache as JSON under a custom key to retain this information.

Reconciliation

The Polymarket API returns either all active (open) orders or specific orders when queried by the Polymarket order ID (venueorderid). The execution reconciliation procedure for Polymarkert is as follows:

- Generate order reports for all instruments with active (open) orders, as reported by Polymarket.
- Generate position reports from contract balances reported by Polymarket, per instruments available in the cache.
- Compare these reports with Nautilus execution state.
- Generate missing orders to bring Nautilus execution state in line with positions reported by Polymarket.

Note: Polymarket does not directly provide data for orders which are no longer active.

:::warning
An optional execution client configuration, generateorderhistoryfromtrades, is currently under development.
It is not recommended for production use at this time.
:::

WebSockets

The PolymarketWebSocketClient is built on top of the high-performance Nautilus WebSocketClient base class, written in Rust.

Data

The main data WebSocket handles all market channel subscriptions received during the initial connection sequence, up to wsconnectiondelaysecs. For any additional subscriptions, a new PolymarketWebSocketClient is created for each new instrument (asset).

Execution

The main execution WebSocket manages all user channel subscriptions based on the Polymarket instruments available in the cache during the initial connection sequence. When trading commands are issued for additional instruments, a separate PolymarketWebSocketClient is created for each new instrument (asset).

:::note
Polymarket does not support unsubscribing from channel streams once subscribed.
:::

Limitations and considerations

The following limitations and considerations are currently known:

- Order signing via the Polymarket Python client is slow, taking around one second.
- Post-only orders are not supported.
- Reduce-only orders are not supported.

integrations/tardis.md

Tardis

Tardis provides granular data for cryptocurrency markets including tick-by-tick order book snapshots & updates,
trades, open interest, funding rates, options chains and liquidations data for leading crypto exchanges.

NautilusTrader provides an integration with the Tardis API and data formats, enabling seamless access.
The capabilities of this adapter include:

- TardisCSVDataLoader: Reads Tardis-format CSV files and converts them into Nautilus data.
- TardisMachineClient: Supports live streaming and historical replay of data from the Tardis Machine WebSocket server - converting messages into Nautilus data.
- TardisHttpClient: Requests instrument definition metadata from the Tardis HTTP API, parsing it into Nautilus instrument definitions.
- TardisDataClient: Provides a live data client for subscribing to data streams from a Tardis Machine WebSocket server.
- TardisInstrumentProvider: Provides instrument definitions from Tardis through the HTTP instrument metadata API.
- Data pipeline functions: Enables replay of historical data from Tardis Machine and writes it to the Nautilus Parquet format, including direct catalog integration for streamlined data management (see below).

:::info
A Tardis API key is required for the adapter to operate correctly. See also [environment variables](#environment-variables).
:::

Overview

This adapter is implemented in Rust, with optional Python bindings for ease of use in Python-based workflows.
It does not require any external Tardis client library dependencies.

:::info
There is no need for additional installation steps for tardis.
The core components of the adapter are compiled as static libraries and automatically linked during the build process.
:::

Tardis documentation

Tardis provides extensive user [documentation](https://docs.tardis.dev/).
We recommend also referring to the Tardis documentation in conjunction with this NautilusTrader integration guide.

Supported formats

Tardis provides normalized market data-a unified format consistent across all supported exchanges.
This normalization is highly valuable because it allows a single parser to handle data from any [Tardis-supported exchange](#venues), reducing development time and complexity.
As a result, NautilusTrader will not support exchange-native market data formats, as it would be inefficient to implement separate parsers for each exchange at this stage.

The following normalized Tardis formats are supported by NautilusTrader:

Tardis format	Nautilus data type
[bookchange](https://docs.tardis.dev/api/tardis-machine#bookchange)	OrderBookDelta
[booksnapshot](https://docs.tardis.dev/api/tardis-machine#booksnapshot-numberoflevels--snapshotinterval-timeunit)	OrderBookDepth10
[quote](https://docs.tardis.dev/api/tardis-machine#booksnapshot-numberoflevels--snapshotinterval-timeunit)	QuoteTick
[quote10s](https://docs.tardis.dev/api/tardis-machine#booksnapshot-numberoflevels--snapshotinterval-timeunit)	QuoteTick
[trade](https://docs.tardis.dev/api/tardis-machine#trade)	Trade
[tradebar](https://docs.tardis.dev/api/tardis-machine#tradebar-aggregationinterval-suffix)	Bar
[instrument](https://docs.tardis.dev/api/instruments-metadata-api)	CurrencyPair, CryptoFuture, CryptoPerpetual, OptionContract
[derivativeticker](https://docs.tardis.dev/api/tardis-machine#derivativeticker)	Not yet supported
[disconnect](https://docs.tardis.dev/api/tardis-machine#disconnect)	Not applicable

Notes:

- [quote](https://docs.tardis.dev/api/tardis-machine#booksnapshot-numberoflevels--snapshotinterval-timeunit) is an alias for [booksnapshot10ms](https://docs.tardis.dev/api/tardis-machine#booksnapshot-numberoflevels--snapshotinterval-timeunit).
- [quote10s](https://docs.tardis.dev/api/tardis-machine#booksnapshot-numberoflevels--snapshotinterval-timeunit) is an alias for [booksnapshot110s](https://docs.tardis.dev/api/tardis-machine#booksnapshot-numberoflevels--snapshotinterval-timeunit).
- Both quote, quote\10s, and one-level snapshots are parsed as QuoteTick.

:::info
See also the Tardis [normalized market data APIs](https://docs.tardis.dev/api/tardis-machine#normalized-market-data-apis).
:::

Bars

The adapter will automatically convert [Tardis trade bar interval and suffix](https://docs.tardis.dev/api/tardis-machine#tradebar-aggregationinterval-suffix) to Nautilus BarTypes.
This includes the following:

Tardis suffix	Nautilus bar aggregation

[ms](https://docs.tardis.dev/api/tardis-machine#tradebar-aggregationinterval-suffix) - milliseconds	MILLISECOND
[s](https://docs.tardis.dev/api/tardis-machine#tradebar-aggregationinterval-suffix) - seconds	SECOND
[m](https://docs.tardis.dev/api/tardis-machine#tradebar-aggregationinterval-suffix) - minutes	MINUTE
[ticks](https://docs.tardis.dev/api/tardis-machine#tradebar-aggregationinterval-suffix) - number of ticks	TICK
[vol](https://docs.tardis.dev/api/tardis-machine#tradebar-aggregationinterval-suffix) - volume size	VOLUME

Symbology and normalization

The Tardis integration ensures seamless compatibility with NautilusTrader's crypto exchange adapters by consistently normalizing symbols. Typically, NautilusTrader uses the native exchange naming conventions
provided by Tardis. However, for certain exchanges, raw symbols are adjusted to adhere to the Nautilus symbology normalization, as outlined below:

Common Rules

- All symbols are converted to uppercase.
- Market type suffixes are appended with a hyphen for some exchanges (see [exchange-specific normalizations](#exchange-specific-normalizations)).
- Original exchange symbols are preserved in the Nautilus instrument definitions rawsymbol field.

Exchange-Specific Normalizations

- Binance: Nautilus appends the suffix -PERP to all perpetual symbols.
- Bybit: Nautilus uses specific product category suffixes, including -SPOT, -LINEAR, -INVERSE, -OPTION.
- dYdX: Nautilus appends the suffix -PERP to all perpetual symbols.
- Gate.io: Nautilus appends the suffix -PERP to all perpetual symbols.

For detailed symbology documentation per exchange:

- [Binance symbology](./binance.md#symbology)
- [Bybit symbology](./bybit.md#symbology)
- [dYdX symbology](./dydx.md#symbology)

Venues

Some exchanges on Tardis are partitioned into multiple venues.
The table below outlines the mappings between Nautilus venues and corresponding Tardis exchanges, as well as the exchanges that Tardis supports:

Nautilus venue	Tardis exchange(s)
ASCENDEX	ascendex
BINANCE	binance, binance-dex, binance-futures, binance-jersey, binance-options, binance-us
BINANCEDELIVERY	binance-delivery (COIN-margined contracts)
BINANCEUS	binance-us
BITFINEX	bitfinex, bitfinex-derivatives
BITFLYER	bitflyer
BITMEX	bitmex
BITNOMIAL	bitnomial

BITSTAMP	bitstamp
BLOCKCHAINCOM	blockchain-com
BYBIT	bybit, bybit-options, bybit-spot
COINBASE	coinbase
COINFLEX	coinflex (for historical research)
CRYPTOCOM	crypto-com
CRYPTOFACILITIES	cryptofacilities
DELTA	delta
DERIBIT	deribit
DYDX	dydx
FTX	ftx (historical research)
FTXUS	ftx-us (historical research)
GATEIO	gate-io, gate-io-futures
GEMINI	gemini
HITBTC	hitbtc
HUOBI	huobi, huobi-dm, huobi-dm-linear-swap, huobi-dm-options
HUOBIDELIVERY	huobi-dm-swap
KRAKEN	kraken
KUCOIN	kucoin
MANGO	mango
OKCOIN	okcoin
OKEX	okex, okex-futures, okex-options, okex-swap
PHEMEX	phemex
POLONIEX	poloniex
SERUM	serum (historical research)
STARATLAS	staratlas
UPBIT	upbit
WOOX	woo-x

Environment variables

The following environment variables are used by Tardis and NautilusTrader.

- TMAPIKEY: API key for the Tardis Machine.
- TARDISAPIKEY: API key for NautilusTrader Tardis clients.
- TARDISWSURL (optional): WebSocket URL for the TardisMachineClient in NautilusTrader.
- TARDISBASEURL (optional): Base URL for the TardisHttpClient in NautilusTrader.
- NAUTILUSCATALOGPATH (optional): Root directory for writing replay data in the Nautilus catalog.

Running Tardis Machine historical replays

The [Tardis Machine Server](https://docs.tardis.dev/api/tardis-machine) is a locally runnable server with built-in data caching, providing both tick-level historical and consolidated real-time cryptocurrency market data through HTTP and WebSocket APIs.

You can perform complete Tardis Machine WebSocket replays of historical data and output the results in Nautilus Parquet format, using either Python or Rust. Since the function is implemented in Rust, performance is consistent whether run from Python or Rust, letting you choose based on your preferred workflow.

The end-to-end runtardismachinereplay data pipeline function utilizes a specified [configuration](#configuration) to execute the following steps:

- Connect to the Tardis Machine server.
- Request and parse all necessary instrument definitions from the [Tardis instruments metadata](https://docs.tardis.dev/api/instruments-metadata-api) HTTP API.

- Stream all requested instruments and data types for the specified time ranges from the Tardis Machine server.
- For each instrument, data type and date (UTC), generate a .parquet file in the Nautilus format.
- Disconnect from the Tardis Marchine server, and terminate the program.

:::note
You can request data for the first day of each month without an API key. For all other dates, a Tardis Machine API key is required.
:::

This process is optimized for direct output to a Nautilus Parquet data catalog.
Ensure that the NAUTILUSCATALOGPATH environment variable is set to the root /catalog/ directory.
Parquet files will then be organized under /catalog/data/ in the expected subdirectories corresponding to data type and instrument.

If no outputpath is specified in the configuration file and the NAUTILUSCATALOGPATH environment variable is unset, the system will default to the current working directory.

Procedure

First, ensure the tardis-machine docker container is running. Use the following command:

This command starts the tardis-machine server without a persistent local cache, which may affect performance.
For improved performance, consider running the server with a persistent volume. Refer to the [Tardis Docker documentation](https://docs.tardis.dev/api/tardis-machine#docker) for details.

Configuration

Next, ensure you have a configuration JSON file available.

Configuration JSON format

Field	Type	Description	Default
tardiswsurl	string (optional)	The Tardis Machine WebSocket URL.	If null then will use the TARDISWSURL env var.
normalizesymbols	bool (optional)	If Nautilus [symbol normalization](#symbology-and-normalization) should be applied.	If null then will default to true.
outputpath	string (optional)	The output directory path to write Nautilus Parquet data to.	If null then will use the NAUTILUSCATALOGPATH env var, otherwise the current working directory.
options	JSON[]	An array of [ReplayNormalizedRequestOptions](https://docs.tardis.dev/api/tardis-machine#replay-normalized-options) objects.	

An example configuration file, exampleconfig.json, is available [here](https://github.com/nautechsystems/nautilustrader/blob/develop/crates/adapters/tardis/bin/exampleconfig.json):

Python Replays

To run a replay in Python, create a script similar to the following:

Rust Replays

To run a replay in Rust, create a binary similar to the following:

Make sure to enable Rust logging by exporting the following environment variable:

A working example binary can be found [here](https://github.com/nautechsystems/nautilustrader/blob/develop/crates/adapters/tardis/bin/examplereplay.rs).

This can also be run using cargo:

Loading Tardis CSV data

Tardis-format CSV data can be loaded using either Python or Rust. The loader reads the CSV text data from disk and parses it into Nautilus data. Since the loader is implemented in Rust, performance remains
consistent regardless of whether you run it from Python or Rust, allowing you to choose based on your preferred workflow.

You can also optionally specify a limit parameter for the load functions/methods to control the maximum number of rows loaded.

:::note
Loading mixed-instrument CSV files is challenging due to precision requirements and is not recommended. Use single-instrument CSV files instead (see below).
:::

Loading CSV Data in Python

You can load Tardis-format CSV data in Python using the TardisCSVDataLoader.
When loading data, you can optionally specify the instrument ID but must specify both the price precision, and size precision.
Providing the instrument ID improves loading performance, while specifying the precisions is required, as they cannot be inferred from the text data alone.

To load the data, create a script similar to the following:

Loading CSV Data in Rust

You can load Tardis-format CSV data in Rust using the loading functions found [here](https://github.com/nautechsystems/nautilustrader/blob/develop/crates/adapters/tardis/src/csv/mod.rs).
When loading data, you can optionally specify the instrument ID but must specify both the price precision and size precision.
Providing the instrument ID improves loading performance, while specifying the precisions is required, as they cannot be inferred from the text data alone.

For a complete example, see the [example binary here](https://github.com/nautechsystems/nautilustrader/blob/develop/crates/adapters/tardis/bin/examplecsv.rs).

To load the data, you can use code similar to the following:

Requesting instrument definitions

You can request instrument definitions in both Python and Rust using the TardisHttpClient.
This client interacts with the [Tardis instruments metadata API](https://docs.tardis.dev/api/instruments-metadata-api) to request and parse instrument metadata into Nautilus instruments.

The TardisHttpClient constructor accepts optional parameters for apikey, baseurl, and timeoutsecs

(default is 60 seconds).

The client provides methods to retrieve either a specific instrument, or all instruments available on a particular exchange.
Ensure that you use Tardis's lower-kebab casing convention when referring to a [Tardis-supported exchange](https://api.tardis.dev/v1/exchanges).

:::note
A Tardis API key is required to access the instruments metadata API.
:::

Requesting Instruments in Python

To request instrument definitions in Python, create a script similar to the following:

Requesting Instruments in Rust

To request instrument definitions in Rust, use code similar to the following.
For a complete example, see the [example binary here](https://github.com/nautechsystems/nautilustrader/blob/develop/crates/adapters/tardis/bin/example http.rs).

Instrument provider

The TardisInstrumentProvider requests and parses instrument definitions from Tardis through the HTTP instrument metadata API.
Since there are multiple [Tardis-supported exchanges](#venues), when loading all instruments,
you must filter for the desired venues using an InstrumentProviderConfig:

You can also load specific instrument definitions in the usual way:

:::note
Instruments must be available in the cache for all subscriptions.
For simplicity, it's recommended to load all instruments for the venues you intend to subscribe to.
:::

Live data client

The TardisDataClient enables integration of a Tardis Machine with a running NautilusTrader system.
It supports subscriptions to the following data types:

- OrderBookDelta (L2 granularity from Tardis, includes all changes or full-depth snapshots)
- OrderBookDepth10 (L2 granularity from Tardis, provides snapshots up to 10 levels)
- QuoteTick
- TradeTick
- Bar (trade bars with [Tardis-supported bar aggregations](#bars))

Data WebSockets

The main TardisMachineClient data WebSocket manages all stream subscriptions received during the initial connection phase,
up to the duration specified by wsconnectiondelaysecs. For any additional subscriptions made after this period, a new TardisMachineClient is created. This approach optimizes performance by allowing the main WebSocket to handle potentially hundreds of subscriptions in a single stream if they are provided at startup.

When an initial subscription delay is set with wsconnectiondelaysecs, unsubscribing from any of these streams will not actually remove the subscription from the Tardis Machine stream, as selective unsubscription is not supported by Tardis. However, the component will still unsubscribe from message

bus publishing as expected.

All subscriptions made after any initial delay will behave normally, fully unsubscribing from the Tardis Machine stream when requested.

:::tip
If you anticipate frequent subscription and unsubscription of data, it is recommended to set wsconnectiondelaysecs to zero. This will create a new client for each initial subscription, allowing them to be later closed individually upon unsubscription.
:::

Limitations and considerations

The following limitations and considerations are currently known:

- Historical data requests are not supported, as each would require a minimum one-day replay from the Tardis Machine, potentially with a filter. This approach is neither practical nor efficient.

tutorials/index.md

Tutorials

The tutorials provide a guided learning experience with a series of comprehensive step-by-step walkthroughs.
Each tutorial targets specific features or workflows, enabling hands-on learning.
From basic tasks to more advanced operations, these tutorials cater to a wide range of skill levels.

:::info
Each tutorial is generated from a Jupyter notebook located in the docs [tutorials directory](https://github.com/nautechsystems/nautilustrader/tree/develop/docs/tutorials). These notebooks serve as valuable learning aids and let you execute the code interactively.
:::

:::tip

- Make sure you are using the tutorial docs that match your NautilusTrader version:
- Latest: These docs are built from the HEAD of the master branch and work with the latest stable release. See <https://nautilustrader.io/docs/latest/tutorials/>.
- Nightly: These docs are built from the HEAD of the nightly branch and work with bleeding-edge and experimental features. See <https://nautilustrader.io/docs/nightly/tutorials/>.

:::

Running in docker

Alternatively, a self-contained dockerized Jupyter notebook server is available for download, which does not require any setup or
installation. This is the fastest way to get up and running to try out NautilusTrader. Bear in mind that any data will be
deleted when the container is deleted.

- To get started, install docker:
  - Go to [docker.com](https://docs.docker.com/get-docker/) and follow the instructions
- From a terminal, download the latest image
  - docker pull ghcr.io/nautechsystems/jupyterlab:nightly --platform linux/amd64
- Run the docker container, exposing the Jupyter port:
  - docker run -p 8888:8888 ghcr.io/nautechsystems/jupyterlab:nightly
- When the container starts, a URL with an access token will be printed in the terminal. Copy that URL and open it in your browser, for example:
  - <http://localhost:8888>

:::info
NautilusTrader currently exceeds the rate limit for Jupyter notebook logging (stdout output),
this is why loglevel in the examples is set to ERROR. If you lower this level to see
more logging then the notebook will hang during cell execution. A fix is currently
being investigated which involves either raising the configured rate limits for
Jupyter, or throttling the log flushing from Nautilus.

- <https://github.com/jupyterlab/jupyterlab/issues/12845>
- <https://github.com/deshaw/jupyterlab-limit-output>

:::